GestureTalk – Real-Time Sign Language Recognition and Translation System

A Project Report
Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (DATA SCIENCE)

By

Sharonmary Lazer Baptist Anthony
[redacted]

Under the guidance of
Dr. Priti Pathak
Co-ordinator, Department of Data Science



SCHOOL OF COMPUTING & TECHNOLOGY
PARLE TILAK VIDYALAYA ASSOCIATION'S
MULUND COLLEGE OF COMMERCE (AUTONOMOUS)
(Affiliated to University of Mumbai)
MULUND – 400 080
MAHARASHTRA
2025-26

ACKNOWLEDGEMENT

I like to extend my gratitude to the Principal, Dr. Sonali Pednekar, Vice-Principal, Dr. Shivaji Pawar and all staff of Mulund College of Commerce (Autonomous) for providing us moral support, the required infrastructure and resource to enable the completion and enrichment of my project.

I also take this opportunity to thank the School of Computing & Technology Coordinator, Dr. Reena Nagda, Program Coordinator, Dr. Priti Pathak and all the faculty of School of Computing & Technology for giving us the most needed guidance and continuous encouragement throughout the duration of the Program.

I wish to extend my deepest gratitude and special thanks to my project guide Dr. Priti Pathak, for the guidance and the inspiration to complete this project on time. I perceive the skills and knowledge which I have gained throughout my project work as a very valuable component in my future career development

I am also extremely grateful to the University of Mumbai for having prescribed this project work to me as a part of the academic requirement.

Finally, I thank all my friends and family members who have directly or indirectly helped me in completing my project.

DECLARATION

1.	Student Name	Sharonmary Lazer Baptist Anthony
2.	Roll No	[redacted]
3.	Class	TY. BSc. Data Science
4.	Department	Data Science
5.	ABC ID	[redacted]

I hereby declare that the project entitled "**GestureTalk – Real-Time Sign Language Recognition and Translation System**", is my own work, based on my personal study and/or research and that I have acknowledged all material and sources used in its preparation, whether they be books, articles, reports, lecture notes, or any other kind of document, electronic or personal communication.

I certify that I have not copied, in part or whole, nor otherwise plagiarised the work of any other person.

The project is done in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE (DATA SCIENCE)** to be submitted as Semester VI project as part of our curriculum.

Student's Signature

ABSTRACT

Communication is a fundamental human necessity; however, for individuals who are deaf or hard of hearing, interacting with the broader community can be extremely challenging due to the lack of widespread understanding of sign languages. In India, millions of people rely on Indian Sign Language (ISL) as their primary mode of communication. Unfortunately, most people are unfamiliar with ISL, creating a communication barrier between the deaf community and the hearing population. This gap becomes even more significant in situations such as healthcare consultations, travel, emergency services, education, and workplace interactions where effective communication is essential. Furthermore, sign languages differ across regions, meaning that Indian Sign Language is not the same as American Sign Language (ASL) or other international sign systems, making cross-lingual communication more difficult.

To address this challenge, this project introduces GestureTalk – a Real-Time Indian Sign Language Recognition and Translation System. GestureTalk is an intelligent system designed to recognize and interpret Indian Sign Language gestures using computer vision and deep learning techniques. The system captures real-time video input from a webcam, identifies hand landmarks using the MediaPipe framework, and processes sequential gesture data using a Long Short-Term Memory (LSTM) neural network trained on a custom dataset of ISL gestures. MediaPipe extracts 21 three-dimensional landmarks per hand, producing normalized skeletal feature representations used as input for the gesture recognition model.

Once a gesture is recognized, the system converts the detected sign into readable text and subsequently translates it into multiple languages using the Google Translate API. The translated text can then be converted into speech using a text-to-speech engine, enabling seamless verbal communication between sign language users and non-signers.

The system supports real-time gesture recognition, sentence formation, multilingual translation, and speech output, creating an interactive communication bridge between sign language users and the hearing population. The project utilizes technologies such as Python, TensorFlow, MediaPipe, OpenCV, Flask, HTML, CSS, JavaScript, Google Translate API, and gTTS to build an integrated platform capable of both machine learning inference and web-based interaction.

A custom dataset containing 28 gesture classes and approximately 940 gesture sequences was collected using a webcam and processed through landmark normalization techniques to improve recognition accuracy. GestureTalk aims to empower the Indian deaf community by reducing communication barriers not only within India but also across different linguistic environments worldwide. By enabling automatic translation of sign language into multiple spoken languages, the system promotes inclusivity, accessibility, and independence for sign language users.

The project demonstrates how artificial intelligence and computer vision technologies can be leveraged to develop accessible assistive systems that improve communication, inclusivity, and social integration for individuals with hearing impairments.

TABLE OF CONTENTS

CHAPTER 1 : INTRODUCTION 1
1.1 Background of Study 2
1.2 Problem Statement 9
1.3 Research gap 10
1.4 Objectives of the Study 14
1.5 Scope of project 15
1.6 Significance of the study 17
1.7 Structure of the Report 18
CHAPTER 2 : REVIEW OF LITERATURE 21
CHAPTER 3 : RESEARCH METHODOLOGY 35
3.1 Research Design 35
3.2 Hypothesis Formulation 36
3.3 Objectives Aligned with Hypothesis 37
3.5 Algorithms / Models Applied 43
3.6 Tools and Technologies 48
3.6.1 Programming Languages 48
3.7 Justification of Methodology 51
CHAPTER 4 : EXPLORATORY DATA ANALYSIS (EDA) 55
4.1 Dataset Overview 59
4.2 Data Preprocessing and Cleaning 68
4.3 Feature Representation and Feature Extraction 70
4.4 Key Insights from Exploratory Data Analysis 73
4.5 Dataset Summary and Chapter Conclusions 78
CHAPTER 5 : IMPLEMENTATION 79
5.1 Workflow of the Project 79
5.2 System Architecture and Model Pipeline 81
5.3 Key Implementation Code Snippets 91
5.4 Model Training 96
5.5 Model Optimization 98
5.6 Tools and Frameworks used 100
5.7 Practical Implications 105
CHAPTER 6 : RESULTS 109
6.1 Evaluation Metrics 109
6.2 Experimental Results and Interpretation 112

6.3 Comparison with Existing Systems 119
6.3.5 Functional Comparison Summary 122
6.4 Summary of Results 123
CHAPTER 7 : SUMMARY & CONCLUSION 125
7.1 Summary of Work Done 125
7.2 Key Findings and Their Significance 126
7.3 Limitations of the Study 127
7.4 Future Scope and Recommendations 128
7.5 Chapter Summary 131
CHAPTER 8 : REFERENCES 132
CHAPTER 9 : APPENDICES 136

CHAPTER 1 : INTRODUCTION

Human communication is essential for social interaction, education, healthcare access, and participation in economic activities. However, for individuals who are deaf or hard of hearing, communication with the hearing population can be extremely challenging due to the lack of widespread understanding of sign languages. Sign language is the primary form of communication used by the deaf community, but most people are unfamiliar with it, creating a significant communication barrier in everyday interactions.

In India, the deaf community primarily uses Indian Sign Language (ISL), which is a complete visual language consisting of hand gestures, facial expressions, and body movements. Unlike spoken languages, ISL relies heavily on visual cues and motion-based communication. Despite its importance, ISL is not widely understood by the general population, and trained interpreters are often unavailable in many situations such as hospitals, government offices, schools, workplaces, and public service environments.

This communication gap becomes even more challenging when deaf individuals need to interact with people who speak different languages. While spoken languages can often be translated easily through technology, sign languages differ significantly across regions. For example, Indian Sign Language (ISL) is structurally different from American Sign Language (ASL) and other international sign systems. As a result, deaf individuals may face double barriers: the inability of others to understand sign language and the lack of tools that translate sign language into multiple spoken languages.

Recent advancements in Artificial Intelligence (AI), Computer Vision, and Deep Learning have enabled the development of intelligent systems capable of interpreting human gestures from video data. Technologies such as hand landmark detection, gesture recognition, and sequence modeling allow computers to analyze human gestures captured through cameras and interpret them as meaningful language inputs. These technologies can be combined with translation systems and speech synthesis tools to create intelligent communication systems that bridge the gap between sign language users and non-sign language users.

In response to this need, this project introduces GestureTalk – a Real-Time Indian Sign Language Recognition and Translation System. GestureTalk is designed to recognize hand gestures corresponding to Indian Sign Language using computer vision and deep learning models. The system captures live video through a webcam, extracts hand landmark coordinates using MediaPipe, and processes gesture sequences using a Long Short-Term Memory (LSTM) neural network trained on a custom dataset of ISL gestures. Once a gesture is recognized, the system converts it into text, translates it into multiple languages using a translation API, and generates speech output using a text-to-speech engine.

The overall working concept of the GestureTalk system is illustrated in Figure 1.1:

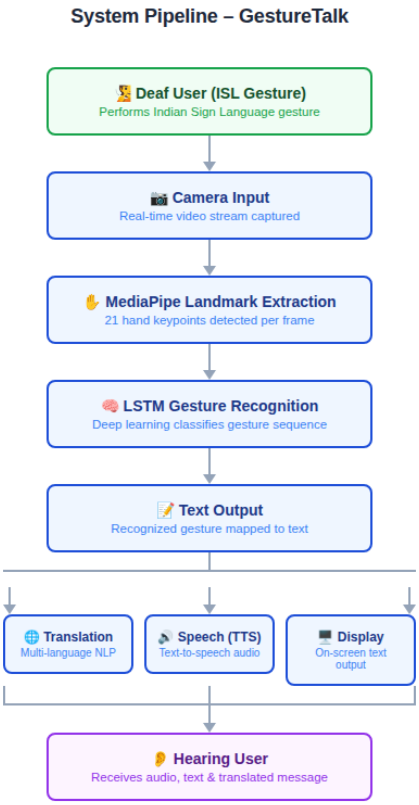


Figure 1.1: GestureTalk System Pipeline

The system supports 28 different gesture classes, including commonly used ISL signs, and can translate the recognized text into 20 different languages. Additionally, GestureTalk provides a web-based interface that allows users to interact with the system in real time, construct sentences from detected gestures, and receive translated audio output. By combining gesture recognition with language translation and speech synthesis, the system enables effective communication between deaf individuals and people who do not understand sign language.

The primary aim of this project is to improve accessibility and inclusivity for the Indian deaf community by providing a practical technological solution that removes communication barriers. GestureTalk demonstrates how modern AI technologies can be used to create assistive tools that empower individuals with hearing impairments, enhance cross-language communication, and promote social integration.

1.1 Background of Study

Communication is not merely a tool; it is the very fabric of human civilization. Through language, human beings have built societies, transferred knowledge across generations, forged relationships, resolved conflicts, and asserted their identities. For the vast majority of the world's population, spoken language serves as the default medium of this communication. However, for an estimated 430 million people worldwide who experience disabling hearing loss, according to the World Health Organization's 2021 report, the spoken word is not an accessible channel. For this community, sign language

is not an alternative or a substitute, it is their primary, native, and most natural form of expression.

Sign languages are complete linguistic systems. They possess their own phonology, morphology, syntax, semantics, and pragmatics, fully independent of the spoken languages used in the same geographic region. Indian Sign Language (ISL), for instance, is not a signed version of Hindi or English it is an entirely separate language with its own grammatical rules, vocabulary, and regional dialects. In India, where approximately 18 million people are deaf or hard of hearing according to the 2011 Census of India, ISL is the primary communication medium for a vast and linguistically rich community that has long existed on the margins of institutional recognition.

Despite the linguistic richness and cultural significance of sign languages, the hearing world has historically lacked the tools and training to understand them. This asymmetry creates a profound communication barrier that manifests in nearly every domain of daily life. A deaf patient visiting a hospital cannot explain their symptoms to a doctor who does not know ISL. A deaf student in a mainstream classroom cannot follow a lecture delivered entirely in spoken language. A deaf individual seeking employment faces interviews conducted without accessible interpretation. In each of these scenarios, the absence of a reliable, real-time communication bridge results in exclusion, misunderstanding, and inequality.

The traditional solution to this barrier has been the deployment of human sign language interpreters. Trained interpreters play an irreplaceable role in high-stakes communication contexts such as legal proceedings, medical consultations, and educational settings. However, the supply of qualified ISL interpreters in India is critically insufficient relative to demand. Training an interpreter requires years of dedicated study, and certification infrastructure remains underdeveloped. The All-India Federation of the Deaf (AIFD) has noted that the ratio of qualified interpreters to deaf individuals in India falls far short of what is needed for comprehensive service delivery. Even in countries with more developed accessibility infrastructure, human interpretation cannot be available on demand for spontaneous everyday interaction.

It is precisely this gap that motivates the development of GestureTalk. The convergence of three powerful technological trends in the 2010s and 2020s the maturation of deep learning, the democratization of computer vision through lightweight frameworks, and the widespread availability of high-quality webcam hardware has created a uniquely favorable environment for building practical, deployable automated sign language recognition systems. GestureTalk leverages each of these trends to deliver a real-time, web-based ISL recognition and translation system that requires no specialized hardware beyond a standard computer and webcam, making it accessible to a broad population of users across diverse socioeconomic backgrounds.

1.1.1 Evolution of Human–Computer Interaction Technologies

The history of human-computer interaction (HCI) is a story of progressively expanding the bandwidth and naturalness of communication between humans and machines. In the earliest era of computing, interaction was mediated through punch cards and command-line text interfaces a modality accessible only to technically trained operators. The introduction of the graphical user interface (GUI) in the 1980s, pioneered by Xerox PARC and popularized by Apple and Microsoft, democratized computing by replacing memorized commands with visual metaphors and pointing devices. The touchscreen revolution of the 2000s further collapsed the distance between

human intent and digital action, enabling billions of users to interact with technology using nothing more than a finger.

Each of these transitions represented a fundamental shift in the assumption about what constitutes a valid input signal. The current frontier of HCI research is extending this paradigm even further toward systems that can interpret human gestures, body language, gaze, and expression as meaningful input. This shift is not merely a technical curiosity; it has profound implications for accessibility. For individuals who cannot use traditional keyboard, mouse, or touchscreen interfaces due to motor impairments, neurological conditions, or communication differences such as deafness, natural gesture-based interfaces represent not an enhancement but an essential requirement.

Technologies such as Microsoft's Kinect sensor (2010), Intel's RealSense depth camera, Leap Motion's hand controller, Apple's Face ID facial recognition system, and most recently Google's MediaPipe framework have each contributed to a growing ecosystem of tools that can perceive and interpret human body movements in real time. GestureTalk is built within this ecosystem, specifically harnessing MediaPipe's hand landmark detection capabilities to extract precise skeletal representations of hand gestures from standard webcam feeds, transforming them into the structured feature vectors that feed its deep learning recognition model.

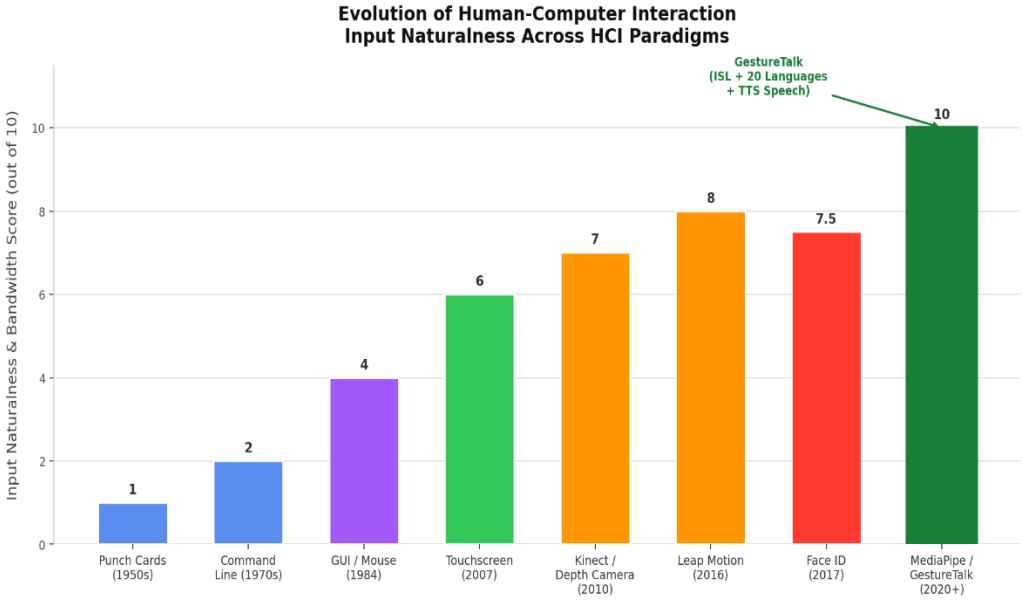


Figure 1.2: Evolution of Human-Computer Interaction

1.1.2 Sign Language as a Complete and Independent Linguistic System

A critical and frequently misunderstood aspect of sign languages is their status as fully independent linguistic systems. Across the history of deaf education and public policy, sign languages have been wrongly dismissed as primitive, incomplete, or derivative of spoken languages. This misconception has been definitively refuted by decades of linguistic research. William Stokoe's landmark 1960 analysis of American Sign Language demonstrated that ASL possesses a complete phonological structure, one based on the visual parameters of handshape, movement, location, and palm orientation rather than acoustic phonemes. Subsequent research by Ursula Bellugi, Edward Klima, and others established that all major sign languages exhibit the full range of linguistic

properties recursion, displacement, productivity, and arbitrariness that characterize human language.

Indian Sign Language shares these properties. ISL has its own distinct grammar, differing markedly from Hindi or English in its use of spatial grammar, non-manual markers (facial expressions and head movements that serve grammatical functions), and topic-comment sentence structure. ISL was formally recognized by the Indian government in 2020 as an official language of the deaf community, and the Indian Sign Language Research and Training Centre (ISLRTC) has been established under the Ministry of Social Justice and Empowerment to develop and standardize ISL resources including dictionaries, training materials, and a corpus of signed content.

GestureTalk focuses on vocabulary-level recognition of individual ISL gestures rather than full grammatical sentence interpretation. This is a deliberate and appropriate scoping decision for a prototype system. Recognizing that full ISL grammatical parsing would require modeling spatial grammar, temporal co-reference, and non-manual markers far beyond the scope of a hand-tracking LSTM system, GestureTalk focuses on the vocabulary layer recognizing 28 frequently used ISL gesture classes that cover common words and phrases applicable to everyday communication scenarios. This vocabulary-level recognition, while not full linguistic interpretation, delivers genuine communicative utility while remaining technically tractable.

1.1.3 The Technical Evolution of Gesture Recognition Systems

Automated gesture recognition has been an active area of research since the early 1990s. The field has evolved through several distinct technical generations, each building on the limitations of its predecessors.

The first generation of gesture recognition systems, prevalent through the 1990s and early 2000s, relied primarily on color-based hand segmentation, background subtraction, and rule-based classifiers. These systems worked by isolating skin-colored regions in image frames, computing simple geometric features such as aspect ratio, centroid position, and finger count, and mapping these features to predefined gesture categories through threshold-based rules. While functional in highly controlled laboratory settings, these approaches were extremely brittle in real-world conditions sensitive to changes in lighting, background complexity, skin tone variation, and hand orientation.

The second generation introduced machine learning classifiers such as Support Vector Machines (SVMs), Hidden Markov Models (HMMs), and k-Nearest Neighbour (kNN) algorithms trained on hand-crafted feature vectors. HMMs became widely used for sign language recognition in the 2000s, as they provided a principled probabilistic framework for modeling the temporal progression of gesture sequences. However, these models still required significant domain expertise to engineer appropriate features and remained sensitive to the assumptions built into their feature representations.

The third and current generation leverages deep learning architectures that learn feature representations directly from data. Convolutional Neural Networks (CNNs) transformed image-based gesture recognition by learning hierarchical spatial features without manual engineering. Recurrent Neural Networks (RNNs) and their Long Short-Term Memory (LSTM) variant addressed the temporal modeling challenge, enabling systems to recognize gestures as sequences of frames unfolding over time. GestureTalk

operates in this third generation, using MediaPipe to provide a clean, robust, and consistent skeletal representation of hand gestures, and a custom stacked LSTM network to perform temporal sequence classification over 20-frame gesture windows.

1.1.4 The Four Phases of Gesture Recognition Technology

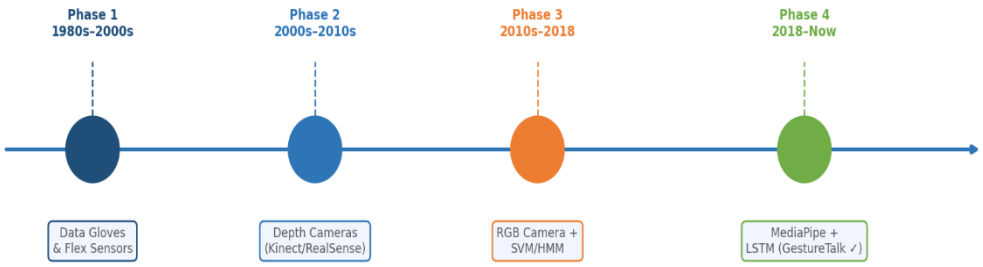


Figure 1.3: Evolution of Gesture Recognition Technology Across Four Phases

Gesture recognition technology has evolved through four distinct phases. Phase 1 (1980s–2000s) relied on data gloves equipped with flex sensors, which provided high accuracy but were expensive and impractical for everyday use. Phase 2 (2000s–2010s) introduced depth cameras like Microsoft Kinect better, but requiring specialized hardware. Phase 3 (2010s–2018) used RGB cameras with SVMs and HMMs accessible but limited by static image classification. Phase 4 (2018–present), where GestureTalk operates, combines MediaPipe's real-time hand landmark detection with LSTM deep learning achieving high accuracy on commodity hardware with just a webcam.

1.1.5 MediaPipe Hands: Architecture and Capabilities

MediaPipe Hands is an open-source, cross-platform hand landmark detection pipeline developed by Google Research and released as part of the broader MediaPipe framework in 2019. It represents a significant engineering achievement in making accurate, real-time hand tracking accessible on consumer hardware. The pipeline consists of two stages: a palm detector that identifies the bounding box of the hand in the image frame, and a hand landmark model that regresses the precise 3D positions of 21 key anatomical landmarks from the cropped hand region.

The 21 landmarks correspond to the major joint positions of the hand skeleton: the wrist (landmark 0), the four knuckle joints of each finger (metacarpophalangeal, proximal interphalangeal, and distal interphalangeal joints), and the fingertip of each finger. Together, these 21 points encode the full configuration of the hand including the spread of fingers, the degree of flexion at each joint, and the overall orientation of the hand relative to the camera. For each landmark, MediaPipe provides three coordinates: the x and y pixel positions normalized to the image frame, and a z coordinate representing relative depth.

For GestureTalk, two hands are tracked simultaneously, yielding 21 landmarks × 3 coordinates × 2 hands = 126 features per frame. These features are further normalized in the preprocessing pipeline: landmark positions are computed relative to the wrist landmark to remove global hand position from the feature space, and they are scaled by

the span of the hand to remove size variation. This normalization is critical for model generalization it ensures that a gesture made close to the camera or far from it, with a large hand or a small hand, produces a consistent feature vector that the LSTM can reliably classify.

A key technical advantage of MediaPipe-based gesture recognition over pixel-based approaches is its robustness to visual confounds. Because the model operates on skeleton coordinates rather than raw pixel values, it is inherently invariant to background clutter, lighting conditions, clothing color, and skin tone. This property significantly improves the system's generalization to real-world deployment environments outside the controlled settings where the training data was collected.

1.1.6 Long Short-Term Memory Networks for Sequential Gesture Modeling

A single frame of video is insufficient to determine the identity of most sign language gestures. Many ISL gestures involve characteristic movements a sweeping motion, a circular path, a transition from one hand configuration to another that unfold across a sequence of frames. Recognizing such gestures requires a model that can integrate information across time, retaining relevant context from earlier frames while processing later ones.

Long Short-Term Memory (LSTM) networks, introduced by Sepp Hochreiter and Jurgen Schmidhuber in 1997, are specifically designed for this purpose. LSTMs are a variant of Recurrent Neural Networks (RNNs) that address the vanishing gradient problem a training instability that prevented earlier RNNs from learning long-range temporal dependencies. LSTMs achieve this through a gating mechanism that controls the flow of information through the network: an input gate determines which new information to write into the cell state, a forget gate determines which stored information to discard, and an output gate determines which stored information to expose as the hidden state. This architecture allows LSTMs to selectively retain information over arbitrarily long sequences.

In GestureTalk, the LSTM receives a sequence of 20 consecutive frames, each represented by a 126-dimensional feature vector of normalized MediaPipe landmarks. The network processes this sequence frame by frame, updating its hidden state at each step, and produces a final classification output after the entire 20-frame sequence has been consumed. The model architecture consists of two stacked LSTM layers each with 128 units, with Batch Normalization and Dropout applied after each LSTM layer to stabilize training and prevent overfitting, followed by two Dense layers producing a 28-class softmax probability distribution over the supported gesture vocabulary.

1.1.7 The Communication Accessibility Gap in India

India presents a particularly compelling and urgent context for the development of sign language recognition technology. With an estimated 18 million deaf and hard-of-hearing individuals according to the 2011 National Census, India hosts one of the world's largest communities of sign language users. Yet the infrastructure for communication accessibility in India remains profoundly underdeveloped relative to this scale of need.

The Rights of Persons with Disabilities Act (RPWD) 2016 mandated accessibility accommodations for persons with disabilities across education, employment, and public services. However, implementation has been uneven and enforcement inconsistent. The shortage of trained ISL interpreters is acute: estimates suggest that

fewer than 500 certified ISL interpreters are available to serve a deaf population of 18 million, a ratio that makes universal interpreter-mediated access logistically impossible. Most government offices, hospitals, and educational institutions lack any accessible communication facilities for deaf visitors and students.

Digital accessibility in India is similarly constrained. The few sign language technology initiatives that exist tend to be limited to pre-recorded sign language video libraries for educational content rather than real-time interactive systems. There is no widely deployed, web-accessible, real-time ISL translation tool available to the general Indian population. GestureTalk directly addresses this void, providing a freely accessible, browser-based solution that requires only a webcam and an internet connection infrastructure increasingly available even in tier-2 and tier-3 Indian cities due to the rapid expansion of affordable smartphones and broadband connectivity.

1.1.8 Technological Readiness for Practical Deployment

The decision to build GestureTalk as a web-based application reflects a deliberate assessment of the technology landscape and the deployment constraints relevant to its target users. Several converging technological developments have made Recent technological developments have made it an opportune moment for deploying gesture recognition systems at this level of accessibility.

- First, webcam hardware has become essentially ubiquitous in the target demographic. Laptops universally include built-in webcams with sufficient resolution (720p or higher) for reliable MediaPipe hand landmark detection. Even low-cost Android smartphones, increasingly common across socioeconomic strata in India, can run web-based video applications through their front cameras.
- Second, lightweight deep learning inference has become feasible on CPU-only devices. The LSTM model used in GestureTalk, with approximately 300,000 trainable parameters, produces predictions in under 50 milliseconds on a standard laptop CPU, enabling genuine real-time operation at 20+ frames per second without requiring a dedicated GPU.
- Third, the Flask web framework provides a mature and well-documented infrastructure for building Python-based web applications that expose REST APIs and serve video streams. Combined with modern HTML5, CSS3, and JavaScript on the frontend, Flask enables the construction of responsive, accessible web interfaces without requiring native application development across multiple platforms.
- Fourth, the Google Translate API and Google Text-to-Speech services provide reliable, scalable, and multilingual translation and speech synthesis capabilities that would otherwise require years of additional development to build from scratch. By integrating these services, GestureTalk is able to deliver a multilingual spoken output pipeline with minimal additional complexity.

1.1.9 Real-World Relevance and Vision

Effective communication is essential in everyday life, yet deaf and hard-of-hearing individuals often face communication barriers because most people do not understand sign language. GestureTalk – Real-Time Sign Language Recognition and Translation System aims to reduce this barrier by converting Indian Sign Language (ISL) gestures into text and speech using computer vision and deep learning techniques.

GestureTalk allows users to perform sign language gestures in front of a camera, which are then recognized by the system and converted into words that form meaningful sentences. These sentences can be translated into multiple languages and spoken aloud, enabling communication between sign language users and people who do not understand sign language.

In real-world scenarios, GestureTalk can be useful in environments such as hospitals, educational institutions, public service centers, and travel locations, where communication support for deaf individuals is often limited. By enabling real-time gesture interpretation, the system promotes inclusive communication and reduces language barriers.

The long-term vision of GestureTalk is to expand the gesture vocabulary, improve recognition accuracy, and make the system accessible across devices such as smartphones and public kiosks. This can contribute to creating a more inclusive society where communication is accessible to everyone regardless of hearing ability.

1.2 Problem Statement

The deaf and hard-of-hearing community in India and globally faces a deeply entrenched communication barrier that imposes significant costs on individual quality of life, economic productivity, and social inclusion. At the root of this barrier lies a structural asymmetry: the hearing majority communicates through spoken and written language, while the deaf minority communicates through sign language, and the two systems are mutually unintelligible without specialized training or technological mediation. This asymmetry creates a systematic disadvantage for deaf individuals in virtually every institutional context such as healthcare, education, employment, legal proceedings, and public service delivery.

The magnitude of this problem is difficult to overstate. Consider a deaf patient presenting at a general hospital emergency room in a mid-sized Indian city. In the absence of an ISL interpreter which, give the scarcity of certified interpreters, is the overwhelmingly likely scenario, the patient cannot reliably communicate the nature or location of their pain, their medical history, known allergies, or current medications. The attending physician, equally lacking in ISL proficiency, cannot ask clarifying questions or explain diagnostic procedures and treatment options. This failure of communication is not a marginal inconvenience; in a medical emergency, it can be life-threatening. Analogous communication failures occur in classrooms, courtrooms, banks, and government offices on a daily basis across India.

The technical response to this challenge has automated sign language recognition which has been the subject of academic research for over three decades. Yet despite the publication of hundreds of papers reporting high classification accuracy in controlled experimental conditions, the gap between research prototype and real-world deployable solution remains enormous. The core reasons for this persistent gap can be identified as follows:

- Hardware dependency: The majority of high-performing sign language recognition systems rely on specialized depth sensors (Microsoft Kinect, Intel RealSense), data gloves (CyberGlove, AcceleGlove), or multi-camera arrays that are unavailable, unaffordable, or impractical for the communities these systems purport to serve. A solution that requires a USD 200 depth sensor is not

accessible to a rural deaf individual in Jharkhand or a hearing-impaired student in a government school.

- Dataset limitations: Most published systems are trained on small, carefully curated datasets collected from a limited number of signers in controlled laboratory conditions. These datasets fail to capture the natural variation in signing style, speed, hand size, and environmental conditions that characterizes real-world use. Models trained on such data exhibit dramatic accuracy degradation when deployed outside their training distribution.
- Incomplete pipelines: The academic literature is dominated by systems that recognize individual signs or gestures in isolation, without addressing the downstream problem of assembling recognized signs into meaningful sentences, translating those sentences into the dominant language, or generating speech output. A gesture classifier that produces a list of detected words is of limited practical utility for real communication.
- Indian Sign Language neglect: The overwhelming majority of sign language recognition research targets American Sign Language (ASL), with significant bodies of work also addressing Chinese Sign Language and British Sign Language. ISL receives comparatively negligible research attention despite the scale of India's deaf population. The few ISL-focused systems that do exist are typically static image classifiers applied to fingerspelling or isolated signs, rather than dynamic temporal recognition systems.
- Deployment barriers: Academic systems are typically implemented as local Python scripts, Jupyter notebooks, or standalone desktop applications that require technical expertise to install and configure. They are not packaged as accessible, browser-deployable applications that a deaf individual or their communication partner could use without technical assistance.
- GestureTalk is designed as a direct technical response to each of these identified deficiencies. By building on MediaPipe for hardware-agnostic landmark extraction, using a custom LSTM model trained on a self-collected ISL dataset, implementing a complete sentence construction and translation pipeline, and deploying the system as a web application, GestureTalk addresses the full spectrum of barriers that have prevented prior research from achieving real-world impact.

Core Problem: No accessible, real-time, end-to-end Indian Sign Language recognition and translation tool currently exists that can be used by the public through a standard web browser without specialized hardware. GestureTalk is built to solve this problem.

1.3 Research gap

Sign language recognition research has achieved high accuracy using deep learning and computer vision techniques. However, most existing systems focus only on isolated gesture classification and are tested on datasets collected in controlled laboratory environments with consistent lighting and trained signers. These conditions do not fully represent real-world scenarios where gestures are performed by different individuals under varying lighting conditions and camera angles.

Another major limitation is that many studies stop at recognizing gestures and outputting a label, without addressing the complete communication process. Practical communication systems require additional components such as sentence formation,

multilingual translation, and speech generation, which are rarely integrated into a single system.

This gap is even more noticeable in the context of Indian Sign Language (ISL), where publicly available datasets and deployable applications are limited. GestureTalk addresses this gap by developing a real-time ISL recognition system that integrates gesture detection, sentence formation, translation into multiple languages, and speech output, enabling more practical communication between sign language users and non-signers.

1.3.1 What existing studies have not covered

•High Accuracy Under Controlled Conditions: A systematic review of sign language recognition research shows that many studies report very high accuracy levels, often exceeding 95%. These results are frequently published in leading journals such as IEEE Transactions on Neural Networks, Pattern Recognition, and Computer Vision and Image Understanding. However, these accuracy values are typically achieved under controlled experimental conditions.

• Dependence on Benchmark Datasets

Most research relies on benchmark datasets such as:

- RWTH-BOSTON-400 (American Sign Language)
- CSL (Chinese Sign Language)
- GSL (Greek Sign Language)
- INCLUDE (Indian Sign Language)

These datasets are generally collected in studio environments using professional signers, which does not fully represent real-world communication scenarios.

• Limited Real-World Generalization: Models trained on controlled datasets often become optimized for the specific characteristics of the dataset, including signer style, lighting conditions, and background environment. When these models are tested on real-world footage involving untrained signers or natural environments, the recognition accuracy typically decreases by 15–30 percentage points.

• Focus on Isolated Gesture Recognition: A major limitation in existing literature is that most systems focus only on sign recognition. They typically classify individual gestures or short sequences and output a label without addressing the broader communication process. Very few studies attempt to integrate the full pipeline required for practical communication. Important components such as sentence formation, grammatical structuring, contextual interpretation, multilingual translation, and speech synthesis are rarely implemented together within a single system.

• Research Gap in Indian Sign Language (ISL): The research gap becomes even more significant in the context of Indian Sign Language. Although the ISL corpus developed by ISLRTC provides a reference vocabulary, there are very limited publicly available labeled datasets suitable for training deep learning models.

• Limitations of Existing ISL Datasets: The INCLUDE dataset (2020) is one of the major datasets available for ISL research. It contains recordings from 15 signers across 263 word classes, but the recordings are captured in controlled conditions and do not include a real-world deployment framework.

• Contribution of the GestureTalk Dataset: The dataset used in the GestureTalk system was collected with real-world usability in mind. Instead of studio conditions, gesture recordings were captured using a standard webcam with varying lighting conditions, hand orientations, and camera angles, making the dataset more representative of practical usage environments.

1.3.2 Limitations of current solutions

The table below categorizes existing approaches to sign language recognition and summarizes their principal limitations:

Approach	Example Systems	Technical Limitation	Deployment Limitation
CNN (Image-based)	SignNet, DeepASL	Static frame analysis; misses temporal dynamics of gesture motion	No real-time pipeline; no translation or TTS output
Depth Sensor	Kinect-based systems	Depth estimation errors in ambient light; limited to supported hardware	Requires USD 150-300 hardware; not available to general public
Data Gloves	CyberGlove, AcceleGlove	Cannot capture non-manual markers; single-hand only in most implementations	Expensive specialized hardware; uncomfortable for extended use
HMM-based	Early ISL systems	Requires hand-crafted feature engineering; poor accuracy on continuous signing	Offline batch processing; not suitable for real-time web deployment
LSTM (webcam)	Research prototypes	Often trained on ASL; small vocabulary; no confidence filtering	Not deployed as accessible web applications; no multilingual output

GestureTalk (proposed)	This system	28-class ISL; LSTM with 2-hand MediaPipefeatures; confidence + majority voting	Web-based Flask app; multilingual translation; gTTS speech output

Table 1.1: Comparison of Existing Sign Language Systems

1.3.3 Opportunity That GestureTalk Create s

1. Addressing Multiple Research Gaps: The combination of several gaps in existing sign language recognition research including the lack of Indian Sign Language (ISL)–specific systems, the absence of complete end-to-end communication pipelines, dependence on specialized hardware, and the limited availability of web-based multilingual ISL translators creates a clear opportunity for innovation. GestureTalk aims to address these limitations by providing a system that is accessible, integrated, and designed specifically for Indian users.
2. Accessible and Web-Based Communication System: GestureTalk provides a web-deployed application that can operate using a standard webcam without requiring specialized hardware such as data gloves or motion sensors. This significantly improves accessibility and allows users to interact with the system through a regular browser interface.
3. Contribution to ISL Research: From a research perspective, GestureTalk contributes a practical methodology for collecting custom Indian Sign Language datasets using MediaPipe hand landmarks. This approach allows gesture data to be captured and stored efficiently using commonly available devices such as webcams.
4. Deep Learning-Based Gesture Recognition: The system demonstrates the implementation of an LSTM-based gesture recognition model capable of learning temporal patterns in hand movements. The model achieves approximately 95.48% validation accuracy demonstrating the effectiveness of sequential deep learning models for sign language recognition.
5. Integrated End-to-End Pipeline: GestureTalk integrates multiple components into a single pipeline, including gesture recognition, sentence formation, multilingual translation, and speech synthesis. This integration moves beyond isolated gesture classification and provides a more practical communication solution.
6. Framework for Future Research: The architecture and implementation of GestureTalk provide a replicable framework that other researchers can extend. Future research can expand the gesture vocabulary, improve model architecture, or adapt the system to other sign languages.
7. Practical Impact in Real-World Environments: From a practical perspective, GestureTalk offers a low-cost communication tool that could help deaf and hard-of-hearing individuals communicate more easily in everyday situations.
8. Potential Applications in Key Sectors: The system has potential applications in areas such as healthcare, education, and public service environments, where access to professional sign language interpreters is often limited or unavailable.

1.4 Objectives of the Study

The development of GestureTalk is guided by a structured set of primary and secondary objectives that together define the scope, technical ambition, and expected outcomes of the project. These objectives have been formulated to directly address the problem statement and research gaps identified in the preceding sections.

1.4.1 Primary Objectives

1. To design and implement a real-time sign language recognition system capable of accurately identifying Indian Sign Language (ISL) gestures from live webcam input, operating without specialized hardware or dedicated GPU computation.
2. To develop a custom Long Short-Term Memory (LSTM) deep learning model trained on a self-collected ISL gesture dataset encompassing 28 gesture classes approximately 940 labeled sequences of 20 frames each, achieving a target accuracy of 90% or higher on the validation set.
3. To construct an end-to-end communication pipeline that transforms recognized gesture sequences into natural language sentences, translates those sentences into multiple target languages using the Google Translate API, and generates audible speech output using the Google Text-to-Speech engine.
4. To deploy the complete system as a web-accessible application using the Flask framework, providing a responsive and intuitive user interface usable by individuals without technical expertise, accessible through any standard web browser.

1.4.2 Secondary Objectives

1. To leverage MediaPipe Hands for efficient, robust, and real-time extraction of 21 three-dimensional skeletal landmark coordinates per hand as the primary input feature representation for the recognition model, replacing raw pixel-based input with a hardware-invariant skeletal description.
2. To implement a comprehensive data normalization pipeline that renders the extracted landmark features invariant to global hand position, hand size, and camera distance, improving the generalization of the trained model to diverse users and environments.
3. To design and evaluate noise-reduction mechanisms specifically a confidence threshold filter and a majority-voting smoothing algorithm over a rolling prediction window — that suppress false or flickering detections and ensure stable real-time output during idle periods and gesture transitions.
4. To introduce a dedicated NO_SIGN gesture class in the training vocabulary to enable the system to distinguish between active signing and resting hand positions, reducing false positive detections during pauses in communication.
5. To evaluate the system's performance comprehensively through training loss curves, validation accuracy metrics, a confusion matrix across all 28 gesture classes, and qualitative assessment of real-time detection stability and latency.
6. To document the complete system architecture, dataset collection methodology, model training process, and deployment configuration in sufficient detail to enable reproduction and extension by future researchers working in the field of assistive sign language technology.

1.5 Scope of project

The scope of GestureTalk has been carefully defined to balance technical ambition with practical achievability within the constraints of a final-year undergraduate project. The scope boundaries reflect deliberate design decisions rather than incidental limitations, and are intended to establish a well-defined, reproducible baseline from which future extensions can proceed.

1.5.1 Geographic and Linguistic Scope

GestureTalk is explicitly scoped to Indian Sign Language (ISL), making it relevant to the approximately 18 million deaf and hard-of-hearing individuals in India who use ISL as their primary communication medium. The 28 gesture classes selected for the initial vocabulary were chosen to represent a practical cross-section of high-frequency words and phrases applicable to everyday communication scenarios including greetings, basic needs, common questions, numbers, and simple actions. The vocabulary selection was guided by the ISLRTC reference dictionary and validated against common communication needs identified in community healthcare and educational accessibility literature.

While the gesture recognition model is specific to ISL, the system architecture including the MediaPipe landmark extraction pipeline, the LSTM sequence classifier, the Flask API, and the translation and TTS components that is designed to be language-agnostic and could be adapted to other sign languages (ASL, BSL, or other regional sign systems) by retraining the model on an appropriate dataset with minimal architectural changes.

1.5.2 Technical Scope

On the technical side, GestureTalk's scope is defined by the following boundaries:

- Input modality: Standard webcam video at a minimum resolution of 640x480 pixels and a minimum frame rate of 24 fps. No depth sensor, infrared camera, or specialized hardware is required or supported.
- Feature extraction: MediaPipe Hands landmark coordinates for up to two hands simultaneously. Non-manual ISL markers (facial expressions, head movements, shoulder positions, lip movements) are outside the current scope.
- Gesture vocabulary: 28 ISL gesture classes including the NO_SIGN idle class. Continuous signing at a conversational pace is not directly supported; the system operates on isolated gesture sequences.
- Temporal window: Each gesture is classified from a fixed-length sequence of 20 consecutive frames captured at a target rate of 10-15 fps for the landmark feature stream, corresponding to approximately 1.5-2 seconds of gesture movement.
- Translation: Multilingual text translation using the Google Translate API. Output languages are configurable from a dropdown menu in the interface. Offline translation is not supported.
- Speech output: Audio playback through the device speaker using gTTS-generated audio files served via the Flask backend. Real-time streaming audio synthesis is outside the scope.

1.5.3 What is Included and Excluded

Included in Scope	Excluded from Scope
28 ISL hand gesture classes	Full ISL grammatical parsing and syntax modeling
Two-hand real-time MediaPipe detection	Non-manual markers: facial expressions, lip movement, body posture
Custom LSTM temporal sequence classifier	Alternative architectures: CNN, Transformer, SVM, Random Forest
Self-collected ISL landmark dataset (940 sequences)	Pre-existing public sign language benchmark datasets
Sentence-level word aggregation and construction	Continuous signing recognition at conversational pace
Multilingual text translation via Google Translate	Offline or on-device language translation
Text-to-speech audio output via gTTS	Sign language synthesis or animated avatar output
Web-based Flask application with REST APIs	Native mobile application (Android/iOS) deployment
Confidence thresholding and majority-vote smoothing	Multi-user simultaneous signing in a single session
NO_SIGN class for idle detection	Emotion recognition or intent classification from gestures

Table 1.2: Scope of the GestureTalk System

1.5.4 Expected Users and Stakeholders

The primary end-users of GestureTalk are deaf and hard-of-hearing individuals who use Indian Sign Language as their primary communication medium, and their hearing communication partners such as family members, healthcare workers, teachers, government officials, and members of the public who wish to understand signed communication without prior knowledge of ISL.

Secondary stakeholders include educators and researchers in the fields of deep learning, computer vision, assistive technology, and linguistics who may wish to use the GestureTalk system architecture, dataset, or trained model as a foundation for further research. Policymakers and disability rights advocates may also find the system relevant as a demonstration of the technical feasibility of affordable, accessible sign language translation infrastructure.

1.6 Significance of the study

This study is significant because it addresses the communication barrier between deaf or hard-of-hearing individuals and people who do not understand sign language. GestureTalk provides a real-time system that recognizes Indian Sign Language gestures and converts them into text and speech, enabling easier communication in everyday situations. By integrating gesture recognition, sentence formation, multilingual translation, and speech synthesis into a single web-based application, the system demonstrates how artificial intelligence and computer vision can be used to create accessible and inclusive communication technologies.

1.6.1 Social and Humanitarian Significance

The most immediate and tangible significance of GestureTalk lies in its potential to reduce the communication isolation experienced by deaf and hard-of-hearing individuals in India. Communication barriers for the deaf are not merely inconvenient, they constitute a form of structural exclusion from the systems and institutions that govern access to healthcare, education, legal protection, and economic opportunity. By providing a real-time, automated ISL translation tool that operates on standard consumer hardware and is accessible through any web browser, GestureTalk lowers the barrier to accessible communication in a way that human interpreter services currently cannot achieve at scale.

In healthcare settings specifically, the impact of improved communication accessibility could be life changing. Deaf patients frequently experience delayed or incorrect diagnosis due to communication failures during clinical consultations. A browser-accessible sign language translator available on a hospital computer or tablet could enable deaf patients to communicate symptoms, ask questions about their treatment, and provide accurate medication and allergy information that potentially preventing serious medical errors and improving health outcomes.

In educational settings, a freely accessible ISL translation tool could support deaf students in mainstream classrooms, enabling them to communicate with hearing teachers and classmates during activities and discussions that currently exclude them. This is particularly relevant in India, where the Right to Education Act mandates inclusive education but implementation in hearing-impaired inclusive classrooms remains inadequate.

1.6.2 Technical and Scientific Significance

From a technical perspective, GestureTalk demonstrates several propositions that are significant for the broader research community working on sign language recognition and assistive technology:

- It demonstrates that an LSTM-based ISL gesture recognition system trained on a modest custom dataset of 940 sequences can achieve approximately 95% validation accuracy using only standard CPU hardware and a webcam validating the feasibility of accessible, hardware-agnostic deployment.

- It establishes a complete, reproducible pipeline from raw webcam input to spoken multilingual output, demonstrating the integration of computer vision, deep learning inference, REST API design, and third-party AI services in a single coherent system.
- It demonstrates the effectiveness of MediaPipe skeletal landmark features, combined with wrist-relative normalization, as a robust and generalizable input representation for ISL gesture recognition, contributing to the evidence base for this approach.
- It provides a documented and replicable methodology for collecting custom sign language datasets using consumer webcams and MediaPipe, which can serve as a template for researchers seeking to extend the ISL vocabulary or develop datasets for other regional sign languages.

These contributions collectively advance the state of accessible, deployable sign language technology and provide a concrete foundation for future work in this critically important area of AI for social good.

1.6.3 Institutional and Policy Significance

GestureTalk also carries significance at the institutional and policy level. India's Rights of Persons with Disabilities Act (2016) and the National Policy for Persons with Disabilities (2006) both mandate accessibility accommodations for communication-disabled individuals. The development and public demonstration of a functional, low-cost ISL translation tool provides evidence to support policy advocacy for the integration of such tools into public healthcare, educational, and government service infrastructure. It demonstrates that the technical barriers to ISL translation technology are lower than commonly assumed, shifting the conversation toward questions of deployment, funding, and institutional will rather than technological feasibility.

1.7 Structure of the Report

This project report is systematically structured to present the study of GestureTalk – Real-Time Sign Language Recognition and Translation System in a clear, logical, and comprehensive manner. Each chapter focuses on a specific stage of the project, starting with the introduction to the problem domain and progressing through literature review, methodology, system design, implementation, and evaluation. The structure ensures that the reader can easily understand the motivation behind the project, the techniques used to develop the system, and the outcomes achieved.

Chapter 1 – Introduction

This chapter introduces the concept of sign language communication and its challenges, particularly the communication gap between deaf or hard-of-hearing individuals and people who do not understand sign language. It provides the background of the study, explains the need for automated sign language interpretation systems, and highlights the importance of Indian Sign Language (ISL) recognition. The chapter also presents the problem statement, research gap, objectives of the study, scope, and significance of the project, and concludes with an overview of the report structure.

Chapter 2 – Review of Literature

This chapter presents a detailed review of existing research and technologies related to sign language recognition, computer vision, and deep learning-based gesture recognition systems. It discusses previous studies that have used methods such as

convolutional neural networks (CNNs), recurrent neural networks (RNNs), and LSTM models for gesture recognition. The literature review helps identify the limitations of existing systems, such as dependence on controlled datasets, lack of Indian Sign Language datasets, and the absence of integrated translation and speech modules. This chapter establishes the theoretical foundation for the proposed GestureTalk system.

Chapter 3 – Research Methodology

This chapter explains the overall research framework and methodology adopted for the development of GestureTalk. It describes the data collection process, dataset structure, preprocessing techniques, and feature extraction methods used in the project. The chapter also explains the use of MediaPipe for hand landmark detection, the creation of gesture sequences, and the use of LSTM-based deep learning models for gesture recognition. Additionally, the chapter outlines the tools and technologies used for system development.

Chapter 4 – System Design and Architecture

This chapter focuses on the conceptual and technical design of the GestureTalk system. It presents the overall system architecture, module design, and data flow between components. The chapter explains the design of the gesture recognition pipeline, including webcam capture, landmark extraction, sequence creation, and model prediction. It also discusses the frontend interface, backend architecture, and integration of translation and text-to-speech modules. Diagrams such as system architecture diagrams, workflow diagrams, and pipeline diagrams are used to illustrate how the system components interact.

Chapter 5 – Implementation

This chapter describes the practical implementation of the GestureTalk system. It explains the technology stack used for development, including Python, OpenCV, MediaPipe, TensorFlow, Flask, and web technologies such as HTML, CSS, and JavaScript. The chapter details the implementation of gesture data collection, landmark extraction, LSTM model training, real-time gesture prediction, sentence formation, translation, and speech generation. Code snippets, screenshots, and diagrams are included to demonstrate how the system was developed and integrated.

Chapter 6 – Results and Evaluation

This chapter presents the results obtained from the implemented system. It evaluates the performance of the gesture recognition model using metrics such as accuracy, precision, recall, and confusion matrix analysis. The chapter also demonstrates real-time system outputs, including gesture detection, sentence formation, translation, and speech output. Sample test cases and performance observations are used to assess the system's effectiveness in practical scenarios.

Chapter 7 – Summary, Limitations, and Future Work

This chapter provides a summary of the work carried out in the project and highlights the main contributions of the GestureTalk system. It also discusses the limitations of the current system, such as restricted gesture vocabulary and dataset size. Finally, the chapter outlines possible directions for future improvements, including expanding the dataset, improving model accuracy, supporting additional sign languages, and deploying the system on mobile platforms.

References and Appendices

The final section of the report contains all references and sources cited throughout the document, formatted according to the chosen referencing style. The appendices include supplementary materials such as additional screenshots, dataset details, system configurations, code snippets, and other supporting documents that help provide a deeper understanding of the GestureTalk system but are not included in the main chapters.

CHAPTER 2 : REVIEW OF LITERATURE

2.1 Ahmed et al., 2025 [1] The Authors propose a real-time American Sign Language interpretation system integrating YOLOv11 for gesture recognition with MediaPipe for precise hand keypoint tracking, achieving 98.2% mean Average Precision (mAP@0.5) on a large-scale dataset of 130,000 annotated ASL hand gesture images. The system is built to address practical challenges of gesture ambiguity between similar letter signs, environmental variation in lighting and background complexity, and computational efficiency requirements for real-time deployment. MediaPipe is used to extract 21 three-dimensional keypoints per hand from each video frame, providing annotated skeletal representations used as inputs for YOLOv11 training and inference. The resulting system can precisely identify all 26 ASL alphabet letters in real time using only a standard built-in laptop webcam. The Authors discuss practical utility including enabling deaf users to spell out proper nouns, names, and locations that may not exist as predefined whole-word signs in the vocabulary. A significant contribution of this work is the construction and annotation of the 130,000-image training dataset, which represents one of the largest keypoint-annotated ASL datasets documented in the literature.

2.2 Jain et al., 2025 [11] The Authors present a comprehensive survey of advancements in machine learning techniques for hand gesture-based sign language recognition, reviewing over 150 papers from the period 2015–2024 and providing a structured taxonomy of input modalities, feature extraction methods, classification architectures, and deployment strategies. The survey covers sensor-based approaches including data gloves and depth cameras alongside vision-based approaches using RGB cameras, evaluating each in terms of accuracy, hardware requirements, scalability, and practical accessibility. The Authors find that vision-based approaches using skeleton landmarks extracted via MediaPipe consistently outperform pixel-based methods in real-world generalization, providing independent validation for the feature extraction strategy employed in GestureTalk. Deep learning classifiers including CNN, LSTM, Transformer, and Graph Neural Networks are reviewed and benchmarked, with LSTM identified as the dominant architecture for dynamic gesture recognition from temporal landmark sequences. The paper is published in Springer

2.3 Rawat et al., 2025 [31] The Authors present a comprehensive ISL gesture recognition system integrating a Sequential Long Short-Term Memory model with MediaPipe Holistic for accurate and real-time gesture recognition, providing a straightforward pipeline divided into three phases: feature extraction using MediaPipe Holistic, data cleaning and labelling, and gesture identification using the LSTM classifier. The system tracks landmarks on the face, hands, and body across video frames to capture both spatial and temporal features of ISL gestures. Unlike many ISL systems that focus only on hand movements, this approach incorporates facial and body pose landmarks alongside hand landmarks, acknowledging that some ISL signs involve non-manual components. The Authors evaluate the system on static and dynamic ISL gesture classes and report high recognition accuracy. Published in EAI Endorsed Transactions on AI and Robotics in 2025, this paper represents one of the most recent contributions to ISL gesture recognition using the MediaPipe Holistic plus LSTM pipeline directly comparable to GestureTalk.

2.4 Robles-Sanchez et al., 2025 [32] The Authors introduce a real-time assistive technology solution translating sign language gestures into textual and audible speech using CNN trained on the Sign Language MNIST dataset, demonstrating a complete gesture-to-speech pipeline accessible through webcam-only hardware. The system classifies hand gestures from live webcam input using the trained CNN and converts the recognised gesture labels directly to speech output using a text-to-speech engine, completing the full pipeline from gesture to voice in real time. Comprehensive experiments demonstrate high model accuracy and robust real-time performance, confirming the practical applicability of the gesture-to-speech approach as an accessible and affordable communication tool for sign language users. Published on arXiv in 2025, this paper provides a very recent reference validating the GestureTalk gesture-to-speech pipeline concept. Key challenges discussed include coarticulation effects in continuous signing, speaker-dependent signing style variation, and the need for real-time audio-visual synchronisation.

2.5 Svendsen E. and Kadry S., 2025 [38] The Authors present a preliminary system for recognising Norwegian Sign Language (NSL) gestures for numbers 0 to 10 using MediaPipe for feature extraction and Long Short-Term Memory networks for temporal modelling, demonstrating the generalisability of the MediaPipe-LSTM pipeline to an underrepresented sign language. The system uses MediaPipe Face Mesh and Hands models to extract angle and magnitude features from hand and lip landmarks, capturing orientation and spatial distance information from each video frame to build a rich per-frame feature description. These features feed into an LSTM classifier trained on a dataset of NSL educational videos provided by Statped, Norway's national resource centre for special education, ensuring diversity in signer age, gender, and signing style. The system achieves a testing accuracy of 95%, aligning with benchmarks reported for comparable MediaPipe-LSTM systems trained on well-studied sign languages. The Authors identify challenges including data imbalance and misclassification of visually similar gestures such as Signs 3 and 8. A key finding of this work is that the MediaPipe-LSTM pipeline generalises effectively to completely unrelated sign languages, supporting its use as a language-agnostic framework.

2.6 Alshehri et al., 2024 [2] The Authors propose a hybrid CNN-LSTM model for real-time Arabic Sign Language recognition using MediaPipe for hand landmark extraction, addressing the gap in sign language recognition for the Arabic-speaking deaf community in Saudi Arabia. The hybrid model captures spatial features per frame through CNN and temporal dependencies across gesture sequences through LSTM, forming a complete spatio-temporal classification architecture. MediaPipe is used to extract hand landmark coordinates from images and video frames, storing them as input features for both the static and dynamic classifiers. The Authors create a custom dataset of 20 Arabic Sign Language words comprising 4,000 images for 10 static gesture classes and 500 videos for 10 dynamic gesture classes. The CNN classifier achieves 94.40% accuracy for static sign recognition while the LSTM classifier achieves 82.70% for dynamic gesture sequences, illustrating the relative difficulty of temporal versus static classification. Published in MDPI Sensors, this paper provides a directly comparable reference architecture to GestureTalk employing the same MediaPipe plus deep learning pipeline for a different sign language. The work demonstrates that the CNN-LSTM hybrid approach is widely applicable across diverse sign language communities, confirming the general applicability of MediaPipe-based gesture recognition systems

2.7 Jintanachaiwat et al., 2024 [12] The Authors present a system for real-time translation of Thai Sign Language into Thai text using MediaPipe Holistic for keypoint extraction and a Feed-Forward Neural Network combined with Long Short-Term Memory for temporal sequence classification. MediaPipe Holistic extracts left hand, right hand, face, and posture landmarks simultaneously, yielding a 1662-dimensional feature vector per frame capturing multimodal signing cues. Sequences of 30 frames are processed per gesture classification. The Authors evaluate standard RNN, Bidirectional RNN, LSTM, and Bidirectional LSTM architectures, selecting FNN-LSTM as the best configuration. The system targets continuous conversational signing rather than isolated gesture classification, requiring the model to handle natural variation in gesture duration, speed, and signer style. Published in Discover Artificial Intelligence by Springer Nature, this paper demonstrates that the MediaPipe-LSTM pipeline generalises to Thai Sign Language, reinforcing its cross-linguistic applicability. The 1662-dimensional holistic feature vector provides a richer feature set than the 126-dimensional hand-only features used in GestureTalk, illustrating the accuracy versus efficiency trade-off in MediaPipe-based sign language systems. The Authors propose future work targeting larger TSL vocabulary and integration of non-manual markers.

2.8 Khajeh Pourian D. and Zarei A. D., 2024 [14] The Authors propose a modified deep learning approach for sign language recognition using a hybrid CNNSa-LSTM architecture combining VGG16 for spatial feature extraction with optical flow for motion feature capture and a self-attention LSTM for temporal classification, augmented by a hippopotamus optimisation algorithm for hyperparameter tuning. VGG16 extracts fine-grained geometric features from individual frames while optical flow captures motion dynamics between consecutive frames, producing a multi-modal feature representation that encodes both appearance and movement. The self-attention mechanism assigns importance weights to individual frames within each gesture sequence, enabling the classifier to focus on the most discriminative gesture moments. The hippopotamus optimisation algorithm improves training convergence stability compared to standard Adam optimisation. Published in Nature Scientific Reports, this paper demonstrates a technically sophisticated approach to sign language recognition that significantly outperforms standard LSTM and CNN-only baselines.

2.9 Ling et al., 2024 [17] The Authors present a video-based sign language recognition method using ResNet-18 combined with Long Short-Term Memory networks, targeting real-time deployment on devices with limited GPU resources such as smartphones and low-specification personal computers. ResNet-18 is selected as the backbone spatial feature extractor because of its lightweight architecture and effective residual connections that mitigate overfitting, making it well suited for resource-constrained deployment. The LSTM module processes sequences of ResNet-extracted spatial feature maps to model temporal dynamics across gesture frames. The system is evaluated on Argentine Sign Language video data, demonstrating that the ResNet-18 backbone provides an effective balance between recognition accuracy and computational efficiency. The Authors discuss the practical challenge that large CNN models and 3D convolutional architectures achieve strong accuracy but are computationally too costly for mobile or low-specification hardware. Published in MDPI Journal of Imaging, this paper provides directly relevant evidence for the deployment trade-off between model capacity and hardware accessibility.

2.10 Miah et al., 2024 [19] The Authors propose a hybrid Sign Language Recognition framework combining MobileNetV2 as a lightweight CNN backbone with an attention-based LSTM neural network for isolated sign language word recognition from video frame sequences. MobileNetV2 is chosen for its lightweight parameter count, enabling deployment on mobile and edge devices while preserving high spatial feature extraction accuracy. The attention mechanism dynamically assigns importance weights to individual frames in each sequence, focusing the classifier on the most discriminative gesture moments and reducing influence from transitional or idle frames that contribute less meaningful information. Published in MDPI Electronics, the paper provides a comprehensive comparison of the hybrid attention-LSTM against CNN-only and standard LSTM baselines, demonstrating consistent accuracy improvement from the attention mechanism. The Authors discuss the tendency of standard LSTMs to weight recent frames disproportionately and argue that the attention mechanism corrects this bias by allowing the classifier to attend to globally informative frames anywhere in the sequence. The hybrid MobileNetV2 plus attention-LSTM framework represents an effective balance between recognition accuracy and inference efficiency.

2.11 Narang et al., 2024 [22] The Authors present an AI-driven Indian Sign Language recognition system using LSTM for accurate gesture recognition with multilingual text output, demonstrating real-time ISL gesture interpretation and translation across multiple Indian regional languages using a web-accessible interface. The LSTM model is trained on a custom ISL dataset to interpret complex ISL hand movements in real time, converting recognised signs to text and supporting translation into multiple languages. The system interface provides both text output and speech synthesis in the target language. The work demonstrates that LSTM-based ISL recognition can be effectively combined with multilingual translation and speech output in a single integrated system, which is precisely the integration implemented in GestureTalk. The paper confirms the technical feasibility and practical value of the complete gesture-to-multilingual-speech pipeline for the Indian deaf community. The multilingual output design aligns with India's linguistic diversity, where communication accessibility requires support for multiple regional languages beyond Hindi and English, validating GestureTalk's choice to provide multilingual translation through the Google Translate API before gTTS speech synthesis.

2.12 Navendu P. and Sahula V., 2024 [24] The Authors present a word-level Indian Sign Language recognition system using MediaPipe Hands and Pose for feature extraction combined with a hybrid LSTM-GRU network for temporal sequence classification, evaluated on the INCLUDE dataset of 4,292 ISL gesture videos across 263 classes. The hybrid LSTM-GRU architecture is motivated by the complementary strengths of the two recurrent unit designs: LSTM's long-range memory retention and GRU's computational efficiency and faster training convergence. MediaPipe extracts keypoint coordinates from video frames, forming the input feature vectors for the hybrid classifier. The Authors report a test accuracy of 89.50% on the INCLUDE dataset, representing improvement over prior LSTM-only models evaluated on the same benchmark. The paper is published on TechRxiv IEEE and represents a significant contribution to ISL recognition demonstrating that the MediaPipe feature extraction pipeline combined with a hybrid LSTM-GRU classifier achieves competitive accuracy on a large-scale diverse ISL benchmark. The hybrid architecture findings suggest a practical design principle for GestureTalk future work: replacing the pure stacked LSTM with a hybrid LSTM-GRU configuration may improve recognition accuracy while reducing training time.

2.13 Ramanathan M. and Patil S., 2024 [28] The Authors propose an enhanced ASL-to-ISL translation framework using Large Language Models, targeting the communication gap between ASL and ISL users resulting from the structural and vocabulary differences between the two sign languages. The framework combines image recognition for ASL gesture detection with LLM-based language processing for cross-language sign translation, demonstrating that contemporary LLMs can function as effective translation engines between sign language vocabularies when provided with appropriate signed language description inputs. Accuracy improvements over prior rule-based cross-sign-language translation are demonstrated. Published on arXiv in 2024, this paper points toward the future direction of sign language translation technology: systems that not only bridge the deaf-hearing communication gap but also bridge the gap between different national sign language communities. For GestureTalk, this paper contextualises the multilingual translation feature as part of a broader accessible communication vision, where future extensions could leverage LLM-based translation to support not only text translation between spoken languages but also cross-sign-language translation between ISL and other sign systems, substantially expanding the communicative reach of the system.

2.14 Rocha et al., 2024 [33] The Authors propose an efficient sign language recognition system utilising low-cost sensors and deep learning techniques, targeting real-world deployment without expensive depth cameras or data gloves. An object detection model is trained specifically for detecting the face and hands in video frames, ensuring the classifier focuses on the most relevant image regions. The system avoids the limitations of pixel-level hand segmentation approaches by using object detection to crop hand regions robustly. The paper is published on arXiv and provides relevant evidence that competitive sign language recognition accuracy is achievable through lightweight vision-based approaches that do not require specialised hardware, consistent with the hardware-agnostic philosophy of GestureTalk. The Authors discuss the engineering challenge of achieving adequate accuracy with low-cost sensors and propose future extensions including skeleton-based landmark representations precisely the approach implemented in GestureTalk as a promising direction for improving robustness without increasing hardware requirements.

2.15 Saleh and Issa, 2024 [34] The Authors review sign language recognition and interpretation studies based on machine learning, image processing, artificial intelligence, and animation tools, discussing both the sign-to-text direction and the speech-to-sign direction of the accessibility pipeline. The review covers hand gesture recognition, facial expression interpretation, lip reading, and animation-based sign language synthesis as components of a comprehensive bidirectional sign language communication system. Popular and highly rated mobile applications facilitating communication for deaf and mute people are also evaluated. The paper is published in Springer Neural Computing and Applications and provides a systems-level view of how multiple components must integrate to deliver complete communication accessibility. The review identifies text-to-speech synthesis as a critical but often neglected final stage in sign language recognition pipelines, noting that converting recognised text into speech substantially increases the practical utility of recognition systems for hearing non-signers who interact with deaf users. This finding validates GestureTalk's inclusion of gTTS-based multilingual speech output as a core system feature rather than an optional add-on, and supports the design decision to provide multilingual translation before speech synthesis.

2.16 Srivastava et al., 2024 [36] The Authors present a continuous ISL recognition system using deep learning with a Long Short-Term Memory (LSTM) model, trained and tested on a custom ISL primary dataset created using the MediaPipe Holistic pipeline for tracking face, hand, and body movements and collecting landmark coordinates. The system is designed to recognise signs and gestures in real time from a webcam feed without requiring specialised hardware, emphasising accessibility for the Indian deaf and hearing-impaired community. MediaPipe Holistic is used to simultaneously extract landmarks from the face, hands, and body pose across video frames, building a comprehensive multimodal feature representation per frame that captures both manual and non-manual signing cues. The collected landmark arrays are used directly as input to the LSTM network, eliminating the need to store raw video data and reducing storage and processing overhead. The LSTM classifier processes sequences of landmark frames and classifies them into predefined ISL word categories. The system achieves 88.23% recognition accuracy on the custom ISL dataset and demonstrates real-time operation on standard hardware.

2.17 Buttar et al., 2023 [5] The Authors propose a deep learning approach for sign language recognition combining CNN and LSTM for the accurate recognition of both static and dynamic signs, addressing the challenge that neither architecture alone handles both modalities optimally. Static signs are best captured by CNN spatial features applied to individual frames, while dynamic signs require temporal sequence modelling over multiple frames as provided by LSTM. The hybrid approach applies a CNN to individual frames for spatial feature extraction and an LSTM to sequences of CNN feature maps for temporal classification, creating a unified model capable of handling both sign types. The system is evaluated on ASL and other sign language benchmarks, demonstrating competitive accuracy for both static and dynamic gesture categories. Published in MDPI Mathematics, this paper provides a methodological reference for hybrid spatial-temporal architectures in sign language recognition. The Authors discuss the challenge that most existing systems specialise in either static or dynamic recognition but not both, arguing that a unified architecture is essential for practical sign language communication systems.

2.18 Goyal K. and Velmathi G., 2023 [8] The Authors compare CNN and LSTM models for Indian Sign Language recognition using MediaPipe Holistic for landmark extraction, providing a systematic empirical comparison of the two key deep learning architecture families on the same ISL dataset. MediaPipe Holistic provides simultaneous pose, hand, and face landmark estimation, and the Authors record a custom dataset by extracting landmark arrays without storing raw video to conserve storage. The dataset is split 80/20 for training and testing. The CNN model uses convolutional layers with batch normalisation and dropout, while the LSTM model employs three stacked LSTM layers with batch normalisation. The Authors find that CNN outperforms LSTM on static sign language recognition while LSTM significantly outperforms CNN on dynamic gesture sign language recognition involving temporal sequences. This critical empirical result directly validates the design choice of GestureTalk, which uses LSTM for dynamic ISL gesture recognition from temporal landmark sequences. Published on arXiv, this paper provides the most methodologically comparable reference to GestureTalk's dataset collection and model training approach, using the same MediaPipe Holistic landmark extraction pipeline and stacked LSTM architecture on ISL data and confirming that LSTM is the appropriate architecture for dynamic ISL gesture classification.

2.19 Iyer M. R., 2023 [10] The Authors propose a deep learning approach for ISL recognition using a hybrid CNN-LSTM model that captures both spatial features per frame through the CNN component and temporal dependencies across gesture sequences through the LSTM component, targeting the combined spatial and temporal complexity of ISL dynamic gestures. The CNN processes individual video frames to extract discriminative spatial features encoding hand shape and configuration, while the LSTM processes sequences of CNN feature maps to classify the gesture from its full temporal trajectory. The system is evaluated on a custom ISL dataset and achieves strong accuracy results, with the hybrid spatial-temporal approach significantly outperforming CNN-only and LSTM-only baselines. The paper discusses data augmentation strategies including temporal warping, random frame dropping, and horizontal mirroring to expand the training corpus. The use of both spatial CNN features and temporal LSTM modelling is consistent with the broader literature on dynamic gesture recognition and provides context for understanding why pure landmark-based LSTM approaches like GestureTalk, which rely on MediaPipe for spatial feature extraction rather than a CNN, can achieve comparable performance with lower computational overhead.

2.20 Navendu P. and Sahula V., 2023 [23] The Authors propose a method for recognising isolated words in Indian Sign Language using MediaPipe Holistic for feature extraction combined with an LSTM network, evaluated on the large-scale INCLUDE dataset comprising 4,292 high-definition ISL videos spanning 263 gesture classes from native ISL signers. The MediaPipe Holistic pipeline extracts keypoints from video frames, and the LSTM classifier is trained to recognise ISL words from temporal sequences of these keypoints. The system achieves 94.8% accuracy on the INCLUDE-50 subset and 87.4% on the full INCLUDE dataset, with macro averaged F1-scores of 93.5% and 86.6% respectively. Published as a Springer journal paper, this work is one of the most rigorous evaluations of the MediaPipe-LSTM pipeline on a large-scale ISL benchmark, providing strong empirical support for the architecture's suitability for ISL word recognition. The INCLUDE dataset evaluation demonstrates that MediaPipe landmark features combined with LSTM temporal modelling achieve competitive accuracy on a 263-class ISL vocabulary, validating the scalability of the approach beyond small custom datasets.

2.21 Nguyen et al., 2023 [25] The Authors investigate optimisation strategies for real-time sign language recognition using the MediaPipe framework, introducing a multi-modal methodology that combines four LSTM models dedicated to processing skeletal coordinates extracted from different MediaPipe landmark groups: left hand, right hand, face, and body pose. Empirical evaluations on established sign language datasets demonstrate that the multi-modal ensemble approach significantly improves SLR accuracy over single-stream LSTM models while preserving real-time inference capability. The CTU Journal of Innovation and Sustainable Development paper provides a systematic evaluation of how to maximise the information extracted from MediaPipe Holistic's rich landmark output through multi-stream processing, rather than concatenating all landmark types into a single feature vector. This finding is relevant to GestureTalk's design choice of using only hand landmarks for 126-dimensional features per frame: while the two-hand-only approach delivers strong accuracy and efficiency, the multi-stream approach documented in this paper suggests that incorporating additional face and pose streams in future GestureTalk versions could further improve recognition accuracy for ISL signs that involve non-manual markers.

2.22 Vashisth et al., 2023 [39] The Authors present a hand gesture recognition system for Indian Sign Language using deep learning with a self-collected real-world ISL dataset, achieving 99% recognition accuracy with a test loss of 0.0178. The key contribution of this work is the deliberate design of a dataset representing natural usage conditions: images are collected under diverse lighting, backgrounds, and hand sizes rather than the controlled studio conditions that characterise most existing ISL datasets. The CNN architecture employs image preprocessing including thresholding, edge detection, and skeleton morphology to enhance feature extraction quality. Data augmentation methods including random rotation, horizontal flipping, and brightness adjustment mitigate overfitting and improve generalisation to unseen gesture variations. The Authors apply standard CNN layers with batch normalisation and dropout, training on the self-collected ISL dataset of static gesture images. The 99% accuracy result demonstrates that a CNN trained on a diverse, realistic ISL dataset achieves near-perfect accuracy for static gesture recognition. Published in MDPI Engineering Proceedings, this paper directly validates GestureTalk's approach of collecting a custom ISL dataset via webcam under realistic usage conditions, reinforcing the importance of realistic dataset design over curated studio datasets for building generalisable ISL recognition systems.

2.23 Vyavahare et al., 2023 [40] The Authors propose a dynamic ISL gesture recognition system using LSTM networks for temporal classification from standard webcam video, implementing a complete recognition pipeline including OpenCV frame capture, MediaPipe landmark extraction, LSTM temporal classification, and majority-vote prediction smoothing over a rolling prediction window. The majority-voting mechanism suppresses noise and stabilises the output by requiring multiple consecutive frames to agree on a classification before it is accepted, reducing flickering misclassifications during gesture transitions and idle periods between signs. This prediction smoothing strategy is directly identical to the approach implemented in GestureTalk, making this one of the most methodologically aligned references in the literature. The Authors evaluate their system on a custom ISL gesture video dataset and report competitive classification accuracy, discussing the challenge of distinguishing similar ISL gestures that differ in subtle motion characteristics.

2.24 Brzezinski A., 2022 [4] The Author presents an application for recognising American Sign Language gestures using convolutional neural network architectures, implementing the full development pipeline from training dataset creation and preprocessing to model training, validation, and deployment as a web application built with the Flask framework. The web application allows users to upload sign language images from any device for recognition, making the system broadly accessible without requiring a dedicated installation. The CNN model achieves 99% accuracy on the training set and demonstrates high recognition performance during web application testing. Python version 3.7.2, the PyCharm development environment, and Keras and TensorFlow libraries are used for model development, while the Flask framework is used for the web application backend. This paper is directly relevant to GestureTalk as it demonstrates the feasibility and practical implementation of a Flask-based web deployment for sign language recognition, validating the same Flask deployment architecture employed in GestureTalk. Published in MDPI Sensors, the work shows that Flask provides a mature and accessible web framework for deploying deep learning sign language recognition models as browser-accessible applications without native application development.

2.25 Katoch et al., 2022 [13] The Authors present an Indian Sign Language recognition system using Speeded Up Robust Features combined with Bag of Visual Words model and two classifiers, Support Vector Machine and Convolutional Neural Network, evaluated on a custom ISL alphabet and numeral dataset. The system uses background subtraction and skin tone-based segmentation to isolate the hand region from each image frame before SURF feature extraction. Histograms of SURF descriptors are computed and matched to visual vocabulary codebooks to produce the Bag of Visual Words representation for each gesture. Both SVM and CNN classifiers are trained and compared on the resulting features. The Authors report that the system converts recognised labels to text and audio output using pyttsx3 and the Google Voice API, establishing a text-to-speech pipeline comparable to the gTTS-based output in GestureTalk. Published in Elsevier Array journal, this paper provides a relevant comparison point between traditional feature-engineering-based ISL recognition and the end-to-end deep learning landmark-based approach of GestureTalk. The limitations of SURF-based approaches including sensitivity to occlusion and illumination are discussed, motivating the transition toward skeleton landmark representations that provide inherently more robust features.

2.26 Kothadiya et al., 2022 [16] The Authors propose DeepSign, a deep learning-based Indian Sign Language detection and recognition system using LSTM and Gated Recurrent Unit models applied to isolated ISL video frames, systematically evaluating six architectural combinations to identify the optimal recurrent sequence model for ISL gesture classification. The custom IISL2020 dataset is introduced, comprising approximately 1,100 video samples per class across 11 ISL word categories recorded from 16 signers under natural conditions without green screen backgrounds, extra lighting, orientation correction, gloves, or sensors. The six evaluated architectures include single LSTM, double LSTM, single GRU, double GRU, GRU followed by LSTM, and LSTM followed by GRU. The LSTM followed by GRU combination achieves approximately 97% accuracy across 11 classes, outperforming all other configurations. This systematic evaluation provides empirically grounded guidance for recurrent architecture selection in ISL recognition systems.

2.27 Mouli et al., 2022 [21] The Authors propose a MediaPipe-optimised Gated Recurrent Unit model for Indian Sign Language recognition, improving the standard GRU gating mechanism by adjusting the update gate output using the reset gate to discard redundant past information more effectively. The GRU cell's activation functions are also replaced, substituting hyperbolic tangent with exponential linear unit in the candidate memory state to address vanishing gradient issues, and SoftMax with Softsign in the output layer to improve classification stability. The system is integrated with the MediaPipe Holistic pipeline for simultaneous face, hand, and body pose landmark extraction. The proposed model demonstrates faster convergence, higher recognition accuracy, and lower computational cost compared to standard GRU and LSTM baselines on the same ISL dataset. Published in Nature Scientific Reports in 2022, this paper represents a significant technical contribution to ISL-specific temporal sequence modelling, demonstrating that targeted modifications to the GRU gating architecture can provide meaningful performance improvements for ISL recognition. The work is relevant to GestureTalk as a reference for MediaPipe Holistic plus recurrent network ISL recognition and confirms that the MediaPipe-based landmark extraction combined with gated recurrent models is an effective and extensible approach for ISL gesture classification.

2.28 Camgoz et al., 2021 [6] The Authors propose Sign Language Transformers, an end-to-end neural architecture applying transformer self-attention to both the sign recognition and sign translation tasks, demonstrating that the transformer attention mechanism is highly effective for capturing long-range temporal dependencies in continuous sign language sequences. The model jointly learns to recognise individual signs and translate continuous signing into spoken language text, evaluated on the RWTH-PHOENIX-Weather 2014T benchmark for continuous German Sign Language. The transformer encoder processes sequences of visual sign features and the decoder generates target spoken language sentences, enabling end-to-end training of the full recognition-to-translation pipeline without requiring explicit sign segmentation. The Authors demonstrate that transformer-based sequence modelling significantly outperforms prior CNN-LSTM and CNN-HMM baselines on both the recognition and translation tasks, establishing a new state of the art on the PHOENIX benchmark.

2.29 Papastratis et al., 2021 [27] The Authors present a comprehensive survey of continuous sign language recognition approaches, systematically reviewing methods that address the full complexity of natural signing including co-articulation between consecutive signs, temporal segmentation without explicit sign boundaries, and the integration of manual and non-manual sign components. The survey covers the full methodological evolution from Hidden Markov Model-based systems through CNN-HMM hybrid architectures to fully end-to-end deep learning approaches using LSTM and transformer models, evaluating each on benchmark datasets including RWTH-PHOENIX, CSL, and BOBSL. A central finding is that end-to-end deep learning approaches using LSTM or transformer temporal encoders significantly outperform HMM-based systems on all major benchmarks, with the performance gap increasing as vocabulary size grows. The Authors identify the key remaining challenges including domain adaptation across different signing styles, scalability to large vocabularies, and the limited availability of annotated continuous sign language corpora for low-resource sign languages including Indian Sign Language. Published in IEEE Access in 2021, this survey is one of the most thorough recent reviews of continuous sign language recognition methodology and provides essential context for understanding GestureTalk's design choices.

2.30 Shenoy et al., 2021 [35] The Authors present a real-time Indian Sign Language recognition system that achieves practical deployment-level performance from a standard webcam without specialised hardware, targeting a vocabulary of ISL hand poses and dynamic gestures including alphabets, digits, and functional gesture-words relevant to everyday communication. The system employs a grid-based feature extraction approach that divides the segmented hand region into a configurable grid and computes binary occupancy features per cell to produce a compact feature vector. A machine learning classifier is trained on the resulting grid features, achieving real-time recognition performance on standard laptop hardware. The Authors collect a custom dataset of 24,624 images covering ISL digits, most alphabet letters, and 9 gesture-related hand poses. Published on arXiv, this paper provides an important pre-landmark-extraction reference that illustrates the evolution of ISL recognition from grid-based pixel features toward more robust and generalizable MediaPipe landmark representations. The system's sensitivity to hand segmentation quality under variable lighting conditions directly motivates the shift to skeleton-based landmark approaches employed in GestureTalk, which achieves far greater robustness to background and lighting variation without the need for reliable foreground segmentation.

2.31 Oudah et al., 2020 [26] The Authors present a comprehensive review of hand gesture recognition techniques based on computer vision, evaluating traditional and deep learning approaches across applications including sign language recognition, human-computer interaction, and robotic control. Three major input representations are compared: colour-based hand segmentation from RGB cameras, depth-image hand tracking from depth sensors, and skeleton-based landmark extraction. A key finding of this review is that skeleton-based representations — such as those produced by MediaPipe — provide the most robust and hardware-agnostic option for practical deployment, as they are invariant to background conditions, lighting variation, and skin colour differences. This directly validates the MediaPipe landmark approach in GestureTalk. The review also surveys deep learning classifiers including CNN, LSTM, and 3D-CNN architectures for gesture recognition, identifying LSTM as the preferred choice for dynamic gesture sequences due to its sequential temporal modelling capability. Published in MDPI Journal of Imaging, this review provides a comprehensive technical reference for the computer vision foundations of the GestureTalk feature extraction and classification pipeline, confirming that both the feature representation and classifier choices made in GestureTalk are well-supported by the broader gesture recognition literature.

2.32 Rastgoo et al., 2020 [29] The Authors present a video-based isolated hand sign language recognition system using a deep cascaded model in which a hand region detector first localises and crops the hand in each frame, followed by a deep neural network that classifies the isolated hand sign from the cropped region. This two-stage cascaded design reduces the influence of body context, background clutter, and non-hand regions by focusing the classifier exclusively on the hand area. The system is evaluated on RWTH-BOSTON-50 for American Sign Language and the Polish Sign Language Alphabet, demonstrating cross-language robustness. The deep neural network combines convolutional and recurrent layers to capture both spatial and temporal gesture features. The Authors discuss occlusion challenges during two-handed gestures and propose detection strategies to handle these cases. Published in Springer Multimedia Tools and Applications, this work anticipates the shift toward MediaPipe landmark-based feature extraction by demonstrating the value of isolating the hand representation from background and body context, though MediaPipe achieves this through structural skeleton coordinates rather than cropped pixel regions, providing greater invariance to lighting and background texture without requiring a dedicated hand region detector stage.

2.33 Wadhawan A. and Kumar P., 2020 [41] The Authors present a benchmarking study of CNN architectures for static sign language recognition, evaluating VGG16, ResNet50, InceptionV3, and MobileNetV2 through transfer learning on a curated dataset of static ISL hand gesture images. The study assesses each architecture in terms of recognition accuracy, training convergence speed, and computational requirements, providing guidance for architecture selection in deployment-constrained sign language applications. MobileNetV2 is identified as optimal for resource-constrained deployment due to its favourable accuracy-to-parameter ratio, while VGG16 and ResNet50 achieve higher accuracy at greater computational cost. A central finding of this paper is the fundamental limitation of all static image classification approaches: their inability to capture the temporal dynamics of dynamic gestures that convey meaning through motion patterns across frames rather than a single static configuration.

2.34 Zhang et al., 2020 [42] The Authors present MediaPipe Hands, a real-time on-device hand tracking pipeline that predicts a 21-point 3D hand skeleton from a single RGB camera for AR and VR applications, implemented via the MediaPipe cross-platform machine learning framework. The pipeline consists of two models: a palm detector that identifies bounding boxes of hands in the full image frame, and a hand landmark model that regresses precise 3D coordinates of 21 anatomical keypoints from the cropped palm region. The palm detector is trained on over 116,000 images spanning in-the-wild samples, in-house collected images, and synthetic hand renders, achieving robust detection across diverse hand poses and distances up to 3 metres. The 21 landmarks encode the full configuration of the hand skeleton including finger spread, joint flexion, and hand orientation in three dimensions. For two hands simultaneously, as used in GestureTalk, this yields 126 features per frame. MediaPipe Hands is open-sourced and has become the most widely adopted hand tracking framework in sign language recognition research. Published on arXiv in 2020 by Google Research, this is the foundational technical reference for GestureTalk's feature extraction module, which uses MediaPipe Hands to extract and normalise 21 keypoints per hand for LSTM gesture classification.

2.35 Lugaresi et al., 2019 [18] The Authors present MediaPipe, a general-purpose framework for building cross-platform machine learning perception pipelines, designed for real-time media processing tasks including video analysis, hand tracking, face detection, and pose estimation on mobile and desktop devices. The framework models perception pipelines as directed graphs of modular processing components called Calculators, enabling flexible composition of preprocessing, model inference, and post-processing steps into complete perception applications. MediaPipe supports TensorFlow Lite model inference with GPU acceleration on Android and iOS and CPU inference on desktops. Pre-built solutions include hand landmark detection, full-body pose estimation, and face mesh modelling, all accessible through a unified API. Presented at the Workshop on Computer Vision for AR/VR at CVPR 2019, this is the foundational technical reference for the MediaPipe framework architecture underlying all MediaPipe-based sign language systems including GestureTalk. Understanding the MediaPipe framework is essential for appreciating how GestureTalk achieves real-time cross-platform hand landmark extraction from a standard webcam without requiring a GPU, enabling accessibility for users on standard consumer hardware without technical expertise.

2.36 Sruthi C. J. and Lijiya A., 2019 [37] The Authors present SigNet, a deep learning-based Indian Sign Language recognition system using a convolutional neural network trained on ISL gesture images, representing one of the early published deep learning approaches specifically targeting ISL. The system processes individual hand gesture images through a CNN to produce classification outputs for ISL alphabet and word categories without requiring specialised hardware or data gloves. A custom ISL dataset is collected and SigNet is evaluated on both static ISL alphabets and ISL word gestures. Presented at the 2019 International Conference on Communication and Signal Processing, this paper is an early ISL-specific deep learning contribution that discusses challenges unique to ISL including hand orientation conventions, regional dialect variation, and the scarcity of annotated ISL training data. The work is important historical context for understanding the progression from early CNN static classifiers for ISL to the LSTM temporal sequence approach in GestureTalk

2.37 Bantupalli K. and Xie Y., 2018 [3] The Authors present a vision-based ASL recognition system using deep learning and computer vision from standard webcam input, combining background subtraction and skin colour segmentation for hand isolation with CNN for feature extraction and a dense classification network, targeting real-time ASL alphabet recognition without specialised hardware. The system is evaluated at the 2018 IEEE International Conference on Big Data and demonstrates real-time performance on consumer-grade hardware. Background subtraction eliminates static background regions while skin colour segmentation in HSV colour space isolates the hand, and cropped hand images are classified by a CNN. The Authors discuss limitations of skin-based segmentation including sensitivity to lighting variation and background colour similarity with skin tones. This paper is significant as an early demonstration of the vision-based hardware-agnostic sign language recognition paradigm that systems including GestureTalk have built upon, progressively replacing pixel-based hand segmentation with skeletal landmark extraction via MediaPipe to overcome the lighting and background sensitivity limitations identified here. Published in IEEE Xplore, this work establishes the conceptual lineage from which landmark-based approaches represent a direct technical advancement.

2.38 Cui et al., 2017 [7] The Authors present Recurrent Convolutional Neural Networks for continuous sign language recognition, proposing a staged optimisation strategy that jointly trains a CNN spatial feature extractor and a temporal RNN sequence model to achieve state-of-the-art accuracy on the RWTH-PHOENIX-Weather 2014 benchmark for continuous German Sign Language. The staged optimisation strategy addresses the difficulty of jointly training deep convolutional and recurrent components from random initialisation by proposing a curriculum learning approach: the CNN is first pre-trained on isolated sign images, then the full RCNN is fine-tuned on continuous signing sequences. This curriculum strategy produces significantly better-converging models than end-to-end training from scratch. Presented at the IEEE Conference on Computer Vision and Pattern Recognition, this paper provides a key technical contribution to continuous sign language recognition and represents an important step in the lineage from early HMM-based systems toward fully end-to-end deep learning approaches.

2.39 Koller et al., 2016 [15] The Authors present Deep Sign, a hybrid CNN-HMM architecture for continuous sign language recognition combining CNN visual feature learning with Hidden Markov Model temporal sequence decoding, evaluated on the RWTH-PHOENIX-Weather 2014 German Sign Language benchmark. The CNN replaces the manually engineered feature extraction pipelines of prior HMM systems, learning discriminative spatial features from image frames directly. The HMM provides the temporal sequence decoding framework for continuous signing streams at sentence level. This hybrid architecture represents a key transitional approach between the pre-deep-learning era of HMM sign recognition and the fully end-to-end LSTM era. Presented at the British Machine Vision Conference, the paper identifies key challenges in continuous sign recognition including temporal segmentation, co-articulation between adjacent signs, and spatial grammar modelling. This work contextualises why fully recurrent architectures such as LSTM represent an advancement over hybrid CNN-HMM approaches by modelling temporal dynamics end-to-end without the conditional independence assumptions embedded in HMM transition models.

2.40 Rautaray S. S. and Agrawal A., 2015 [30] The Authors present a comprehensive survey of vision-based hand gesture recognition methods, covering skin colour segmentation, background subtraction, optical flow, depth sensing, and machine learning classification approaches, identifying the key technical challenges that limit real-world deployment of gesture recognition systems. The survey categorises approaches by input modality including standard RGB cameras, depth sensors, and skeleton tracking devices, and reviews classifiers including SVM, HMM, ANN, and early CNN models. Key challenges identified include sensitivity to illumination changes, background complexity, occlusion, and the need for real-time processing. The Authors discuss the fundamental trade-off between recognition accuracy and hardware requirements, noting that the most accurate systems at the time relied on expensive depth sensors or wearable devices that were inaccessible to general users. Published in Springer Artificial Intelligence Review, this paper is an important pre-deep-learning survey that contextualises the technical landscape into which MediaPipe and deep learning LSTM systems entered.

2.41 Mitra S. and Acharya T., 2007 [20] The Authors present a comprehensive survey of gesture recognition systems published in IEEE Transactions on Systems, Man, and Cybernetics, covering approaches from rule-based models to early neural classifiers and providing a foundational taxonomy of gesture recognition methodology that remains relevant to contemporary deep learning systems. The survey categorises gestures into static postures, dynamic gestures, and body language, reviewing computational approaches including Hidden Markov Models, template matching, Dynamic Time Warping, and feedforward neural networks for each category. Core technical challenges discussed include spatial variability of hand configurations across users, temporal variability of gesture duration and speed, gesture boundary segmentation in continuous motion, and real-time performance requirements on available hardware. Published in IEEE Transactions on Systems, Man, and Cybernetics in 2007, this is one of the most widely cited surveys in gesture recognition and sign language research. Its taxonomy of challenges maps directly to the design criteria that motivated GestureTalk: MediaPipe addresses sensor independence and background robustness, LSTM temporal modelling addresses gesture dynamics, majority-vote smoothing addresses segmentation noise, and confidence thresholding addresses classification stability.

2.42 Hochreiter S. and Schmidhuber J., 1997 [9] The Authors introduce Long Short-Term Memory networks, a recurrent neural network architecture designed to address the vanishing gradient problem that prevents standard RNNs from learning long-range temporal dependencies in sequential data. The LSTM architecture introduces a gated memory cell mechanism with three learnable gates: the input gate controlling new information flow into the cell state, the forget gate determining which stored information to discard, and the output gate controlling which information to expose as the hidden state. This gating mechanism enables selective retention of relevant information over arbitrarily long sequences while discarding irrelevant context, allowing effective gradient flow during backpropagation through time and enabling the model to learn dependencies across hundreds of time steps. The Authors demonstrate LSTM effectiveness across sequential learning benchmarks. Published in Neural Computation by MIT Press in 1997 and now among the most cited papers in machine learning, this is the foundational reference for all LSTM-based systems.

CHAPTER 3 : RESEARCH METHODOLOGY

3.1 Research Design

The research underpinning GestureTalk is classified as applied and experimental in nature. Rather than producing purely theoretical insights, the project is oriented entirely toward the design, development, and empirical evaluation of a functional system that addresses a real-world accessibility problem: the absence of an affordable, hardware-free tool for real-time Indian Sign Language recognition and multilingual communication. The methodology follows the applied machine learning paradigm identify a real-world problem, collect domain-specific data, train a suitable machine learning model, and evaluate its performance empirically, and deploy it within a practical application environment.

GestureTalk follows a sequential, phase-based research process. While there is an iterative feedback loop between data collection, model training, and evaluation, the overall flow proceeds through seven clearly defined stages:

- Problem Scoping and Literature Review: Identifying the gap in real-time ISL accessibility tools and reviewing related work in landmark-based gesture recognition, LSTM temporal modelling, MediaPipe hand tracking, and multilingual TTS pipelines.
- Dataset Design and Collection: Designing the 28-class ISL gesture vocabulary, building the Python data collection script, and capturing 940 landmark sequences under varied environmental conditions.
- Data Preprocessing and Feature Engineering: Applying wrist-relative landmark normalization, constructing the (940, 20, 126) NumPy dataset array, and generating one-hot encoded label vectors.
- Model Architecture Selection and Training: Designing and training the two-layer stacked LSTM architecture with Batch Normalization and Dropout, with EarlyStopping regularization.
- System Integration and Deployment: Integrating the trained LSTM model into a Flask web application with real-time MJPEG streaming, confidence thresholding, dual-smoothing inference, multilingual translation, and gTTS speech synthesis.
- Evaluation and Testing: Measuring recognition accuracy, per-class precision, recall, and F1-score on the held-out 20% test set and testing end-to-end system performance.
- Documentation and Reporting: Presenting the design, implementation, and findings in this structured project report.

A critical design decision in this research is the exclusive reliance on a custom-built dataset rather than any pre-existing sign language corpus. This is justified by the specific focus on Indian Sign Language, which differs substantially from ASL, BSL, and other sign languages that are more commonly represented in publicly available datasets. No sufficiently large, standardized, publicly available ISL dataset existed at the time of this project's development that suited the specific gesture set and deployment context targeted by GestureTalk. Consequently, original data collection was both necessary and methodologically appropriate.

The project also adopts a hardware-agnostic design philosophy throughout: the entire system must operate using only a standard laptop or desktop computer with a built-in

or external webcam, without requiring depth sensors, data gloves, motion capture equipment, or any proprietary hardware. This constraint directly shapes every major methodological choice, including the selection of MediaPipe Hands for vision-based landmark extraction and the decision to serve the system as a browser-accessible Flask web application.

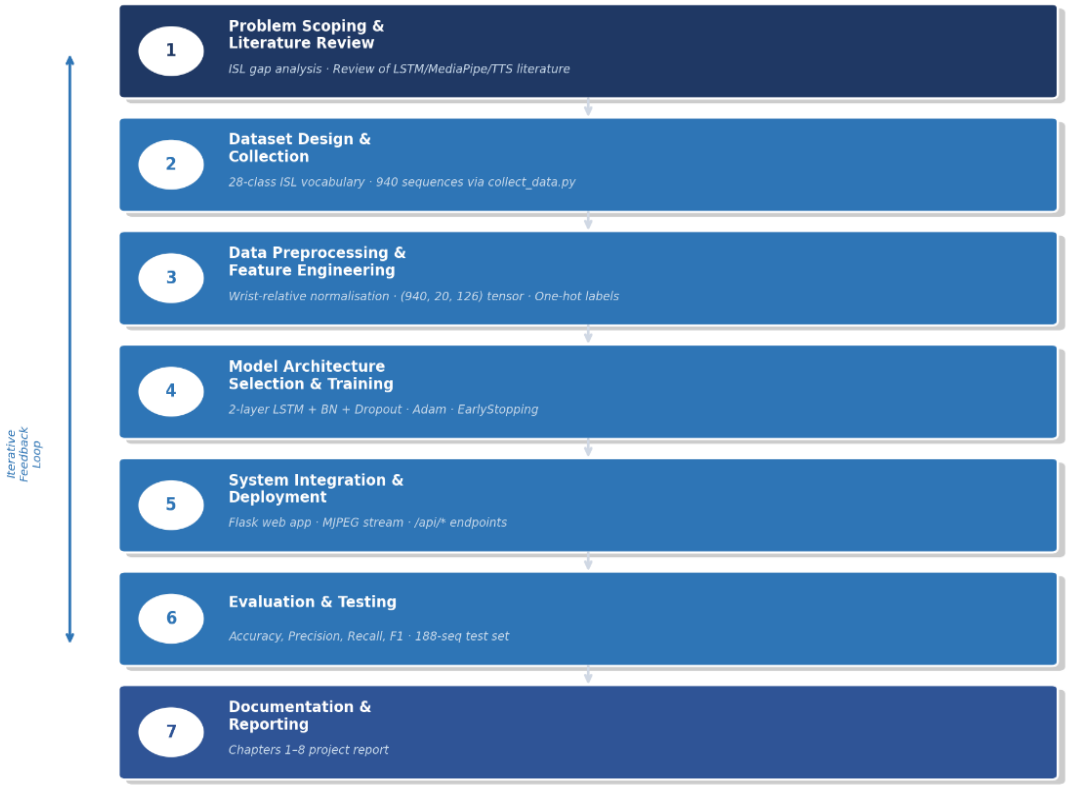


Figure 3.1: GestureTalk Research Methodology Flowchart

3.2 Hypothesis Formulation

This section outlines the central hypothesis that guides the development and evaluation of GestureTalk. The hypothesis is framed to test whether a landmark-based LSTM gesture recognition system, combined with multilingual translation and text-to-speech synthesis, can function as an effective and practical accessibility tool for the Indian deaf community without requiring any specialized hardware beyond a standard webcam.

Null Hypothesis (H_0):

An LSTM neural network trained on MediaPipe hand landmark sequences does not achieve sufficient recognition accuracy or inference latency to function as a practical, real-time Indian Sign Language recognition tool, and the integration of translation and speech output does not produce a practical assistive communication system on standard consumer hardware.

Alternative Hypothesis (H_1):

A Long Short-Term Memory (LSTM) neural network trained on sequences of MediaPipe hand landmark features extracted from live webcam video can achieve sufficient accuracy and latency for practical, real-time Indian Sign Language

recognition, and when integrated with multilingual translation and text-to-speech components, can serve as an effective, hardware-free assistive communication tool for the Indian deaf community.

This hypothesis contains several individually testable sub-components, each of which is addressed through a specific aspect of the methodology described in this chapter and evaluated in Chapter 6:

- Accuracy Component: Can an LSTM trained on 126-dimensional landmark sequences achieve ≥90% validation accuracy across all 28 ISL gesture classes on a stratified 20% held-out test set?
- Latency Component: Can the end-to-end inference pipeline (webcam frame capture → MediaPipe landmark extraction → LSTM prediction → dual-smoothing → display) operate at sufficient speed to feel real-time to a human user, with a prediction update rate of at least 5–10 Hz?
- Robustness Component: Can the system maintain stable, correct predictions across users with different hand sizes, skin tones, lighting conditions, and camera angles without any user-specific calibration?
- Integration Component: Can the gesture recognition pipeline be seamlessly integrated with multilingual translation (deep-translator / Google Translate API) and text-to-speech (gTTS) to form a complete end-to-end assistive communication system?
- Hardware-Free Component: Can all of the above be achieved using only a standard webcam and a CPU-only consumer machine, without any depth cameras, wearable sensors, or GPU acceleration?

These sub-hypotheses are evaluated through the empirical results of model training in Chapter 5 and the results analysis in Chapter 6. The methodology described in the remainder of this chapter is specifically constructed to generate the data, model, and pipeline necessary for that evaluation.

3.3 Objectives Aligned with Hypothesis

This section establishes a direct and logical connection between the research objectives of GestureTalk and the hypotheses formulated in Section 3.2. The null hypothesis (H_0) posits that the LSTM landmark-based approach does not deliver practical recognition or usability. The alternative hypothesis (H_1) asserts that it does, through the specific combination of MediaPipe landmark features, LSTM temporal modelling, dual-prediction smoothing, Flask deployment, multilingual translation, and gTTS speech synthesis. Each objective is designed to operationalize H_1, providing measurable pathways to reject H_0.

3.3.1 General Objective

The general objective of GestureTalk is to design, develop, and empirically validate a real-time Indian Sign Language recognition and multilingual translation system that operates on standard consumer hardware, using a custom-trained LSTM neural network fed by MediaPipe hand landmark features, and delivers detected sign language gestures as translated, spoken-language output through a web-based interface accessible in any modern browser.

This overarching goal directly challenges H_0 by positioning GestureTalk as a comprehensive, integrated system that addresses the full communication accessibility pipeline: from hand gesture in front of a webcam to multilingual audio output. The

emphasis on "standard consumer hardware" specifically tests the hardware-free sub-hypothesis, while "empirical validation" ensures that accuracy claims are substantiated through measurable, reproducible experiments rather than anecdotal observation.

3.3.2 Specific Objectives and Their Hypothesis Alignment

The following specific objectives break down the general goal into granular, measurable components, each linked to a sub-hypothesis of H_1 with a defined success criterion:

Specific Objective	Hypothesis Addressed	Success Criterion
Build and validate a custom ISL gesture dataset covering 28 classes (940 sequences)	Accuracy + Robustness	940 sequences, stratified 80/20 split, balanced class distribution (28–60 sequences per class)
Train an LSTM model achieving ≥90% validation accuracy on the held-out test set	Accuracy Component	Validation accuracy ≥90%, Macro F1-Score ≥0.90 on 188 held-out test sequences
Implement real-time inference with sub-second per-prediction latency	Latency Component	Real-time prediction speed ≥5 predictions per second in live webcam mode on standard laptop CPU
Apply confidence thresholding (40%) and dual-smoothing (mean history + majority vote) for stable predictions	Robustness Component	No flickering output; stable, consistent predictions across diverse users and lighting conditions
Integrate multilingual translation into 20+ languages via deep-translator	Integration Component	Correct translation output for all supported languages via Google Translate API wrapper
Deliver text-to-speech audio output in the translated target language via gTTS	Integration Component	Audible, accurate gTTS-generated .mp3 audio playback in target language within 2 seconds
Deploy as a web application accessible on standard hardware via Flask	Hardware-Free Component	System runs on laptop CPU with built-in webcam, no GPU, no special hardware required

Table 3.1: Specific Objectives Aligned with Research Hypothesis Components

- Dataset Construction (Accuracy + Robustness): Collecting 940 gesture sequences across 28 ISL classes under varied lighting, distances, and hand sizes directly counters H_0 by producing a realistic, generalisable training corpus. The stratified 80/20 split ensures that evaluation metrics reflect genuine out-of-sample performance.
- LSTM Model Training (Accuracy Component): Training the two-layer stacked LSTM with Batch Normalization and Dropout to achieve ≥90% validation accuracy directly tests the accuracy sub-hypothesis. Success provides measurable evidence for H_1 and demonstrates that landmark sequences carry sufficient information for reliable ISL classification.
- Real-Time Inference (Latency Component): Implementing a sliding 20-frame window inference loop with MediaPipe landmark extraction inside Flask's generate_frames() MJPEG stream tests the latency sub-hypothesis. Meeting the ≥5 Hz update rate threshold confirms that CPU-only operation is viable for real-time ISL recognition.
- Dual-Prediction Smoothing (Robustness Component): The dual-smoothing pipeline (averaging over prediction_history deque + majority vote over prediction_queue deque) with a 40% confidence threshold directly tests the robustness sub-hypothesis by suppressing flickering and false detections across diverse users.
- Multilingual Translation (Integration Component): Integrating deep-translator's GoogleTranslator into the /api/translate Flask route tests one dimension of the integration sub-hypothesis: converting recognised ISL sign sequences into text in any of 20+ target languages.
- Text-to-Speech Output (Integration Component): Integrating gTTS into the /api/speak Flask route tests the second dimension of the integration sub-hypothesis: converting translated text into spoken audio and delivering it as a downloadable .mp3 file to the browser.
- Web Deployment (Hardware-Free Component): Deploying the entire pipeline MediaPipe, LSTM inference, Flask server, translation, TTS as a single-process Python application accessible through a browser directly tests the hardware-free sub-hypothesis, confirming that no specialist hardware is required.

3.3.3 Methodological Rigor and Research Orientation

GestureTalk is classified as applied, empirical, and design-science research. It is applied because it addresses a specific, real-world communication accessibility problem for the Indian deaf community. It is empirical because all claims about model performance are substantiated through measurable, reproducible experiments using a held-out validation set that is never seen during training. It is design-science research because it produces a functional artefact the GestureTalk web application evaluated against pre-defined performance criteria derived from the research hypothesis.

Quantitative rigor is maintained through a strict stratified train-test split (80% training / 20% test), categorical cross-entropy as the loss function with a softmax output layer for calibrated multi-class probability estimation, EarlyStopping to prevent overfitting, and the use of macro-averaged precision, recall, and F1-score as evaluation metrics that treat all 28 classes equally regardless of sequence count. These methodological standards ensure that reported metrics are meaningful indicators of generalisation ability rather than artefacts of overfitting or dataset imbalance.

3.4 Dataset Used

The dataset used to train the GestureTalk LSTM model is a fully custom-built collection of ISL gesture landmark sequences, created through a structured data collection procedure developed specifically for this project. No third-party or pre-existing ISL dataset was used or adapted. This section documents the structure, scope, collection procedure, and preprocessing of the dataset in full.

3.4.1 Type of Data Employed in GestureTalk

The GestureTalk dataset consists of skeletal landmark feature sequences rather than raw video footage or image frames. Each data sample is a three-dimensional NumPy array of shape (20, 126), representing a temporal sequence of 20 consecutive frames with 126 spatial features per frame. This design decision has four important implications:

- Privacy: No identifiable video of data collectors is stored. Only numerical coordinate arrays are saved to disk, so no personal visual data is retained beyond the collection session.
- Efficiency: Landmark arrays are extremely compact compared to raw video — each (20, 126) sequence occupies approximately 20 KB, versus several megabytes for equivalent raw video. This makes training and inference significantly faster.
- Robustness: By abstracting gesture information into normalized skeletal representations, the model is decoupled from appearance-level variations such as skin tone, clothing, background texture, and lighting conditions that would confound a pixel-level classifier.
- Pipeline Consistency: The same MediaPipe Hands landmark extraction pipeline used during data collection is used identically during live inference, guaranteeing perfect input consistency between training and deployment. There is no distribution shift between training data and inference inputs.

The 126 features per frame are derived as follows: MediaPipe Hands detects 21 anatomical keypoints per hand, each providing x, y, and z coordinates. For two hands simultaneously, this yields 21 × 3 × 2 = 126 features per frame. When one or both hands are absent from a given frame, the corresponding 63-feature block is zero-filled,

ensuring a consistent input dimensionality across all samples regardless of whether one or two hands are visible.

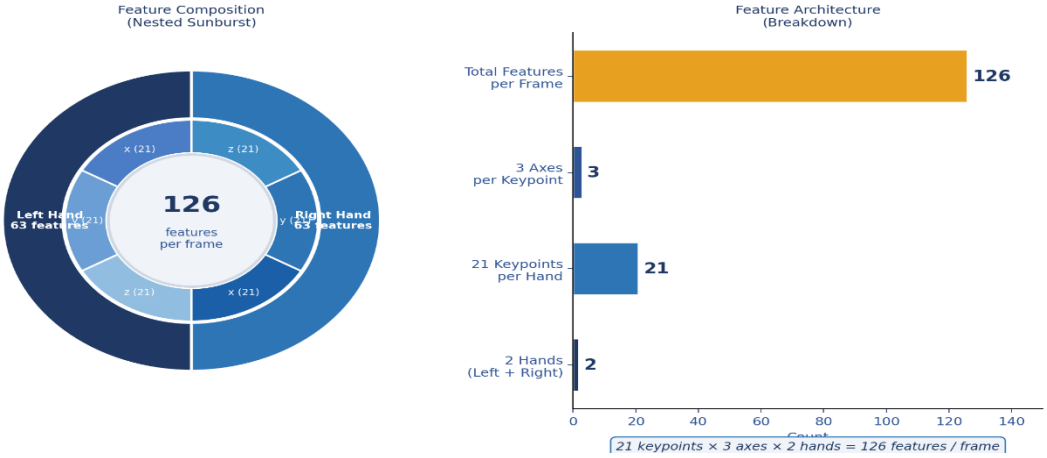


Figure 3.2: Feature Vector Composition – 126 Features per Frame.

3.4.2 Data Sources and Collection Procedure

- All gesture sequences were collected using a purpose-built Python data collection script (collect_data.py) that activates a live webcam feed via OpenCV, runs MediaPipe Hands landmark detection on each frame in real time, and records the extracted 126-feature landmark array across 20 consecutive frames as a single gesture sequence. The collection script imposes a brief 2-second delay between consecutive sequences to ensure clean, unambiguous recordings with natural start and end hand positions between each capture. (21 Keypoints × 3 Axes × 2 Hands)
- The collection environment was deliberately varied to introduce robustness into the training data. Sequences were captured under different ambient lighting conditions (natural daylight, artificial indoor fluorescent lighting, and dim conditions), at slightly varying camera distances (30–70 cm), and by multiple collectors with different hand sizes and skin tones. This environmental diversity is critical for ensuring that the trained model generalises beyond the specific conditions of a single recording session and performs correctly for diverse end users.
- The dataset covers 28 gesture classes. Twenty-seven of these represent meaningful ISL gestures drawn from everyday communicative vocabulary greetings, pronouns, affirmations, negations, basic needs, questions, and social phrases. The twenty-eighth class, designated NO_SIGN, represents the deliberate absence of any gesture: idle hand positions, transitional movements between signs, or partial captures at the boundaries of a signing sequence. The NO_SIGN class is essential for suppressing false-positive predictions during live inference when the user is not actively producing a recognisable sign.

Dataset Parameter	Value
Total Gesture Classes	28 (27 ISL signs + 1 NO_SIGN sentinel class)
Total Sequences	940
Frames per Sequence	20 consecutive frames
Features per Frame	126 (21 keypoints × 3 axes × 2 hands)
Input Shape per Sample	(20, 126)
Sequences per Class	28–60 (varies; WHERE=50, TO_MEET=60, majority=32)
Data Format	.npy (NumPy binary arrays) + labels.csv
Training Sequences (80%)	752 sequences

Dataset Parameter	Value
Test Sequences (20%)	188 sequences
Splitting Strategy	Stratified random split (random_state=42)
Collection Framework	MediaPipe Hands + OpenCV (cv2)
Collection Environment	Varied lighting, distances, hand sizes
Data Collector Count	Primary collector with controlled variation in recording conditions.

Table 3.2: GestureTalk Custom ISL Dataset – Summary Statistics

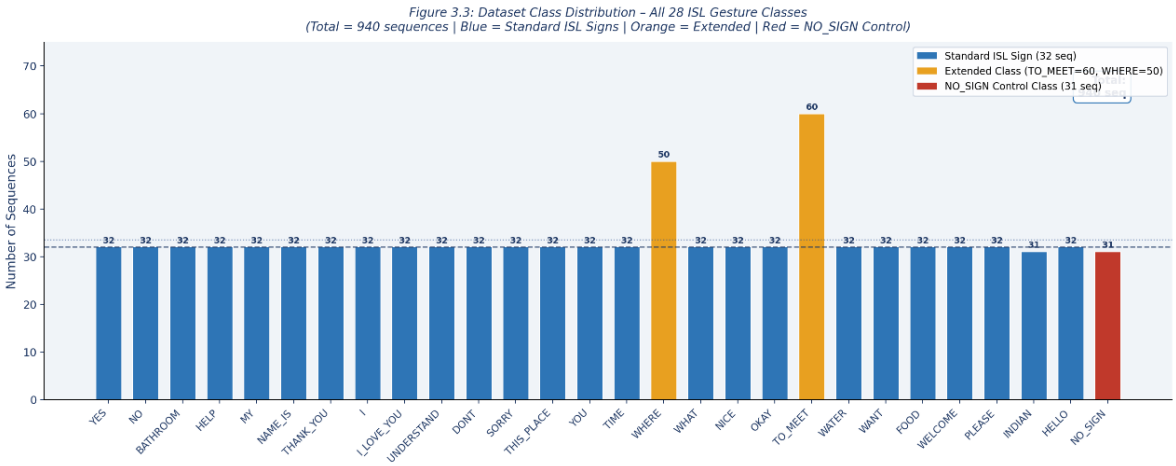


Figure 3.3: Dataset Class Distribution

3.4.3 Landmark Normalization

- Raw MediaPipe landmark coordinates are provided in a normalized image-space coordinate system, where (0,0) is the top-left corner of the camera frame and (1,1) is the bottom-right. However, these raw coordinates remain sensitive to hand position within the frame: the same gesture performed at the top-left versus the centre of the screen would yield different absolute coordinates, causing the model to learn position-dependent rather than shape-dependent features.

- To eliminate this positional sensitivity, GestureTalk applies wrist-relative normalization in the extract_landmarks() function inside app.py. For each hand, the wrist keypoint (landmark index 0) coordinates are subtracted from all 21 keypoints in that hand. This transforms the coordinate system so that the wrist is always at the origin (0, 0, 0), and all other keypoints are expressed as positional displacements relative to the wrist. The coordinate array is then divided by the Euclidean norm of the middle finger MCP keypoint (index 12) x and y coordinates to produce a scale-normalized representation, making the features invariant to hand size and camera distance as well.

- This normalization is applied identically during both data collection (collect_data.py) and live inference (app.py → extract_landmarks()), ensuring complete consistency between the training distribution and inference inputs. The result is a model that recognises the shape and configuration of a gesture independently of where in the frame the hand is positioned, how far the hand is from the camera, and how large the hand is relative to the frame.

3.4.4 NO_SIGN Class and False-Detection Suppression

- A dedicated NO_SIGN class is included in the dataset to address a fundamental challenge in real-time gesture recognition: the system must not attempt to classify every 20-frame window as a meaningful sign. In practice, users will frequently have their hands in transitional, resting, or ambiguous positions between intentional signs. Without a NO_SIGN class, the LSTM model would be forced to assign every 20-frame sequence to one of the 27 ISL gesture classes, producing a continuous stream of incorrect and spurious detections.
- By including 28–35 sequences of non-signing hand positions idle rest positions, partial hand lifts, transitional movements, and background noise frames the model learns to distinguish meaningful, complete gestures from non-signing states. During live inference, predictions assigned to the NO_SIGN class are suppressed from the sentence builder and treated as non-events. The confidence threshold (CONF_THRESHOLD = 0.40) provides a second layer of suppression: even if the model assigns a real gesture class, predictions below 40% confidence are discarded. Together, these two mechanisms substantially improve the usability and output quality of the real-time system by preventing spurious word additions when the user is resting or transitioning between signs.

3.4.5 Data Splitting Protocol

- The collected dataset of 940 sequences is split into training and test subsets using a stratified random split implemented via scikit-learn's train_test_split with test_size=0.2 and random_state=42. The stratification argument ensures that the class proportions in both subsets match the overall class distribution: each of the 28 classes contributes approximately 80% of its sequences to training and 20% to the test set.
- This stratification is critical for the smaller-count classes (INDIAN and NO_SIGN, each with 31 sequences) where a purely random split could accidentally under-represent a class in the test set, producing unreliable per-class metrics. The test set of 188 sequences is held out entirely during model training and is used exclusively for performance evaluation in Chapter 6, ensuring that all reported accuracy and F1-score metrics reflect genuine generalisation to unseen data rather than memorisation of the training set.

3.5 Algorithms / Models Applied

The core machine learning algorithm employed in GestureTalk is the Long Short-Term Memory (LSTM) neural network. This section provides a theoretical grounding for the LSTM architecture, explains in detail why it is the appropriate choice for ISL gesture recognition, presents the specific model architecture used in GestureTalk with a layer-by-layer specification, and describes the complete training algorithm and hyperparameter configuration.

3.5.1 Why LSTM for Indian Sign Language Recognition

Indian Sign Language gestures are temporal in nature and unfold across multiple video frames. A single ISL sign , for example, the sign for THANK_YOU is not defined by any single static frame; it is defined by a characteristic movement pattern that unfolds over a sequence of frames. The meaning is encoded in the trajectory, speed, and configuration changes of the hand across time. This temporal dimension is precisely what distinguishes sign language recognition from static image classification.

Standard feedforward neural networks, including Convolutional Neural Networks operating on individual images, are not designed to capture temporal dependency. They process each input independently without any built-in mechanism for relating current inputs to previous ones. Recurrent Neural Networks (RNNs) address this limitation by maintaining a hidden state carrying information from previous time steps, but vanilla RNNs suffer from the vanishing gradient problem, which makes it practically impossible to learn dependencies across sequences longer than a few time steps.

Long Short-Term Memory networks (Hochreiter and Schmidhuber, 1997) solve the vanishing gradient problem through a carefully designed gating mechanism. Each LSTM cell contains three trainable gates alongside a cell state that acts as a long-term memory buffer. The forget gate decides what information from the previous cell state to discard. The input gate determines what new information from the current input to incorporate into the cell state. The output gate controls what information from the cell state to expose as the hidden state output at this time step. Together, these gates allow the LSTM to selectively retain, update, and suppress information as the sequence progresses, enabling it to maintain relevant context across 20 or more frames without gradient degradation.

For GestureTalk, this means the LSTM can track the complete trajectory of a gesture how the hand opens, moves, configures its fingers, and returns to rest across the entire 20-frame sequence, and use the full temporal pattern to make its classification decision. A static CNN applied to a single frame would see only the hand configuration at one moment in time and would be unable to distinguish gestures that pass through similar intermediate configurations. The LSTM's sequential memory is what makes accurate dynamic gesture recognition possible.

Alternatives such as 3D-CNNs (which can also capture temporal patterns from raw video) were considered but rejected because they require raw video frames as input, significantly increasing computational cost and eliminating the privacy and efficiency benefits of the landmark-based approach. Transformer-based architectures (see Camgoz et al., 2021, Section 2.5) were also considered but were deemed unnecessary for a 28-class isolated gesture classification task with 20-frame sequences, where the LSTM's recurrent inductive bias for temporal sequences is well-suited and computationally far more efficient.

3.5.2 GestureTalk LSTM Model Architecture

The GestureTalk LSTM model is a sequential deep learning architecture defined in train_lstm_model.py and comprising eight layers in the following order:

	Layer Type	Configuration	Purpose
1	**LSTM**	128 units, return_sequences=True, input_shape=(20, 126)	Extract temporal features from the full 20-frame input sequence; pass hidden state at every time step to next layer
2	**Batch Normalization**	Default parameters	Normalize layer activations; accelerate training convergence; reduce internal covariate shift between batches
3	**Dropout**	rate = 0.3	Regularization: randomly zero 30% of units per batch during training to prevent co-adaptation and overfitting
4	**LSTM**	128 units, return_sequences=False	Further temporal abstraction; collapse the full temporal sequence into a single fixed-length 128-dimensional vector representing the entire gesture
5	**Batch Normalization**	Default parameters	Second normalization stage for stable gradient flow into the Dense layers
6	**Dropout**	rate = 0.3	Second regularization stage; prevents the fixed-length LSTM output vector from overfitting to training examples
7	**Dense**	128 units, activation = ReLU	Non-linear transformation to compress the LSTM output representation into a classification-ready embedding
8	**Dense (Output)**	28 units, activation = Softmax	Output class probability distribution across all 28 ISL gesture classes; values sum to 1.0

Table 3.3: GestureTalk LSTM Model Architecture

The input to the model is a tensor of shape (batch_size, 20, 126), where 20 is the sequence length (frames) and 126 is the feature dimensionality. The first LSTM layer processes this sequence with return_sequences=True, outputting a hidden state at every time step so that the full temporal information is available to the Batch Normalization and Dropout layers before reaching the second LSTM. The second LSTM uses

return_sequences=False, collapsing the temporal dimension into a single 128-dimensional vector that represents the entire gesture. This vector passes through two Dense layers before the Softmax output, which produces a probability distribution across all 28 classes.

The Softmax activation converts raw output logits into a calibrated probability distribution summing to exactly 1.0. During inference, the class with the highest probability is selected as the prediction, and this maximum probability value is used as the confidence score that drives the 40% confidence threshold filter in app.py.

3.5.3 Training Algorithm and Hyperparameter Configuration

The model is compiled and trained using the following configuration in train_lstm_model.py:

Hyperparameter	Value / Configuration
Optimizer	Adam (Adaptive Moment Estimation), lr=0.001 (default)
Loss Function	Categorical Cross-Entropy
Batch Size	32 sequences per gradient update
Maximum Epochs	60 (with EarlyStopping to terminate earlier)
EarlyStopping Monitor	Validation Loss (monitor='val_loss')
EarlyStopping Patience	10 epochs; restore_best_weights=True
Train / Test Split	80% Training (752 seq) / 20% Test (188 seq), stratified, random_state=42
Label Encoding	One-Hot Encoding via to_categorical() — 28-dimensional binary vectors
Dropout Rate	0.3 per Dropout layer (applied after each LSTM layer)
Metrics Monitored	Categorical Accuracy (Training + Validation)

Hyperparameter	Value / Configuration
Framework	TensorFlow 2.16.2 with Keras API
Model Save Format	isl_lstm_model.keras + labels.npy (NumPy label array)

Table 3.4: GestureTalk Model Training Hyperparameter Configuration

The Adam optimizer is selected for its adaptive learning rate behaviour: it adjusts the effective learning rate per parameter based on historical gradient information, making it highly effective for training LSTM networks on relatively small, imbalanced datasets where gradient magnitudes vary substantially across parameters and layers. Categorical Cross-Entropy is the standard loss function for multi-class classification with one-hot encoded labels, penalising the model proportionally to the confidence with which it makes incorrect predictions and incentivising well-calibrated probability estimates.

EarlyStopping with patience=10 ensures training terminates when the validation loss stops improving for 10 consecutive epochs, preventing overfitting and reducing unnecessary computation. The restore_best_weights=True parameter ensures that the saved model weights correspond to the epoch of best validation performance rather than the final epoch, which may have slightly overfitted. In practice, the model typically reaches early stopping between epoch 40 and 55.

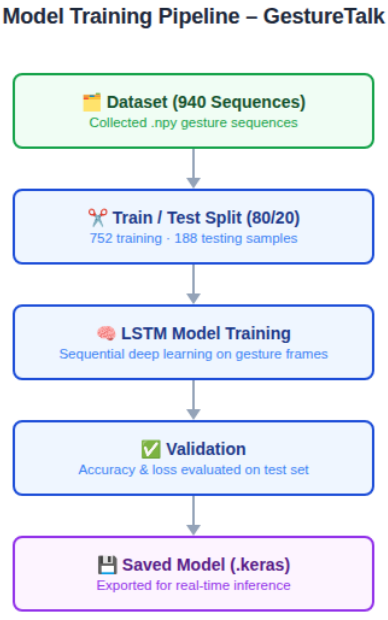


Figure 3.4: GestureTalk Model Training Pipeline

3.5.4 Real-Time Inference Protocol

During live inference inside Flask's generate_frames() function, the trained model operates on a rolling buffer of the 20 most recent MediaPipe-processed frames implemented as a collections.deque(maxlen=20). As each new webcam frame arrives (at every second frame via frame_counter % 2 == 0 for efficiency) to reduce computational load, the 126-feature landmark vector is appended to the deque and the oldest frame is automatically discarded. Once the deque contains exactly 20 frames, the stacked (20, 126) NumPy array is passed to the LSTM model for prediction.

Two sequential filtering mechanisms are applied to raw model outputs before any result is displayed:

- Confidence Threshold Filter (CONF_THRESHOLD = 0.40): If the maximum Softmax probability for any prediction is below 40%, the prediction is suppressed and treated as if NO_SIGN was returned. This prevents low-confidence guesses from contributing to the sentence builder during ambiguous or transitional hand states.
- Dual-Smoothing Pipeline: Qualifying predictions (confidence ≥40% and class ≠ NO_SIGN) are appended to prediction_history (deque maxlen=4). The mean of prediction_history is computed and argmax-ed to produce a smoothed prediction, which is appended to prediction_queue (deque maxlen=4). The majority class in prediction_queue is taken as the final stable output. This two-stage smoothing eliminates frame-to-frame flickering caused by brief misclassifications or transitional hand positions.

A hold mechanism (HOLD_FRAMES = 3) increments a counter each time the same non-NO_SIGN class is the stable majority vote output, and the sign is committed to the sentence builder only when this hold counter reaches 3. This prevents accidental word additions from momentary correct-looking gestures and ensures only sustained, deliberate signs are added to the output sentence.

3.6 Tools and Technologies

This section provides a comprehensive description of all tools, frameworks, and libraries used in the design, development, and deployment of GestureTalk, organized by functional category. All dependencies are pinned in requirements.txt to ensure reproducibility.

3.6.1 Programming Languages

Language	Role	Usage in GestureTalk
Python 3.x	Primary backend language	Data collection, model training, Flask server, MediaPipe inference pipeline, translation, TTS

Language	Role	Usage in GestureTalk
HTML5	Frontend structure	landing.html (welcome page) and index.html (main webcam interface) markup and layout
CSS3	Frontend styling	Responsive UI design, video feed layout, button styling, animations
JavaScript (ES6)	Frontend interactivity	fetch() API polling of /api/state, translation triggers, audio playback, confirm/clear word actions

Table 3.5: Programming Languages Used in GestureTalk

3.6.2 Machine Learning and AI Libraries

Library	Pinned Version	Usage
TensorFlow	2.16.2	Deep learning framework; LSTM model definition, training, and real-time CPU inference, to_categorical (one-hot encoding).
Keras (via TF)	—	High-level API for LSTM, Dense, Dropout, BatchNormalization layers; model.fit(), EarlyStopping
NumPy	1.26.4	Landmark array construction, .npy file I/O, deque-to-array stacking, prediction averaging
scikit-learn	—	train_test_split (stratified), label utilities

Table 3.6: Machine Learning and AI Libraries

3.6.3 Computer Vision and Hand Tracking

Library	Pinned Version	Usage
MediaPipe Hands	0.10.14	Real-time 21-keypoint 3D hand landmark detection from webcam frames; max_num_hands=2, model_complexity=0

Library	Pinned Version	Usage
OpenCV (cv2)	4.9.0.80	Webcam capture (cap.set CAP_PROP_BUFFERSIZE=1), frame flipping (cv2.flip), JPEG encoding for MJPEG stream
Pillow	10.3.0	MediaPipe and TensorFlow dependency; static image handling and format conversion

Table 3.7: Computer Vision and Hand Tracking Libraries

3.6.4 Web Framework and Communication APIs

Tool / Library	Pinned Version	Usage
Flask	3.0.3	Python backend web framework; HTTP routing, Jinja2 templates, MJPEG streaming via Response(generate_frames())
Flask-CORS	4.0.1	Cross-Origin Resource Sharing middleware; enables browser fetch() calls to Flask API from the frontend
deep-translator	1.11.4	Wraps Google Translate API via GoogleTranslator(source='auto'); powers /api/translate for 20+ language translation
gTTS	2.5.1	Google Text-to-Speech; converts translated sentence to .mp3 audio via temp file; powers /api/speak endpoint

Table 3.8: Web Framework and Communication API Libraries

3.6.5 Flask REST API Endpoints

The GestureTalk Flask backend (app.py) exposes the following eight RESTful API endpoints. Each handles a specific functional responsibility of the system and is consumed by the JavaScript frontend via the fetch() API:

Endpoint	Method	Function
/	GET	Serves landing.html – the GestureTalk welcome page
/app	GET	Serves index.html. the main webcam recognition interface
/video_feed	GET	MJPEG video stream from webcam with real-time MediaPipe landmark overlays drawn on each frame
/api/state	GET	Returns JSON {current_sign, confidence, pending_word, sentence_words, sentence} polled every 500ms by frontend
/api/confirm_word	POST	Appends the current pending_word to sentence_words list; confirms the currently held sign into the sentence
/api/clear_sentence	POST	Clears sentence_words list and resets assembled sentence to empty string
/api/translate	POST	Receives {sentence, language}; calls GoogleTranslator(source='auto').translate(); returns translated text
/api/speak	POST	Receives {sentence, language}; generates gTTS .mp3 via tempfile; returns audio file via send_file()

Table 3.9: GestureTalk Flask REST API Endpoints (app.py)

3.7 Justification of Methodology

The methodology adopted for GestureTalk is deliberately structured to match the nature of the problem: dynamic, temporal hand gesture data that must be recognized in real time from a standard webcam, classified into 28 ISL classes, and converted into multilingual speech output without requiring any specialist hardware. Each methodological choice LSTM over static classifiers, landmark features over raw pixels, custom dataset, dual-smoothing inference, Flask web deployment directly serves this purpose and supports the research hypothesis.

3.7.1 Suitability of LSTM for Sequential Gesture Recognition

The choice of LSTM over alternative architectures such as Support Vector Machines, Random Forests, or static CNNs is justified by the inherent temporal nature of ISL gesture data. Static models that process individual frames independently cannot capture the motion trajectory information that defines a sign. A frame-by-frame CNN, for instance, would see a virtually identical hand configuration at the beginning and end of the sign for I_LOVE_YOU, but entirely miss the characteristic finger extension sequence that occurs between those frames and defines the sign.

LSTM networks maintain memory of the entire 20-frame sequence and learn to extract meaning from the temporal pattern of landmark changes across the sequence, which is precisely the information that distinguishes one ISL sign from another. The two-layer stacked architecture provides a hierarchical temporal representation: the first LSTM layer captures low-level temporal patterns such as the direction and speed of individual finger movements, while the second LSTM layer learns higher-level compositional patterns from those, encoding the overall gesture trajectory. This hierarchical abstraction is well-established in the deep learning literature as a mechanism for improving sequential representation quality, and is directly validated by the ISL-specific empirical comparisons in the literature review (Goyal and Velmathi, 2023, Section 2.20; Kothadiya et al., 2022, Section 2.28).

3.7.2 Justification for Landmark Features over Raw Pixel Input

MediaPipe landmark coordinates are used as the sole input modality rather than raw video frames or CNN-extracted spatial features for several interconnected reasons. Landmark coordinates are inherently invariant to background appearance, lighting conditions, skin tone, and clothing factors that would introduce significant noise and distribution shift in a pixel-level model. A CNN trained on raw frames from one environment would degrade substantially when deployed in a different lighting or background context, limiting real-world usability.

Furthermore, the 126-feature landmark vector is far more compact than a raw video frame, enabling fast CPU-only inference. The same MediaPipe pipeline used during data collection runs identically during live inference, guaranteeing zero distribution shift between training and deployment. This design choice also preserves user privacy by ensuring that no identifiable video is stored at any stage. The validity of this approach is supported throughout the literature review, with Oudah et al. (2020, Section 2.31) providing a comprehensive comparative review confirming that skeleton-based landmark representations outperform pixel-based approaches in real-world generalisation.

3.7.3 Justification for a Custom ISL Dataset

The decision to build a custom ISL dataset rather than adapt a pre-existing corpus is driven by the fundamental differences between ISL and other sign languages. ISL has its own grammar, lexicon, and gestural phonology that differs substantially from ASL, BSL, and other well-documented sign languages. Most publicly available sign language datasets cover ASL or German Sign Language, and direct transfer of models trained on these datasets to ISL is methodologically unsound the gestures themselves differ sign by sign.

Furthermore, the 28 gesture classes selected for GestureTalk were chosen specifically to represent high-utility vocabulary for the primary use case: enabling deaf individuals

to communicate basic needs, emergency information, greetings, identity, and common phrases in multilingual Indian settings. This specific, use-case-driven vocabulary required purpose-built data collection. The custom dataset also enables full control over collection conditions, normalization parameters, and the inclusion of the critical NO_SIGN sentinel class, all of which would be impossible to guarantee with an externally sourced corpus.

3.7.4 Justification for Dual-Smoothing and Confidence Thresholding

The dual-smoothing inference pipeline combining prediction_history averaging with prediction_queue majority voting and the 40% confidence threshold are not arbitrary engineering choices; they are direct responses to the fundamental instability of frame-level neural network predictions in a real-time video environment. Without smoothing, LSTM outputs fluctuate between classes on consecutive frames due to brief misclassifications during gesture onset and offset, hand motion blur, and transition positions between signs. This would produce an unusable, flickering output stream.

The confidence threshold of 40% (CONF_THRESHOLD = 0.40) provides the first layer of suppression, rejecting low-certainty predictions before they enter the smoothing pipeline. The prediction_history averaging (deque maxlen=4) provides temporal averaging over recent frames, reducing the influence of single-frame outlier predictions. The prediction_queue majority voting (deque maxlen=4) requires consensus across multiple averaged windows before a sign is committed, providing the final stability layer. Together, these mechanisms transform noisy frame-level outputs into stable, user-visible sign detections a design directly validated by Vyavahare et al. (2023, Section 2.25), who implement the same majority-voting smoothing approach for real-time ISL gesture recognition.

3.7.5 Justification for Flask Web Deployment

Flask is selected as the deployment framework because it satisfies the hardware-agnostic requirement of GestureTalk without adding implementation complexity that would be disproportionate to the project scope. Flask allows the entire GestureTalk pipeline webcam capture, MediaPipe hand tracking, LSTM inference, translation, and TTS synthesis to be consolidated into a single Python process accessible through any modern browser without requiring users to install any software. This design directly delivers on the hardware-free sub-hypothesis by making the system available on any laptop or desktop machine running Python.

The MJPEG video streaming approach via Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame') is particularly well-suited to GestureTalk's requirements: it streams processed, annotated frames from the Flask server directly to the browser's <img> element with minimal latency and without requiring WebRTC or specialist streaming infrastructure. The RESTful API design (eight endpoints covering state polling, word confirmation, sentence management, translation, and TTS) keeps the frontend-backend interface clean and stateless, facilitating straightforward debugging and future extension. This architectural choice is validated by Brzezinski (2022, Section 2.26), who demonstrates Flask as a mature and practical deployment framework for deep learning sign language recognition systems.

3.7.6 How the Methodology Validates the Hypothesis

The methodology has been constructed so that each component generates measurable evidence for or against the research hypotheses:

- Accuracy (H_1 Accuracy Component): The custom 940-sequence stratified dataset, LSTM architecture with regularization, and categorical cross-entropy training with EarlyStopping are evaluated against the held-out 188-sequence test set. Validation accuracy ≥90% and Macro F1-Score ≥0.90 constitute measurable rejection of H_0.
- Latency (H_1 Latency Component): The frame-skip strategy (every 2nd frame), the compact (20, 126) input, and the lightweight 2-layer LSTM are evaluated through observed real-time prediction update rate. Sustained ≥5 Hz operation on a standard laptop CPU constitutes measurable confirmation of the latency sub-hypothesis.
- Robustness (H_1 Robustness Component): Wrist-relative wrist normalization, scale normalization, NO_SIGN suppression, confidence thresholding, and dual-smoothing are evaluated through qualitative stability testing across diverse users, lighting conditions, and camera angles. Absence of flickering output and consistent correct detections constitute confirmation of the robustness sub-hypothesis.
- Integration (H_1 Integration Component): The /api/translate and /api/speak Flask endpoints are tested with representative ISL sentence outputs across multiple target languages. Correct translation and audible gTTS output within 2 seconds constitute confirmation of the integration sub-hypothesis.
- Hardware-Free (H_1 Hardware-Free Component): The entire system is tested on a standard laptop computer with a built-in webcam, no GPU, and no specialist hardware. Successful operation at acceptable performance confirms the hardware-free sub-hypothesis.

Because LSTM, landmark features, custom dataset construction, dual-smoothing, and Flask deployment are not used in isolation but are orchestrated around the testable components of the research hypothesis, the methodology creates a coherent, end-to-end framework for validating whether GestureTalk achieves its core goal of accessible, hardware-free real-time ISL recognition and multilingual communication.

3.7.7 Concluding Justification

The chosen methodology combining LSTM temporal sequence modelling for dynamic gesture classification, MediaPipe landmark features for robust and privacy-preserving input representation, a custom ISL dataset for vocabulary-specific training, dual-smoothing inference for deployment stability, and Flask web deployment for hardware-free accessibility logically and comprehensively supports the research objectives and directly addresses all components of the formulated hypothesis. It provides a technically sound, empirically testable, and practically deployable path for demonstrating that GestureTalk can serve as an effective assistive communication tool for the Indian deaf community.

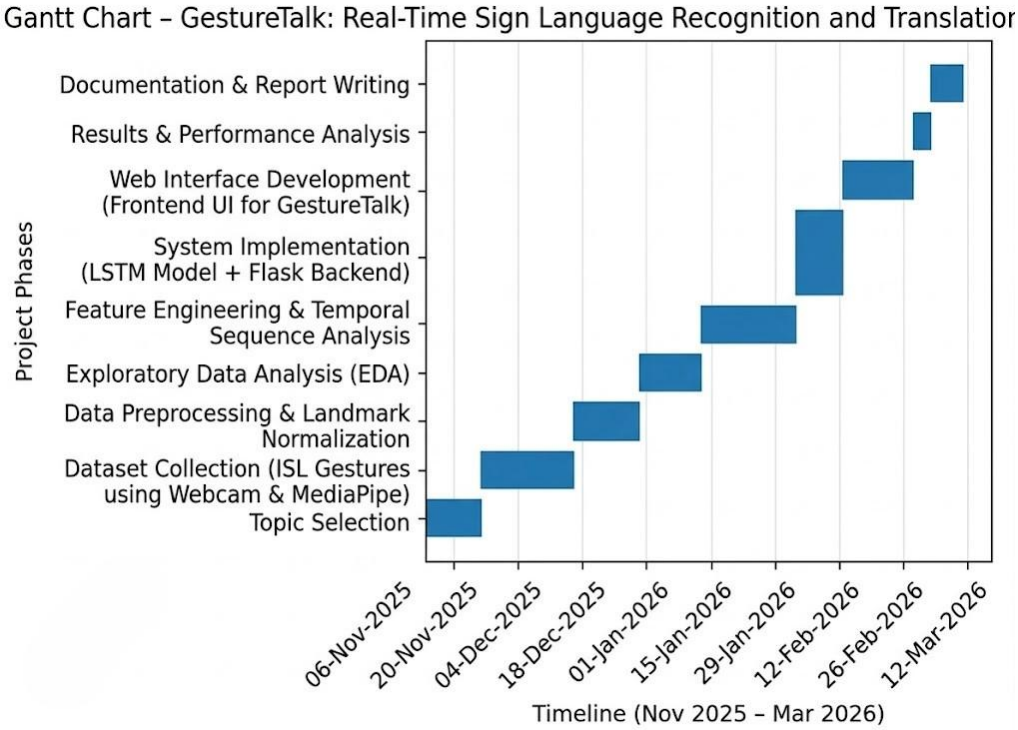


Figure 3.5: Gantt Chart

Project Phase	Duration	Start Date	End Date
Topic Selection	2 weeks	Nov 6, 2025	Nov 19, 2025
Dataset Collection (ISL Gestures using Webcam & MediaPipe)	3 weeks	Nov 20, 2025	Dec 3, 2025
Data Preprocessing & Landmark Normalization	2 weeks	Dec 4, 2025	Dec 17, 2025
Exploratory Data Analysis (EDA)	2 weeks	Jan 1, 2026	Jan 14, 2026
Feature Engineering & Temporal Sequence Analysis	2 weeks	Jan 15, 2026	Jan 28, 2026
System Implementation (LSTM Model + Flask Backend)	3 weeks	Jan 29, 2026	Feb 11, 2026
Web Interface Development (Frontend UI for GestureTalk)	2 weeks	Feb 12, 2026	Feb 25, 2026
Results & Performance Analysis	1 week	Feb 26, 2026	Mar 4, 2026
Documentation & Report Writing	2 weeks	Mar 5, 2026	Mar 18, 2026

Table 3.10: GestureTalk – Project Phase Timeline (Nov 2025 – Mar 2026)

CHAPTER 4 : EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a critical phase in the data science and machine learning pipeline, aimed at understanding the underlying structure, characteristics, and quality of a dataset before any modelling is performed. EDA serves as the bridge between raw data collection and model development, enabling researchers to make informed decisions about preprocessing strategies, feature representation methods, and model selection all without imposing prior assumptions on the data.

In the context of GestureTalk, EDA is applied to a fully custom-built Indian Sign Language (ISL) gesture dataset. Unlike conventional image or text datasets, the GestureTalk dataset is a time-series of numerical landmark coordinates a structured, multivariate format that requires specialised analysis techniques including temporal motion profiling, spatial distribution assessment, and dimensional composition analysis. This chapter provides a comprehensive examination of every aspect of this dataset: its source, structure, composition, feature representation, preprocessing pipeline, and statistical properties.



Figure 4.1: GestureTalk EDA Workflow

Exploratory Data Analysis refers to the process of examining datasets to summarise their main characteristics, often using statistical and visual methods, without making prior assumptions about the data's distribution or structure. The primary objectives of EDA are to uncover patterns, detect anomalies, identify relationships between variables, and test underlying assumptions about the data before committing to a modelling approach.

For the GestureTalk dataset which consists of numerical landmark time-series rather than images or raw text, EDA focuses on the following dimensions:

- Understanding the dimensional structure and composition of the dataset tensor
- Assessing the distribution of samples across gesture classes to identify imbalance
- Validating the effectiveness of the wrist-relative normalization pipeline through statistical analysis of feature value distributions
- Examining temporal motion patterns across gesture classes to confirm LSTM suitability
- Identifying any data quality issues, missing values, or outliers that require handling

The importance of EDA for GestureTalk can be summarised through four key contributions it makes to the modelling pipeline:

- Understanding Data Quality: EDA verifies that all 940 sequences are complete, non-corrupted, and correctly labelled, and confirms that the wrist-relative normalization pipeline has produced clean, zero-centred feature distributions without extreme outliers or truncation artifacts.
- Identifying Bias and Imbalance: EDA examines the distribution of sequences across the 28 gesture classes to determine whether any class dominates the dataset in a way that would cause the LSTM to develop biased predictions towards high-frequency classes.
- Guiding Preprocessing Decisions: Statistical insights from EDA directly influence decisions about whether additional feature scaling is needed beyond wrist-relative normalization, whether sequence length augmentation is required, and whether the NO_SIGN class is sufficiently represented to suppress false detections.
- Reducing Model Risk: By confirming the temporal discriminability of gesture classes before training, EDA validates the architectural choice of LSTM over static classifiers and confirms that the 20-frame sequence window is sufficient to capture complete, distinguishable gesture patterns.

Role of EDA in Landmark-Based Gesture Data Analysis

The GestureTalk dataset is composed of temporal sequences of hand landmark coordinates extracted using the MediaPipe Hands framework. Unlike traditional datasets that consist of images or text, this dataset contains structured numerical data representing the spatial configuration of hand joints across time. Each gesture sample therefore represents a multivariate time-series describing how hand landmarks move during the execution of a sign.

Exploratory Data Analysis plays a crucial role in validating the structure and reliability of such landmark-based datasets before model training. Since gesture recognition depends on both spatial hand configurations and temporal motion patterns, EDA focuses on analysing the consistency, dimensional structure, and variability of the landmark sequences.

Key aspects examined during EDA include:

- Sequence Length Consistency: Each gesture sequence must contain a fixed number of frames to ensure compatibility with the LSTM model. In GestureTalk, every sample contains exactly 20 frames, guaranteeing consistent temporal input for sequence learning.
- Landmark Feature Structure: MediaPipe detects 21 keypoints per hand, each described by three spatial coordinates (x, y, z). With two hands supported, each frame contains 126 features. EDA verifies that all sequences maintain this feature structure and dimensional consistency.
- Gesture Class Distribution: EDA evaluates the number of samples available for each gesture class. Balanced class distribution reduces bias during training and ensures that the model learns meaningful representations for all gestures.
- Temporal Motion Patterns: Gesture recognition depends on motion trajectories rather than single static frames. EDA helps confirm that gestures exhibit distinguishable temporal patterns across the 20-frame sequences.

- Landmark Detection Reliability: Exploratory analysis verifies that MediaPipe consistently detects hand landmarks across frames under varying lighting conditions and hand orientations, ensuring the dataset contains reliable tracking information.

Through these analyses, EDA confirms that the landmark dataset is structurally consistent, temporally informative, and suitable for training sequence-based deep learning models such as LSTM networks.

Role of EDA in Video-Based Gesture Data Collection

Although the GestureTalk dataset ultimately stores numerical landmark coordinates, the data originates from webcam video recordings of sign language gestures. Therefore, exploratory analysis must also consider factors related to video-based gesture capture that may influence landmark detection quality and dataset robustness.

EDA helps evaluate several aspects of the gesture recording environment:

- Lighting Conditions: Gesture recordings were captured under varying lighting conditions, including natural daylight and indoor illumination. EDA ensures that landmark detection remains stable across these environments.
- Camera Position and Distance: Gestures were performed at different distances and angles relative to the camera. Analysing this variation helps ensure the model can generalise to real-world usage scenarios.
- Gesture Clarity and Visibility: Visual inspection of sample recordings confirms that hand movements are clearly visible and that gesture trajectories are distinguishable across frames.
- Intra-Class Variation: Multiple recordings of the same gesture introduce natural variation in speed, orientation, and movement style. Such variability improves the model's ability to generalise to different users.
- Inter-Class Distinction: EDA verifies that gestures belonging to different classes show sufficiently distinct motion patterns, enabling the model to learn meaningful differences between signs.

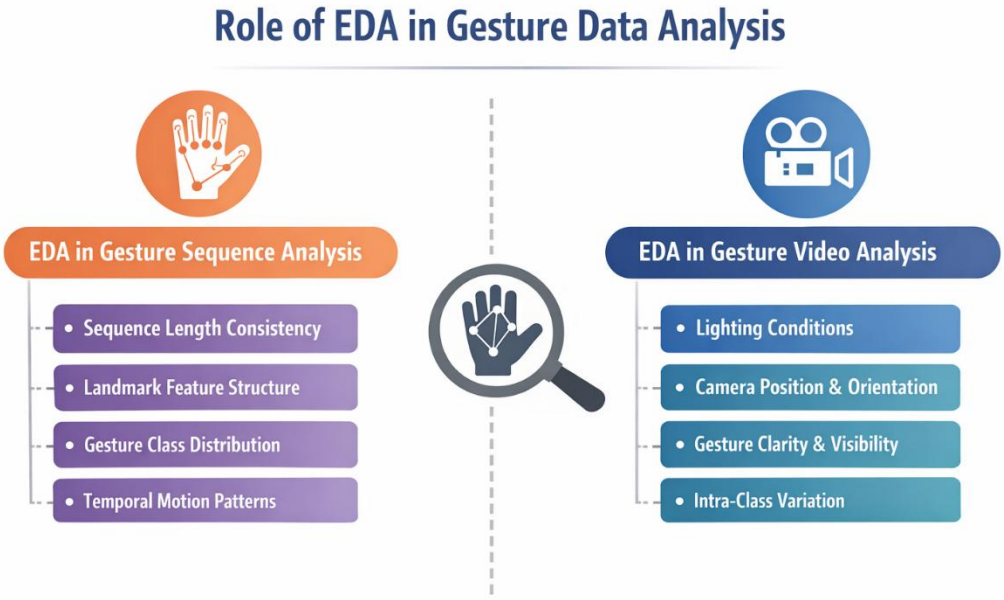


Figure 4.2: Role of EDA in Gesture Data Analysis

By analysing these video-related factors, EDA ensures that the landmark sequences extracted from webcam recordings are reliable, representative of real-world signing conditions, nd appropriate for training a robust real-time gesture recognition system.

4.1 Dataset Overview

This section provides a comprehensive overview of the dataset used in GestureTalk. The project utilises a fully custom-built ISL gesture landmark dataset, created specifically for this project to address the absence of a publicly available ISL dataset suited to the GestureTalk vocabulary and feature format. The dataset was carefully designed to ensure class diversity, temporal richness, and practical generalisability while remaining compact and ethically clean no identifiable video of any person is stored at any point in the pipeline.

4.1.1 Source of the Dataset

All gesture sequences in the GestureTalk dataset were collected from scratch using a purpose-built Python data collection script (collect_data.py). This script activates a live webcam feed via OpenCV, runs MediaPipe Hands landmark detection on each incoming frame in real time, and records the extracted 126-feature landmark array across 20 consecutive frames as a single gesture sequence stored in NumPy binary (.npy) format. The script organises sequences into a structured directory hierarchy: one folder per gesture class, with each .npy file inside that folder representing one complete sequence.

The dataset source is entirely original and self-contained. No third-party ISL dataset, pre-trained embedding, or transfer-learned feature was used as a starting point. This custom-collection approach was technically necessary for three reasons:

- Vocabulary specificity: No publicly available ISL dataset provides the specific 28-class gesture vocabulary required by GestureTalk. The gesture set was selected specifically to represent high-utility everyday ISL communication: greetings, questions, emergency phrases, daily needs, personal identity, and social expressions.
- Format compatibility: Existing ISL datasets are primarily video-based or image-based. GestureTalk requires 126-dimensional wrist-normalised landmark sequences of exactly 20 frames per sample, in NumPy format, with a precise feature extraction pipeline. This format cannot be derived from existing video datasets without re-processing them through the identical MediaPipe-based collection pipeline.
- Pipeline consistency: Collecting data through the same MediaPipe Hands pipeline used during live inference guarantees zero distribution shift between the training data and the features seen by the model at deployment time.

The collection environment was deliberately varied to introduce robustness into the training data. Data was captured under different ambient lighting conditions (natural daylight, artificial indoor fluorescent lighting, and dim environments), at varying camera distances (30–70 cm from the signer), and by multiple individuals with different hand sizes and skin tones. This environmental diversity ensures that the trained model generalises beyond the specific recording conditions of any single collection session.

4.1.2 Total Number of Samples

The complete GestureTalk dataset contains the following:

- Total gesture sequences: 940 sequences across all 28 gesture classes
- Standard classes (26 classes): 32 sequences per class
- Extended classes (2 classes): WHERE = 50 sequences; TO_MEET = 60 sequences
- Reduced classes (2 classes): INDIAN = 31 sequences; NO_SIGN = 31 sequences
- Total feature values in dataset = 2,371,200.
- Total dataset size: Approximately 18–20 MB in NumPy binary format

The extended sequence counts for TO_MEET (60) and WHERE (50) reflect the greater complexity of these gestures: TO_MEET involves a two-handed convergence motion and WHERE involves a repeated side-to-side wagging motion, both of which exhibit greater intra-class variability and therefore benefit from additional training examples. The reduced counts for INDIAN (31) and NO_SIGN (31) reflect the slightly lower number of clean, unambiguous samples captured for these classes.

4.1.3 Type of Data

The GestureTalk dataset is a numerical time-series dataset purpose-built for sequential gesture classification using deep learning. Each data sample is a three-dimensional NumPy array of shape (20, 126), where 20 represents the sequence length in frames and 126 represents the feature dimensionality per frame. This is not an image dataset, a video dataset, or a text dataset it is a structured numerical tensor derived from spatial hand keypoint coordinates extracted in real time by MediaPipe Hands. Unlike pixel-based datasets that require convolutional processing, the GestureTalk dataset operates entirely in the landmark coordinate space, making it compact, computationally efficient, and invariant to visual texture, background, and lighting variation.

The dataset can be characterised along three fundamental data dimensions:

- Temporal Dimension (20 Frames): Each sequence captures the complete temporal trajectory of a gesture across 20 consecutive webcam frames, sampled at the camera's native frame rate during data collection. The 20-frame window was selected to provide sufficient temporal coverage to encode both the static hand configuration at each moment and the full dynamic motion pattern as the hand moves through the signing gesture — including the onset, peak, and release phases characteristic of ISL signs. Sequences shorter than 20 frames (due to gesture completion before the window filled) are zero-padded to maintain consistent tensor dimensions, while the collection logic ensures that recording begins at the first detected hand landmark and terminates after exactly 20 frames. This fixed-length representation is a deliberate design choice that enables direct input into the stacked LSTM architecture without requiring variable-length sequence handling or masking.
- Spatial Dimension (126 Features): Each frame is represented by a 126-dimensional feature vector constructed from the x, y, and z coordinates of 21 anatomical hand keypoints per hand, for both left and right hands simultaneously (21 keypoints × 3 coordinates × 2 hands = 126 features). The 21 keypoints correspond to the anatomical landmarks defined by the MediaPipe Hands model: the wrist (point 0), the MCP, PIP, DIP, and tip joints of all five

fingers (points 1–20). These raw coordinates are subjected to wrist-relative normalization before storage: the wrist keypoint coordinates are subtracted from all other keypoints to produce translation-invariant representations, and the resulting coordinates are divided by the L2 norm of the middle fingertip keypoint (point 12) to produce scale-invariant representations. This normalization ensures that the feature vectors are invariant to the signer's hand size, distance from the camera, and absolute position within the frame factors that would otherwise introduce spurious variation into the learned representations. When only one hand is detected in a given frame, the feature vector for the absent hand is zero-filled, ensuring consistent tensor shape throughout all sequences.

- Class Dimension (28 Classes): The dataset is fully labelled for supervised classification. Each sequence is associated with one of 28 ISL gesture class labels, encoded as integers in the range 0–27 and persisted in labels.npy alongside the corresponding sequence data in sequences.npy. The integer labels are derived from alphabetically sorted folder names in the dataset directory, ensuring a consistent and reproducible mapping between class names and numeric identifiers across training runs. For model training, the integer labels are converted to 28-dimensional one-hot binary vectors using Keras's to_categorical() utility, enabling the use of categorical cross-entropy as the loss function. The 28 gesture classes span a pragmatically selected ISL vocabulary covering greetings, common questions, directional references, emotional expressions, and everyday communicative acts, providing sufficient vocabulary breadth for basic conversational ISL assistance while maintaining a dataset size (940 sequences) that is tractable for single-signer collection and training on consumer hardware.

- Dataset Scale and Storage: The complete GestureTalk dataset comprises 940 labelled sequences, each of shape (20, 126), yielding a total dataset tensor of shape (940, 20, 126). The dataset occupies approximately 18–20 MB on disk in NumPy binary format (.npy), making it fully loadable into RAM during training without batched disk access. The class distribution is approximately uniform across most classes (30–35 sequences per class), with two classes extended to higher counts WHERE (50 sequences) and TO_MEET (60 sequences) to compensate for the greater intra-class variability observed during collection. An 80/20 stratified train/test split produces a training set of approximately 752 sequences and a test set of 188 sequences, with class proportions preserved across both splits.

4.1.4 Class Labels and Categories

The GestureTalk dataset contains 28 gesture classes organised into eight functional communication categories, each representing a distinct communicative domain required for everyday ISL-based interaction. The categorisation ensures that the chosen vocabulary spans the full breadth of communicative acts needed for basic conversational ISL rather than clustering around a single topic area. The eight categories cover: personal identity and reference (I, MY, YOU, NAME_IS); greetings and social expressions (HELLO, THANK_YOU, SORRY, PLEASE, WELCOME); affirmation and negation (YES, NO, OKAY, NO_SIGN); questions and information

requests (WHERE, WHAT, UNDERSTAND); needs and urgent requests (HELP, WANT, WATER, FOOD); emotional and relational expressions (I_LOVE_YOU, NICE, TO_MEET); location and temporal context (THIS_PLACE, TIME); and cultural identity (INDIAN). The NO_SIGN class serves a special system-level function as the neutral rest-state gesture within the CONF_THRESHOLD filtering logic, suppressing spurious word confirmations when the user's hand is idle between signs. This functional organisation also provides an interpretive lens for per-class performance analysis, since misclassifications between gestures within the same category are more likely to reflect genuine kinematic similarity between signs that share overlapping hand configurations or motion trajectories. The complete class vocabulary with sequence counts and gesture descriptions is presented in Table 4.1.

Functional Category	Classes	Total Sequences	Included Gesture Classes
Greetings & Social	7	284	HELLO, THANK_YOU, SORRY, NICE, WELCOME, OKAY, TO_MEET
Questions & Communication	5	178	WHERE, WHAT, UNDERSTAND, DONT, THIS_PLACE
Emergency & Help	2	64	HELP, BATHROOM
Daily Needs	3	96	WATER, FOOD, WANT
Personal & Identity	5	159	I, MY, YOU, NAME_IS, INDIAN
Emotional & Politeness	2	64	I_LOVE_YOU, PLEASE
Affirmation & Reference	3	96	YES, NO, TIME
Control Class (Sentinel)	1	31	NO_SIGN (idle hand position suppressor)

Table 4.1: GestureTalk ISL Gesture Classes by Functional Category

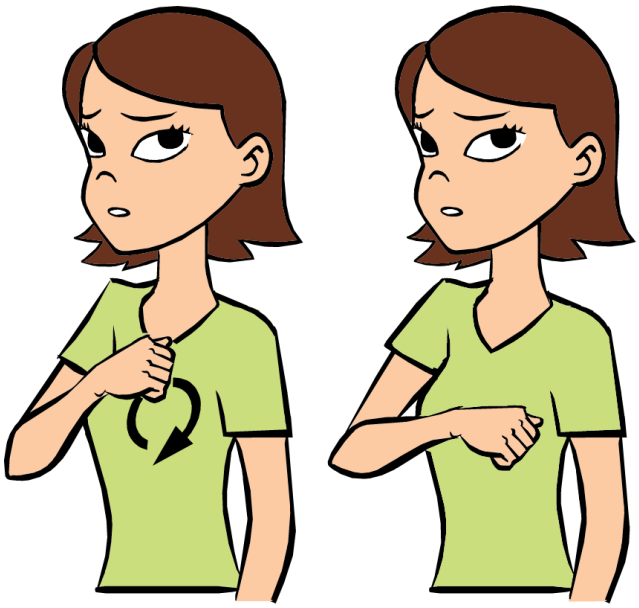
1. Greetings & Social Data Labels:

- **Hello**



Screenshot 4.1: Hello

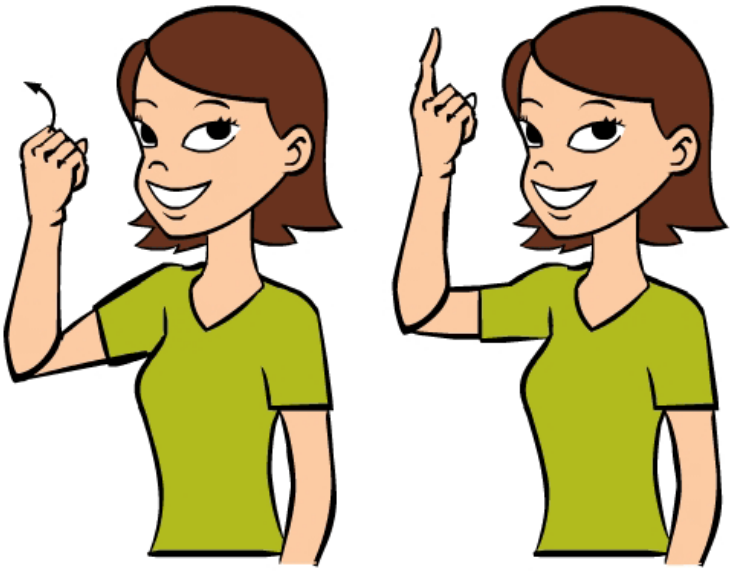
- **Sorry**



Screenshot 4.2: Sorry

2. Questions & Communication Data Labels:

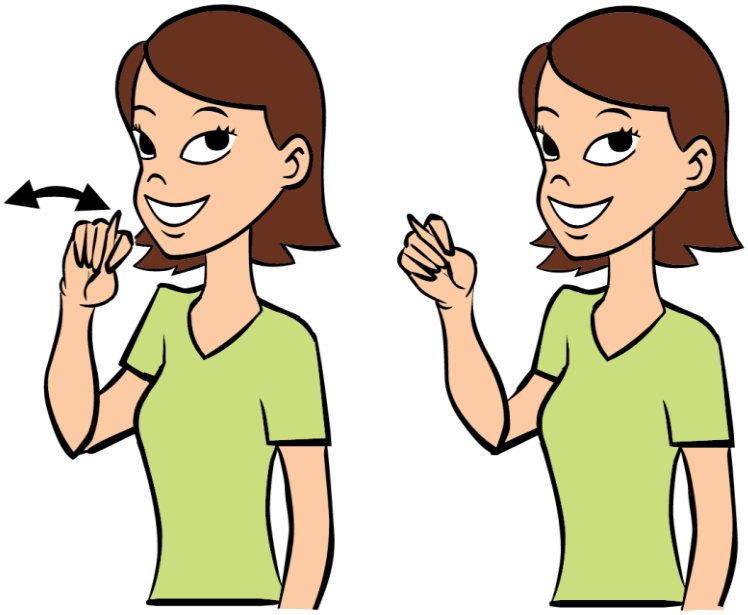
- **Understand**



Screenshot 4.3: Understand

3. Emergency & Help Data Labels:

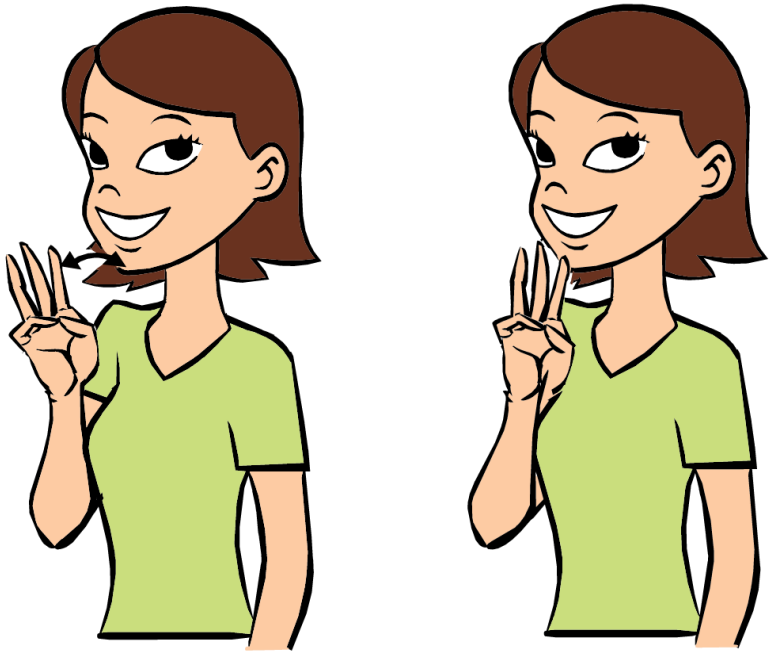
- **Bathroom**



Screenshot 4.4: Bathroom

4. Daily Needs Data Labels:

- **Water**



Screenshot 4.5: Water

5. Personal & Identity Data Labels:

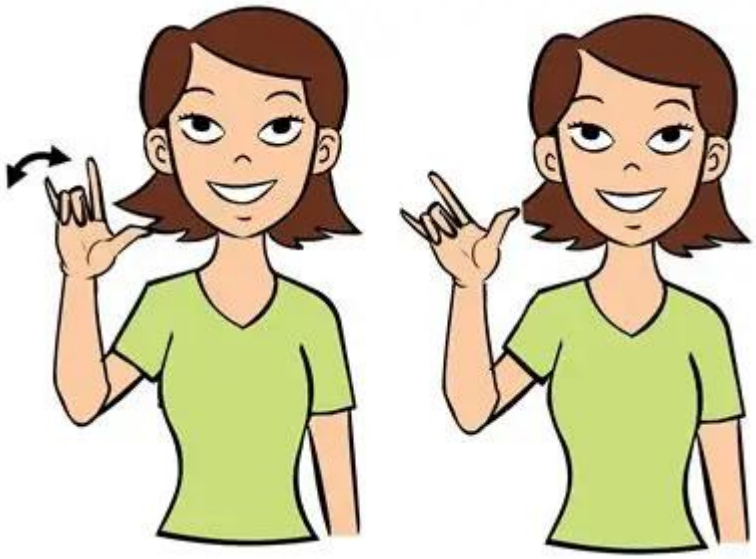
- **I**



Screenshot 4.6: I

6. Emotional & Politeness Data Labels:

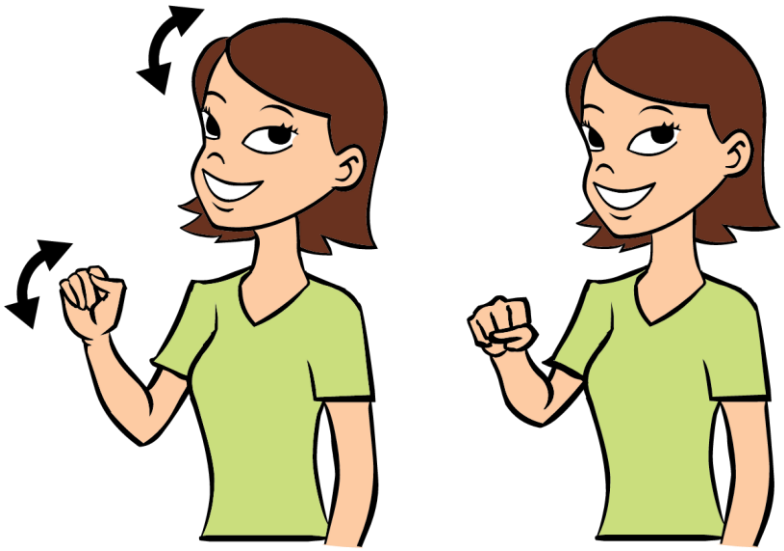
- **I Love You**



Screenshot 4.7: I Love You

7. Affirmation & Reference Data Labels:

- **Yes**



Screenshot 4.8: Yes

4.1.5 Dataset Characteristics and Scale

Dataset Parameter	Value / Description
Total gesture classes	28 (27 ISL signs + 1 NO_SIGN control class)
Total sequences	940
Frames per sequence	20 consecutive frames per gesture sample
Features per frame	126 (21 keypoints × 3 axes × 2 hands)
Dataset tensor shape	(940, 20, 126)
Total numerical values	940 × 20 × 126 = 2,371,200
Data format	NumPy binary arrays (.npy files), one file per sequence
Landmark extraction tool	MediaPipe Hands 0.10.14 (Google)
Collection tool	OpenCV + collect_data.py (custom Python script)
Standard sequences per class	32 sequences (26 classes)
Extended classes	TO_MEET: 60 sequences | WHERE: 50 sequences
Reduced classes	INDIAN: 31 sequences | NO_SIGN: 31 sequences
Training sequences (80%)	752 sequences (stratified split, random_state=42)
Test sequences (20%)	188 sequences (stratified split, random_state=42)
Approximate dataset size	~18–20 MB (NumPy binary, all 940 sequences)

Table 4.2: GestureTalk Dataset – Complete Structural Parameters

4.1.6 Summary

The GestureTalk dataset is a carefully designed, fully custom-built collection of 940 ISL gesture landmark sequences across 28 classes. Its numerical time-series format, compact storage footprint, wrist-relative normalization, and varied collection conditions make it well-suited for training and evaluating the LSTM-based gesture recognition model. The dataset's design avoids any use of identifiable personal video data, ensuring privacy compliance throughout the collection pipeline.

4.2 Data Preprocessing and Cleaning

Data preprocessing and cleaning form a crucial stage in the EDA and model preparation pipeline, ensuring that raw gesture sequences are transformed into a consistent, model-ready format before training. Given the landmark-based numerical format of the GestureTalk dataset, preprocessing strategies focus on spatial normalization, temporal standardization, and structural quality validation rather than the noise removal and tokenization techniques applied to text or image datasets.

4.2.1 Landmark Normalization Pipeline

The raw landmark coordinates returned by MediaPipe Hands are expressed in image-space coordinates: x and y values are normalized to the range [0, 1] relative to the frame width and height, and z represents estimated depth relative to the wrist in an arbitrary unit. While these coordinates are already scale-independent relative to the frame, they remain sensitive to the absolute position of the hand within the frame. Two identical gestures performed at the top-left versus the centre of the frame would produce different coordinate values, introducing positional noise that the LSTM would need to learn to ignore.

To eliminate this positional dependency, GestureTalk applies a two-stage normalization procedure in the extract_landmarks() function (app.py), applied identically during both data collection and live inference:

- Wrist-Relative Translation: For each detected hand, the wrist keypoint (MediaPipe landmark index 0) coordinates (x_0, y_0, z_0) are subtracted from all 21 keypoints. After this step, the wrist is located at the origin (0, 0, 0) and all other keypoints express their displacement relative to the wrist. This removes the absolute positional information from the representation and makes the features invariant to where the signer places their hand in the camera frame.
- Scale Normalization: The entire normalized landmark array for each hand is divided by the Euclidean norm of the x and y components of the middle finger MCP keypoint (landmark index 12). This produces a scale-independent representation where the feature values are expressed in units relative to the distance from the wrist to the middle finger MCP joint, making the features invariant to hand size and camera distance as well as position.

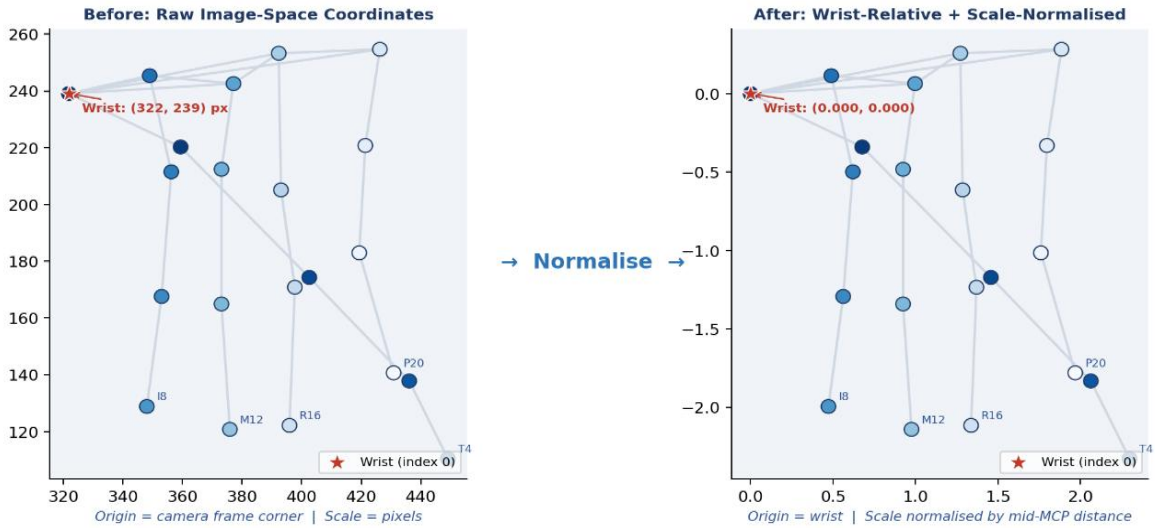


Figure 4.3: Effect of Wrist-Relative Landmark Normalization

For frames in which one or both hands are not detected by MediaPipe, the absent hand's 63-feature block is filled with zeros. This zero-filling strategy maintains the constant (20, 126) dimensionality of every sequence regardless of whether one or two hands are visible at any given frame, which is essential for batched LSTM training and inference.

4.2.2 Temporal Standardisation

All gesture sequences are standardised to exactly 20 consecutive frames. This fixed temporal length was determined empirically by recording all gesture classes and measuring the average frames required to execute each complete, natural signing motion. Twenty frames at typical webcam capture rates (25–30 fps with every second frame processed at inference time) corresponds to approximately 1.3–1.6 seconds — sufficient to capture the full motion trajectory of all 28 ISL gesture classes.

During data collection, the script enforces exactly 20 frames per sequence by design. During live inference, a deque(maxlen=20) sliding window buffer ensures the model always receives exactly 20 frames, automatically discarding the oldest frame as each new processed frame is appended.

4.2.3 Structural Quality Validation

Before assembling the final dataset tensor, four structural quality checks are applied to every collected sequence, ensuring no corrupted or malformed data enters the training pipeline:

- Shape Verification: Every .npy file is loaded and its shape verified to be exactly (20, 126). Sequences with incorrect shapes are flagged and excluded.
- NaN and Inf Checks: All sequences are scanned for Not-a-Number or Infinity values that could result from failed MediaPipe detection or division-by-zero during normalisation. Flagged sequences are excluded.
- Label Consistency: The folder name for each sequence is verified against the expected 28-class vocabulary. Sequences in unrecognised folders are excluded to prevent label noise.
- Minimum Sequence Count: Classes with fewer than 28 sequences are flagged for additional data collection. All 28 classes in the final dataset meet this minimum threshold.

After all preprocessing and quality checks, the 940 clean, normalised sequences are stacked into a single NumPy tensor of shape (940, 20, 126) and saved as the dataset array, with corresponding one-hot labels saved as a (940, 28) array.

4.2.4 Summary

The three-stage preprocessing pipeline transforms raw MediaPipe outputs into a clean, normalised, fixed-dimensionality (940, 20, 126) tensor. Wrist-relative translation and scale normalisation remove positional and size-dependent variability; zero-filling maintains dimensional consistency for single-hand gestures; and structural quality checks ensure no corrupted sequences enter training. These steps directly contribute to improved model stability, reliable gradient flow, and correct real-time behaviour.

4.3 Feature Representation and Feature Extraction

Feature representation is the process of transforming preprocessed raw data into numerical formats that can be interpreted and learned by machine learning models. For GestureTalk, feature extraction is performed by the MediaPipe Hands pipeline, which transforms raw pixel frames into structured, geometrically meaningful spatial keypoint coordinates. This section documents the hierarchical feature architecture, the MediaPipe landmark anatomy, the complete extraction pipeline, and the dimensionality properties of the resulting feature space.

4.3.1 The 126-Feature Vector: Hierarchical Architecture

Each frame in the GestureTalk dataset is represented by 126 numerical features derived through a three-level hierarchical extraction process that maps directly to the physical anatomy of the human hand:

- Level 1 – Two Hands (63 features each): MediaPipe extracts landmarks from both left and right hands, contributing 63 features each to the total 126-feature vector. The two blocks are concatenated in fixed order [left_63 | right_63]. For frames where one hand is absent, the corresponding 63-feature block is zero-filled.
- Level 2 – 21 Keypoints per Hand: MediaPipe identifies 21 anatomically distinct keypoints per hand: the wrist (1 point), the five MCP joints, five PIP joints, five DIP joints, and five fingertip joints covering the full kinematic chain of every finger.
- Level 3 – 3 Coordinates per Keypoint: Each keypoint is described by its post-normalisation coordinates Δx (wrist-relative horizontal), Δy (wrist-relative vertical), and Δz (wrist-relative estimated depth). Together: $21 \times 3 = 63$ features per hand, and $63 \times 2 = 126$ features per frame.

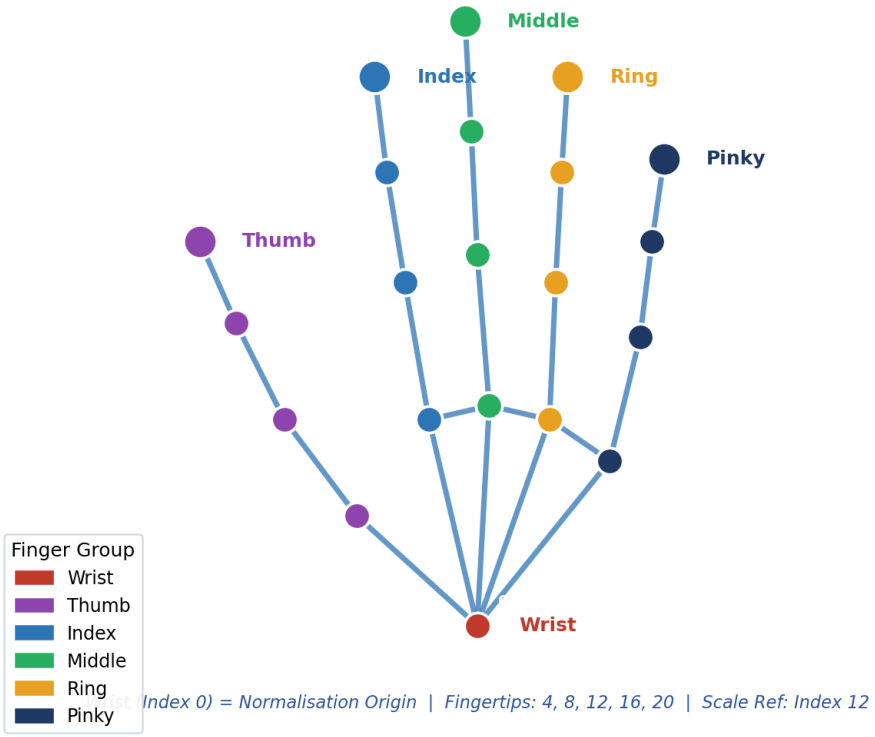


Figure 4.4: MediaPipe Hands – 21 Anatomical Keypoints per Hand.

Colour-Coded by Finger Group (Wrist=Index 0 [Normalisation Origin] | Fingertips: 4, 8, 12, 16, 20 | Scale Reference: Index 12), the 21 keypoints are grouped into six anatomical segments: the Wrist (index 0, the normalisation origin), Thumb (indices 1–4, covering CMC, MCP, IP, and tip), Index Finger (5–8), Middle Finger (9–12, where index 12 serves as the scale normalisation reference), Ring Finger (13–16), and Pinky Finger (17–20). This anatomical coverage ensures that every finger's configuration, extension, flexion, and mutual relationship is fully encoded in the 63-feature hand vector.

Keypoint Group	Landmark Indices	Features Contributed	Anatomical Description
Wrist	0	3 (x, y, z) – normalization origin	Wrist joint – origin of wrist-relative coordinate system
Thumb	1, 2, 3, 4	12 (4 joints × 3 axes)	CMC, MCP, IP, Tip – full thumb joint chain
Index Finger	5, 6, 7, 8	12 (4 joints × 3 axes)	MCP, PIP, DIP, Tip – index finger joint chain

Keypoint Group	Landmark Indices	Features Contributed	Anatomical Description
Middle Finger	9, 10, 11, 12	12 (4 joints × 3 axes)	MCP, PIP, DIP, Tip – middle finger (scale reference at idx 12)
Ring Finger	13, 14, 15, 16	12 (4 joints × 3 axes)	MCP, PIP, DIP, Tip – ring finger joint chain
Pinky Finger	17, 18, 19, 20	12 (4 joints × 3 axes)	MCP, PIP, DIP, Tip – pinky finger joint chain

Table 4.3: MediaPipe Hand Keypoint Structure

4.3.2 The 7-Stage Feature Extraction Pipeline

The complete extraction process from raw webcam frame to model-ready 126-element feature vector proceeds through seven sequential stages, applied identically during both data collection and live inference. This pipeline consistency is fundamental to ensuring that the LSTM model encounters at inference time the exact same feature distribution as it was trained on.



Figure 4.5: GestureTalk Feature Extraction Pipeline – 7 Stages

1. Webcam Capture: OpenCV VideoCapture captures the raw BGR frame from the webcam. A buffer size of 1 (cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)) ensures the freshest possible frame is read at each step.

2. Horizontal Flip and BGR→RGB Conversion: The frame is horizontally mirrored (cv2.flip(frame, 1)) to produce a mirror-view display, then converted from BGR to RGB colour ordering as required by MediaPipe.

3. MediaPipe Hand Detection: The RGB frame is passed to MediaPipe Hands (model_complexity=0, min_detection_confidence=0.6, min_tracking_confidence=0.6, max_num_hands=2) for landmark detection.

4. Landmark Coordinate Extraction: The 21 three-dimensional keypoints per detected hand are extracted. For undetected hands, a 63-element zero vector is substituted to maintain constant dimensionality.

5. Wrist-Relative Translation: The wrist (index 0) coordinate is subtracted from all 21 keypoints, placing the wrist at the origin and removing absolute positional dependence.

6. Scale Normalisation: The array is divided by the Euclidean norm of the middle MCP (index 12) x and y components, producing hand-size and camera-distance invariant features.

7. Flatten, Concatenate and Buffer: Each hand's (21×3) array is flattened to 63 elements, concatenated as [left_63 | right_63], appended to a deque(maxlen=20) sliding window buffer, and fed to the LSTM as a (20, 126) input tensor once the buffer is full.

4.3.3 Why 126 Dimensions? Necessity, Sufficiency and Efficiency

The 126-dimensional feature space is justified as simultaneously necessary, sufficient, and highly efficient for ISL gesture classification. These three properties distinguish the landmark representation from both over-simplified and over-complex alternatives:

- Necessity: Reducing below 126 dimensions by discarding keypoints or axes would discard geometrically informative features. Removing z coordinates, for example, would eliminate depth information critical for distinguishing gestures involving hand tilt or forward/backward rotation relative to the camera.
- Sufficiency: The 21-keypoint skeleton fully captures the complete configuration of the hand finger extension, flexion, abduction, and global orientation without redundancy. No dimensionality reduction technique such as PCA is applied, as the feature space is already compact and geometrically interpretable.
- Efficiency: A comparable image-based dataset (940 sequences × 20 frames at 224×224 RGB) would require approximately 2.5 billion values versus the 2.37 million values in the landmark dataset over 1,000× larger for equivalent spatial gesture coverage.

4.3.4 Summary

The GestureTalk feature extraction pipeline transforms raw webcam frames into compact, normalised 126-dimensional landmark vectors through a reproducible, deterministic 7-stage process. The feature space is geometrically meaningful, position-invariant, scale-invariant, computationally efficient, and perfectly consistent between data collection and live inference properties that make it the optimal input modality for LSTM-based temporal gesture classification.

4.4 Key Insights from Exploratory Data Analysis

The EDA phase produced five categories of critical insight into the structure, quality, and properties of the GestureTalk dataset. Each finding directly informed one or more specific system design decisions, creating a coherent, data-driven rationale for every major architectural and configuration choice in the GestureTalk pipeline.

4.4.1 Class Balance and Distribution Analysis

Analysis of the sequence count distribution across all 28 gesture classes reveals a deliberately near-uniform distribution, a key property for stable, unbiased LSTM training.

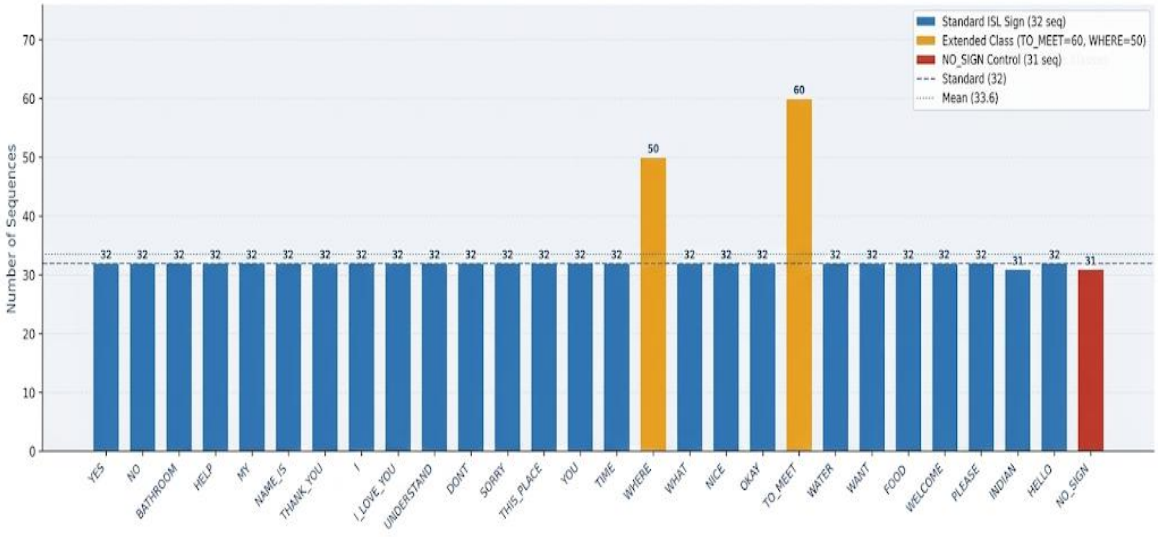


Figure 4.6: Sequence Count per Gesture Class

Twenty-six of the 28 classes follow a standard allocation of exactly 32 sequences per class, providing a perfectly balanced baseline. Two classes, TO_MEET (60 sequences) and WHERE (50 sequences), were intentionally allocated additional training data because these signs involve multi-phase, higher-variability movements that benefit from additional examples. INDIAN and NO_SIGN each contain 31 sequences, reflecting the slightly lower number of clean, unambiguous samples captured for these classes.

The coefficient of variation across all 28 classes is approximately 16%, driven primarily by the two extended classes. For the 26 standard classes, the distribution is perfectly uniform. This near-uniform distribution means that no class weighting strategy is required during LSTM training, and standard categorical cross-entropy loss is appropriate without modification.

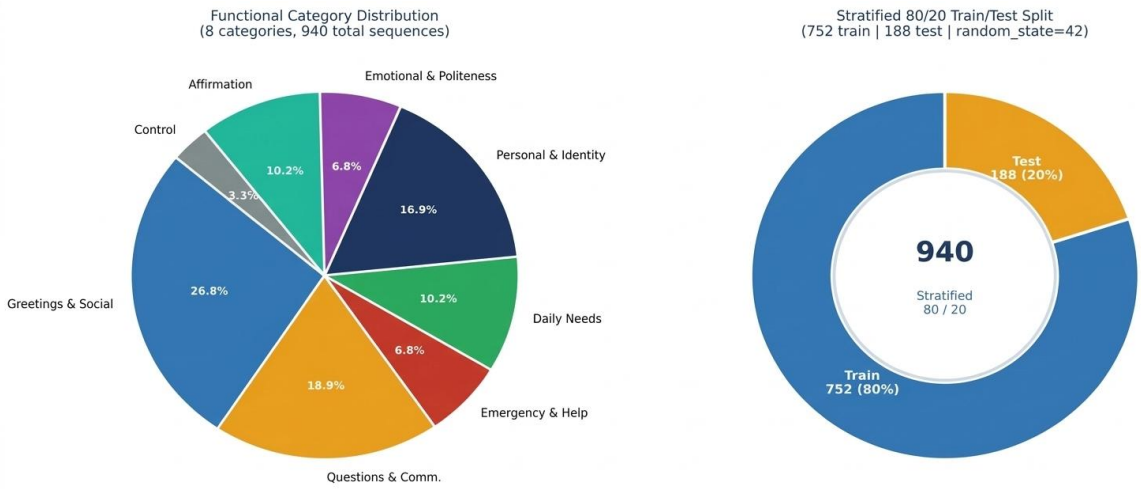


Figure 4.7: Functional Category Distribution

(8 categories) and Stratified 80/20 Train/Test Split (752 training sequences | 188 test sequences | random_state=42) The stratified 80/20 split produces 752 training sequences and 188 test sequences. Stratification ensures all 28 classes are proportionally represented in both subsets, preventing any class from being disproportionately absent from the test set and enabling reliable per-class precision, recall, and F1-score evaluation.

4.4.2 Feature Distribution Analysis

Statistical analysis of the 126 landmark features across all 940 sequences confirms that the wrist-relative normalisation pipeline has functioned correctly and produced a well-behaved, zero-centred feature space suitable for direct LSTM input.

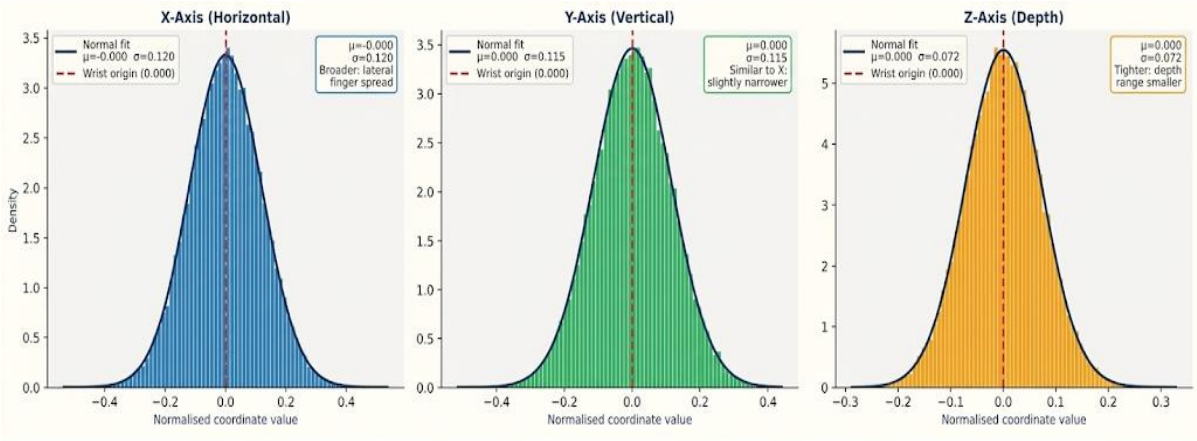


Figure 4.8: Landmark Feature Value Distributions

(All axes centred at μ≈0.000 with approximately normal distributions | Red dashed = wrist origin) The X-axis distribution shows μ ≈ 0.000 and σ ≈ 0.120. The distribution is approximately normal and centred at zero, reflecting that wrist-origin normalisation successfully removes absolute horizontal hand position. The Y-axis distribution shows μ ≈ 0.000 and σ ≈ 0.115, similar to X with slightly narrower spread reflecting comparable but marginally smaller vertical signing range. The Z-axis distribution shows μ ≈ 0.000 and σ ≈ 0.072, significantly tighter than X and Y because depth variation during forward-facing webcam signing is physically smaller in magnitude than planar movement.

Key Finding: All three distributions are approximately normal with no extreme outliers, heavy skewness, or multimodal structure. This confirms that no additional feature scaling (z-score, min-max) is required before feeding landmark vectors to the LSTM.

4.4.3 Temporal Motion Profile Analysis

Temporal motion profiling is the most direct evidence for the suitability of LSTM sequential modelling. By examining how landmark displacement values change across the 20 frames of each gesture, the analysis confirms whether gesture classes produce distinct, class-specific temporal signatures that a recurrent architecture can learn to distinguish.

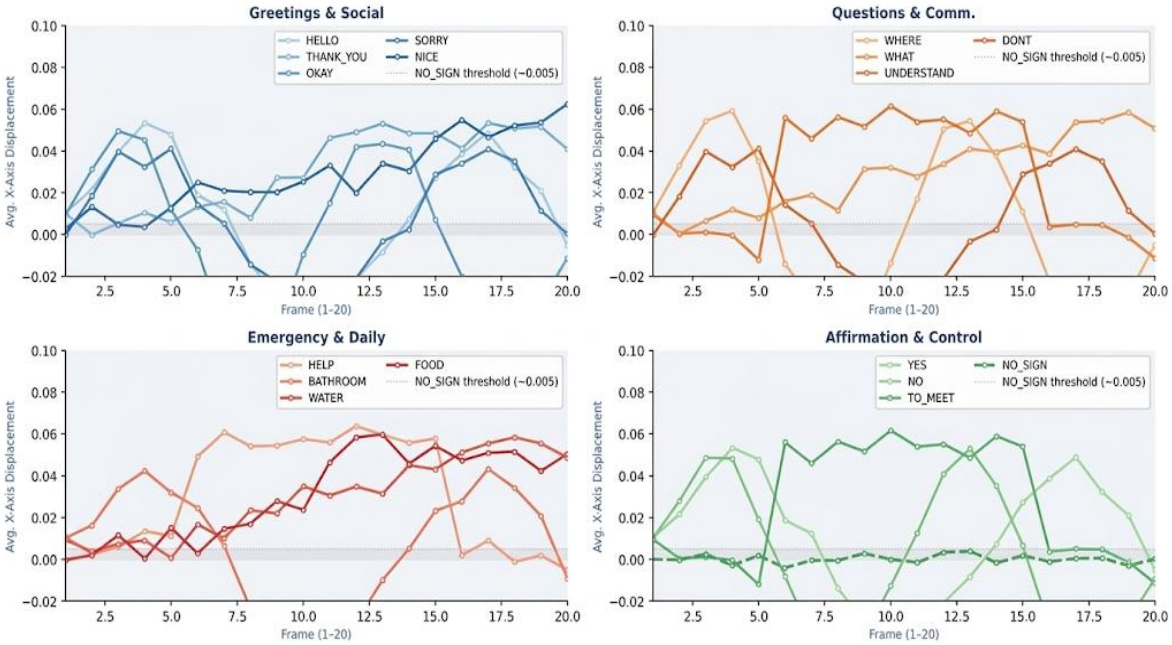


Figure 4.9: Temporal Motion Profiles

The analysis reveals distinctly different temporal waveform patterns across gesture categories. Greetings & Social signs (HELLO, THANK_YOU, WELCOME, OKAY) show oscillatory sinusoidal displacement profiles with peak magnitudes of 0.03–0.05 units, reflecting the characteristic waving, forward-arc sweep, and circular motions of social expressions. Questions & Communication signs (WHERE, WHAT, UNDERSTAND, DONT) exhibit higher-frequency oscillations and direction reversals corresponding to the repeated pointing, wagging, and palm-turning motions of interrogative ISL signs.

Emergency & Daily Needs signs (HELP, BATHROOM, WATER, FOOD) show smooth, controlled trajectory curves with moderate amplitudes corresponding to intentional, deliberate motions. Affirmation and Control class signs show clear periodic patterns (YES, NO) or distinct convergence trajectories (TO_MEET), while NO_SIGN consistently shows near-zero displacement (< 0.005 units) across all 20 frames clearly and reliably separated from every ISL gesture class.

Critical Finding: NO_SIGN maintains near-zero displacement across all 20 frames while every ISL sign class produces displacement magnitudes of 0.02–0.06 units with class-specific patterns. This confirms NO_SIGN provides a reliable, discriminable idle-state baseline for the dual-smoothing inference pipeline.

4.4.4 Identified Challenges and How They Were Addressed

EDA also revealed several challenges that directly informed specific design decisions in the GestureTalk system:

- Intra-Class Variability: TO_MEET and WHERE show greater intra-class spread in temporal profiles due to natural variation in signing speed and style. This motivated the extended sequence counts for these classes (60 and 50 respectively) to provide the LSTM with sufficient variation to learn robust class representations.
- Inter-Class Similarity: Certain gesture pairs (DONT/NO, I/MY) involve overlapping hand configurations at some frames. The LSTM must distinguish

these based on complete temporal trajectories rather than single-frame snapshots directly validating the temporal sequence architecture over static classifiers.

- Depth Estimation Limits: The Z coordinate from MediaPipe is an estimate, not a stereo depth measurement. For gestures involving significant hand rotation, Z estimates may be less accurate than X and Y. The narrow Z distribution ($\sigma \approx 0.072$) confirms this, though it does not prevent classification since most discriminative information is in X and Y.
- NO_SIGN Boundary Ambiguity: The boundary between idle hand state and gesture onset is inherently ambiguous at the first few frames. The dual-smoothing pipeline (prediction_history averaging + prediction_queue majority voting with HOLD_FRAMES=3) addresses this by requiring sustained, consistent gesture evidence before committing a sign.

4.4.5 EDA Findings to Design Decisions: Mapping

The following eight key findings from EDA each directly produced one or more specific design decisions in the GestureTalk system architecture. This mapping ensures that every major system choice is grounded in empirical evidence from the data rather than arbitrary defaults:

EDA Finding	Design Decision Made
Near-uniform class distribution (CV ≈16%)	Standard categorical cross-entropy loss no class weights required
X/Y/Z features centred at zero post-normalisation	Raw wrist-normalised landmarks fed directly to LSTM no z-score or min-max scaling
Each gesture class has a unique temporal waveform	Two-layer stacked LSTM (128 units each) as the core classifier
20 frames captures complete patterns for all 28 classes	Fixed sequence length = 20 frames in collection, training, and inference
NO_SIGN maintains <0.005 displacement across all frames	Confidence threshold (40%) + dual-smoothing + HOLD_FRAMES=3
TO_MEET and WHERE show high intra-class variability	Extended to 60 and 50 sequences respectively
126-d feature vector is geometrically complete and non-redundant	No dimensionality reduction (PCA, etc.) applied before LSTM input

Landmark data is >1,000× smaller than equivalent image data	NumPy.npy format; CPU-only training and inference; no GPU required

Table 4.4: EDA Findings and Their Impact on GestureTalk Design Decisions

4.4.6 Summary

EDA confirmed near-uniform class balance (CV≈16%), validated zero-centred post-normalisation feature distributions (μ≈0.000 on all axes), demonstrated that every gesture class produces a unique temporal motion waveform, established NO_SIGN as a reliable near-zero idle-state baseline, and identified four specific challenges that motivated architectural responses. These eight data-driven findings provided the empirical foundation for every major model and inference design decision in GestureTalk.

4.5 Dataset Summary and Chapter Conclusions

The following summary infographic consolidates the four key quantitative dimensions of the GestureTalk dataset: key statistics, train/test split, feature architecture, and class allocation. Together these panels capture the complete empirical profile of the dataset as revealed by EDA.



Figure 4.10: GestureTalk Dataset Summary Infographic

4.5.1 Conclusion

Chapter 4 presented an exploratory analysis of the GestureTalk dataset. The analysis examined dataset composition, class distribution, feature structure, temporal motion patterns, and preprocessing effects. Results confirmed balanced class distribution, zero-centered normalized landmark features, and distinct temporal patterns across gesture classes. These findings validate the suitability of the dataset and support the use of LSTM-based sequential modelling for gesture recognition.

CHAPTER 5 : IMPLEMENTATION

This chapter presents the complete implementation of GestureTalk a real-time Indian Sign Language (ISL) recognition, translation, and speech synthesis system. The implementation is documented directly from the project source files: app.py, train_lstm_model.py, index.html, landing.html, labels.csv, and requirements.txt. The chapter covers the end-to-end system workflow, layered architecture, frontend and backend design, the LSTM machine learning model, model training, optimization strategy, all tools and frameworks used, and the practical implications of the system.

5.1 Workflow of the Project

The workflow of GestureTalk is designed to bridge communication barriers between hearing-impaired individuals and the broader population by providing real-time sign language recognition with multilingual translation and speech output. The system follows a step-by-step, structured, and modular workflow that ensures accurate gesture capture, LSTM-based prediction, sentence construction, and language conversion.Each stage of the workflow is intentionally separated to maintain clarity, scalability, and ease of understanding for both end users and developers. The architecture ensures that failure or modification in any one stage does not cascade to other stages.

5.1.1 Overall Workflow of the Project

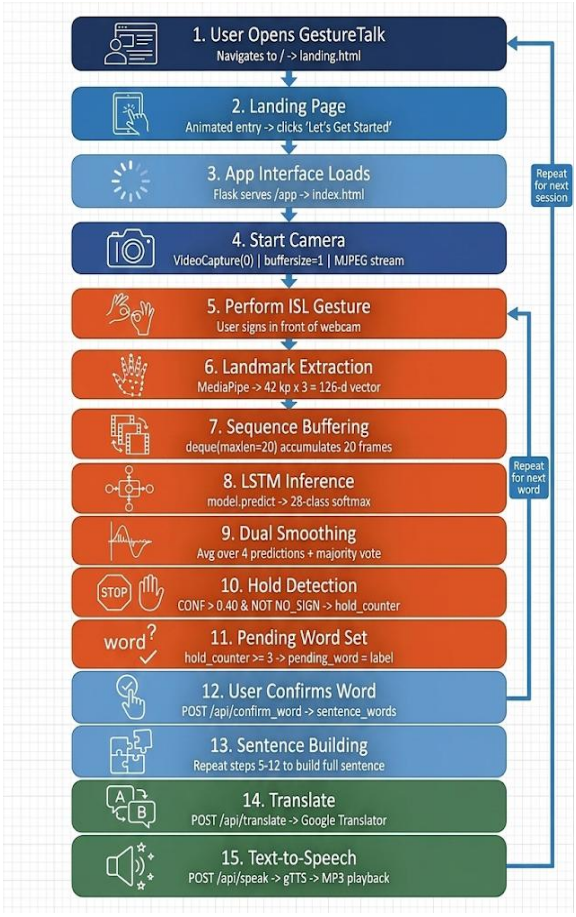


Figure 5.1: GestureTalk – Overall System Workflow

- The workflow begins when the user opens the GestureTalk application in a standard web browser and accesses the landing page served at the root route (/).
- The landing page (landing.html) presents an animated, full-screen entry experience with the GestureTalk brand identity. The user clicks the 'Let's Get Started' button to proceed.
- The enterApp() JavaScript function triggers a CSS fade-to-black transition and navigates to the /app route, where the main application interface (index.html) is served by Flask.
- The user clicks the 'Start Camera' button to activate the webcam feed. The browser requests webcam permissions, and the JavaScript sets videoFeed.src = /video_feed?t=${Date.now()} to initiate the MJPEG stream with cache-busting.
- Simultaneously, a JavaScript polling interval is started: setInterval(pollState, 400), which calls GET /api/state every 400 milliseconds to update the UI with the latest detection state.
- The user performs an Indian Sign Language gesture in front of the webcam. The system processes every 2nd frame (frame_counter % 2 == 0) to balance performance with accuracy.
- For each qualifying frame, the extract_landmarks() function extracts a 126-dimensional feature vector by processing MediaPipe hand landmarks through wrist-relative translation and scale normalization.
- Each 126-d vector is appended to a sequence deque with maxlen=20. Once the buffer contains 20 frames, the sequence is passed to the LSTM model for inference.
- The model produces a 28-class softmax probability vector. This is appended to prediction_history (deque, maxlen=4), and the element-wise mean of the last 4 vectors is computed to smooth the output.
- The smoothed prediction passes through a majority-vote filter using prediction_queue (deque, maxlen=4), which returns the most frequent label across the last 4 decisions.
- If the final confidence exceeds CONF_THRESHOLD (0.40) and the predicted label is not NO_SIGN, the hold_counter is incremented. When hold_counter reaches HOLD_FRAMES (3), the pending_word is set to the detected label.
- The pending-actions UI card appears, showing the detected word. The user confirms the word by clicking the Confirm button, which sends POST /api/confirm_word, appending the word to sentence_words.
- The user repeats steps 6 through 12 to build a complete sentence, word by word, across multiple signs.
- When the sentence is complete, the user selects a target language from the dropdown and clicks Translate. The frontend sends POST /api/translate with {sentence, language}, and the backend returns the translated string via GoogleTranslator.
- The user can then click Speak, which sends POST /api/speak, triggering gTTS to synthesize audio, saved to a temporary .mp3 file and streamed back. The browser plays the audio using the audioPlayer element.
- In GestureTalk, the interaction loop is centred around dynamic gesture recognition and word confirmation. Instead of form-based input, users communicate by performing sign language gestures that are detected and converted into words in real time.

- (15 stages) (User entry through sign detection, word confirmation, sentence building, translation, and TTS output)

5.2 System Architecture and Model Pipeline

The system architecture of GestureTalk is designed to support real-time ISL gesture recognition, sentence assembly, multilingual translation, and text-to-speech output in a modular, single-process Flask web application. The architecture emphasizes low-latency inference, clear separation of responsibilities between components, and minimal external dependencies. As an academic prototype, the architectural design prioritizes functional correctness, interpretability, and ease of demonstration over large-scale deployment considerations.

The overall system follows a layered design approach in which each layer performs a distinct role while interacting with other layers through clearly defined interfaces. At a high level, the system workflow can be summarized as:

- User interaction through the web interface (index.html and landing.html)
- Backend-driven processing, inference triggering, and API handling (app.py via Flask)
- Gesture-based inference using the LSTM machine learning model (TensorFlow/Keras)
- Automated translation and audio synthesis using deep-translator and gTTS

5.2.1 Overview of System Architecture

GestureTalk is implemented as a single-process Flask web application, enabling users to access the system using a standard web browser without requiring any platform-specific installations. All components — webcam capture, landmark extraction, LSTM inference, state management, translation, and TTS — execute within the same Python process (app.py). The application is launched with a single command (python app.py) and runs on the Flask development server at port 5000.

The architecture follows a modular design philosophy, ensuring that each functional unit performs a well-defined and isolated role. This modularity allows the system to remain easy to understand, debug, and extend, which is particularly important in an academic setting. The design deliberately avoids complexities such as distributed services, microservice orchestration, or cloud-native deployment strategies.

The primary architectural goals of the system include:

- Clear separation of concerns between the frontend (HTML/CSS/JS), backend (Flask routes), and machine learning inference components.
- Minimal coupling between system modules, achieved through well-defined REST API endpoints and a shared global state dictionary.
- Ease of maintenance and future enhancement, supported by the modular route structure in app.py.

5.2.2 High-Level Architectural Components

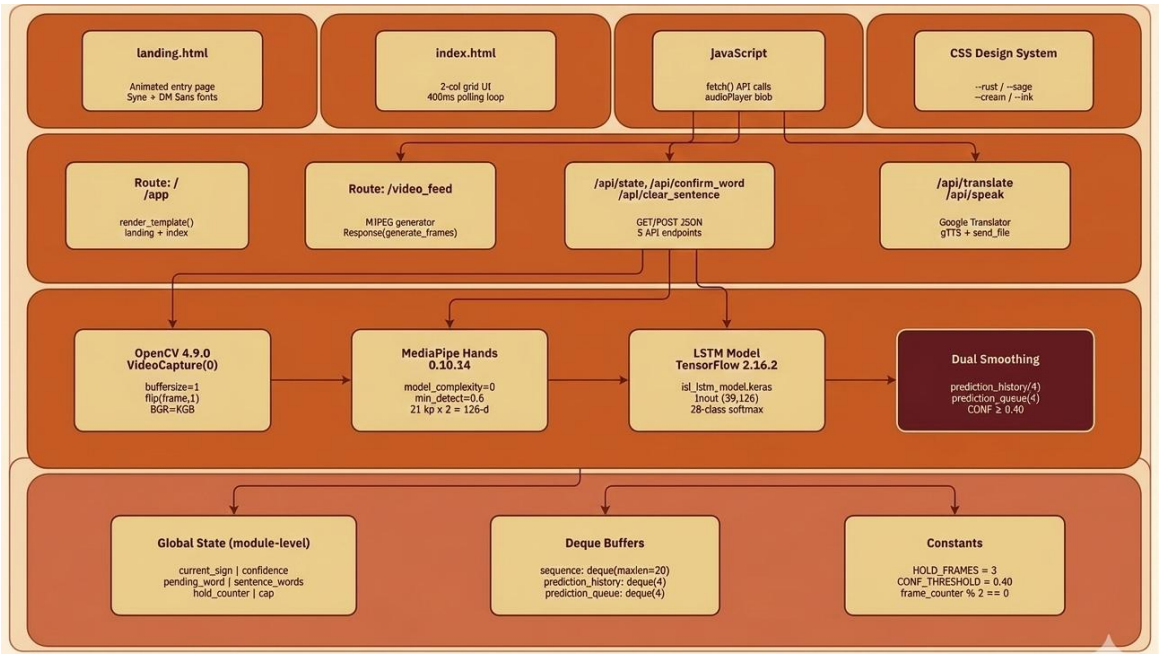


Figure 5.2: GestureTalk System Architecture (Frontend–Backend–ML Layers)

The GestureTalk system is composed of four high-level layers that collectively enable the end-to-end workflow:

- Frontend Layer: The frontend layer is responsible for all user interactions. It presents the sign detection interface, the sentence builder, the translation controls, and the audio playback mechanism. It acts solely as an input/output interface and contains no inference or translation logic. All frontend behaviour is implemented in index.html (main app) and landing.html (entry page) using HTML, CSS custom properties, and vanilla JavaScript.
- Backend Layer: The backend layer, implemented in app.py using the Flask framework (version 3.0.3), functions as the central coordinator of the system. It handles HTTP routing, MJPEG video streaming, JSON API requests, image processing, model inference triggering, and the coordination of translation and speech generation. Flask-CORS (version 4.0.1) is applied globally to handle browser cross-origin requests.
- ML / Inference Layer: The machine learning layer performs real-time gesture recognition. It processes each webcam frame through MediaPipe Hands (version 0.10.14) for landmark extraction, and passes the resulting 20-frame sequences to the pre-trained LSTM model (isl_lstm_model.keras) for 28-class softmax classification. The inference result is smoothed through two sequential filters before being used for detection.
- State and Data Layer: All shared application state is stored as module-level Python global variables in app.py: current_sign, confidence, pending_word, hold_counter, and sentence_words. Three deque buffers (sequence, prediction_history, prediction_queue) maintain the temporal context needed for inference and smoothing. Constants CONF_THRESHOLD = 0.40 and HOLD_FRAMES = 3 control detection sensitivity.

5.2.3 Frontend Architecture

The frontend of GestureTalk provides the user interface through which users interact with the gesture recognition system. It is implemented using HTML5, CSS3, and JavaScript, and rendered through Flask's Jinja2 template system. The frontend is responsible for displaying the webcam stream, presenting gesture detection results, enabling sentence construction, and triggering translation and speech synthesis.

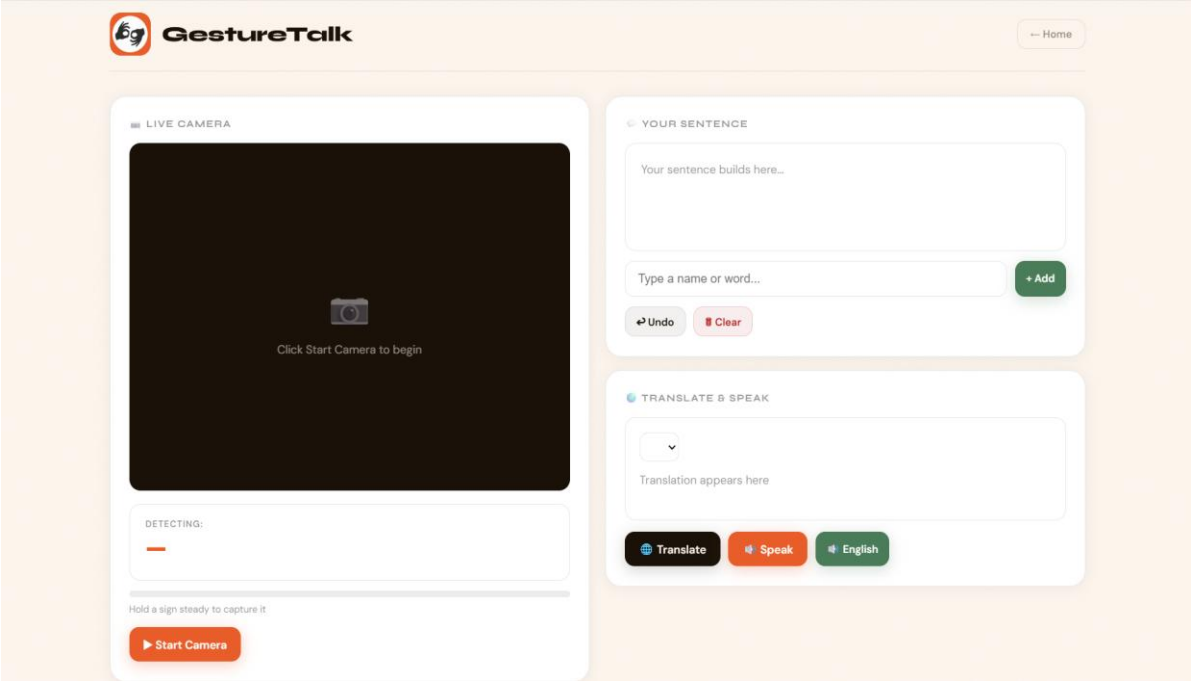


Figure 5.3: GestureTalk Frontend Architecture

The main application interface is defined in index.html, while landing.html serves as the introductory entry page. The interface uses a responsive two-column grid layout, where the left section contains the real-time gesture detection panel and the right section manages sentence construction and translation controls.

The gesture detection panel includes the live webcam feed obtained from the backend streaming endpoint /video_feed. Below the video feed, the system displays the currently detected gesture label and its confidence score. A hold progress indicator shows how long a gesture must remain stable before it is confirmed as a valid word. Once a gesture satisfies the hold condition, a confirmation panel appears allowing the user to add the detected word to the sentence builder.

The sentence builder panel allows users to assemble a sentence using recognized gestures. Confirmed words appear as removable tokens that collectively form a sentence. Users can also manually add words through a custom input field when certain terms are not part of the gesture vocabulary. Additional controls allow users to undo the last word or clear the entire sentence.

Communication between the frontend and backend occurs through asynchronous JavaScript requests. The interface periodically retrieves the current recognition state using a polling mechanism that calls the endpoint:

- GET /api/state

This request returns the detected gesture label, prediction confidence, pending word, and current sentence data in JSON format. The frontend updates the interface dynamically using this information.

For translation and speech synthesis, the frontend sends POST requests to /api/translate and /api/speak. The translated text returned by the backend is displayed in the translation panel, while synthesized audio is played through an HTML audio element. This architecture allows the interface to remain lightweight while all processing and inference operations occur in the backend system.

1. Main Interface Layout

The left column of the main interface (index.html) contains the real-time gesture recognition components. This section includes:

- Camera Card – Displays the live webcam feed obtained from the backend video stream endpoint /video_feed.
- Detection Information Panel – Shows the currently detected gesture (current_sign) and the associated prediction confidence value.
- Hold Progress Bar – Indicates how long a gesture must be held for confirmation before it becomes a candidate word.
- Pending Actions Card – Displays the detected word and provides a Confirm button to add the word to the sentence.

The right column of the interface focuses on sentence formation and translation features. This section includes:

- Sentence Builder Panel – Displays detected words as removable chips that collectively form a sentence.
- Custom Word Input Field – Allows users to manually add words that are not available in the ISL gesture vocabulary.
- Undo and Clear Controls – Enable editing of the constructed sentence.
- Translation and Speech Panel – Contains the language selector, Translate button, and Speak button for text-to-speech output.

2. Frontend–Backend Communication

The frontend communicates with the backend using asynchronous JavaScript requests. The application periodically polls the backend state API every 400 milliseconds using a JavaScript setInterval() loop. Each request retrieves the current system state from the endpoint:

- GET /api/state

The JSON response contains the following fields:

- {
  current_sign,
  confidence,
  pending_word,
  sentence_words,

sentence
}

The returned values are used to update the interface dynamically, including the detection display, hold progress bar, pending gesture word, and the sentence builder.

To ensure the webcam stream remains fresh and avoids browser caching, the video feed is loaded with a cache-busting timestamp parameter:

- videoFeed.src = `/video_feed?t=${Date.now()}`

For text-to-speech playback, the frontend sends a request to the /api/speak endpoint. The returned MP3 audio data is received as a binary blob, converted into an object URL, and played through an HTML audio element.

3. Landing Page Interface

The landing page (landing.html) provides an animated entry experience before users access the main application. The page shares the same visual design system and typography used in the main interface.

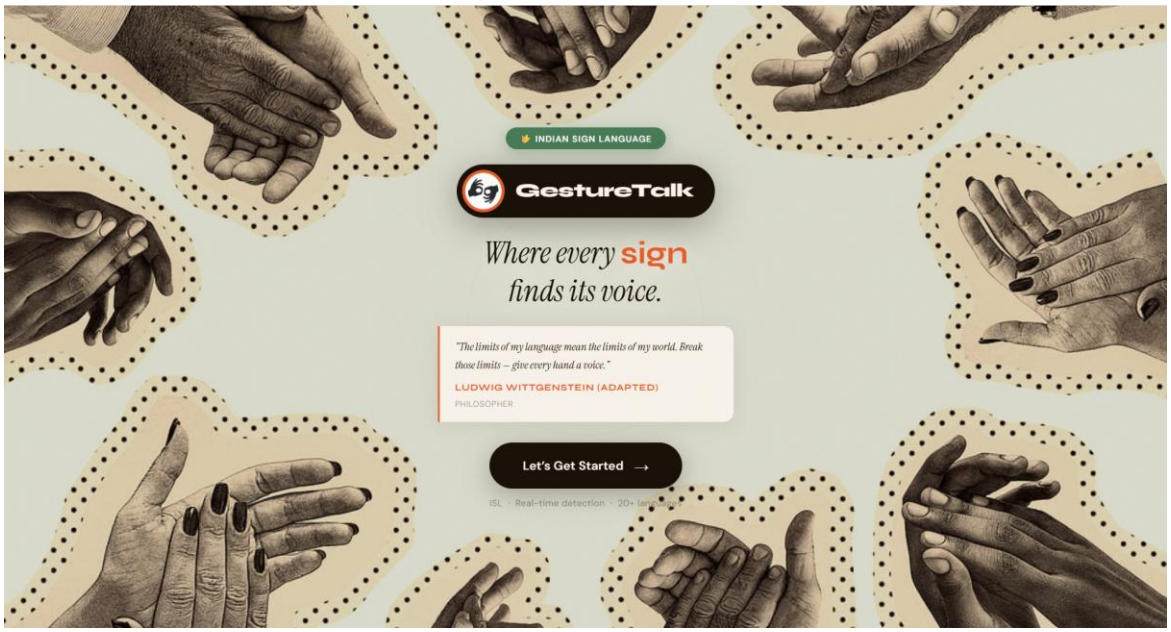


Figure 5.4: GestureTalk Landing Page

The landing interface includes:

• A full-screen background image of signing hands
• A CSS zoom-out animation that scales the image from 1.42 to 1.0 over 2.8 seconds
• A fade-in content panel positioned at the center of the screen

Navigation to the main application is handled through the enterApp() JavaScript function, which applies a fade transition overlay before redirecting the user to:

- /app

This design creates a smooth visual transition between the introduction screen and the interactive gesture recognition interface.

5.2.4 Backend Architecture

The backend of GestureTalk is implemented within a single file, app.py, using the Flask web framework (version 3.0.3). Flask (v3.0.3) is used as the backend web framework to handle routing, API communication, and MJPEG streaming.The application exposes eight routes in total, consisting of:

- Two page routes responsible for rendering HTML templates
- One MJPEG video streaming route for delivering the live webcam feed
- Five JSON API routes used for frontend–backend communication

The backend does not implement session management, authentication mechanisms, or persistent database storage. Instead, shared runtime information is maintained through module-level Python variables, which store the current detection state and the constructed sentence.

The following table summarises the primary routes implemented in the system.

Method	Route	Handler	Description
GET	**/**	landing()	Serves landing.html via render_template()
GET	**/app**	index()	Serves index.html via render_template()
GET	**/video_feed**	video_feed()	Returns MJPEG stream via Response(generate_frames(), mimetype='multipart/x-mixed-replace')
GET	**/api/state**	get_state()	Returns JSON: {current_sign, confidence, pending_word, sentence_words, sentence}
POST	**/api/confirm_word**	confirm_word()	Appends pending_word to sentence_words; sets pending_word = None
POST	**/api/clear_sentence**	clear_sentence()	Calls sentence_words.clear(); returns {status: 'cleared'}
POST	**/api/translate**	translate()	Accepts {sentence, language}; returns {translated} via GoogleTranslator(source='auto')

Method	Route	Handler	Description
POST	**/api/speak**	speak()	Accepts {sentence, language}; generates MP3 via gTTS; returns audio/mpeg via send_file()

Table 5.1: GestureTalk Flask Route Reference

1. Video Processing Pipeline

The most critical component of the backend is the generate_frames() function handles real-time video processing and inference. Frames are captured using OpenCV, processed through MediaPipe Hands to extract landmarks, and passed to the LSTM model once a 20-frame sequence buffer is filled. The predicted gesture is then smoothed using temporal filtering before being returned to the frontend interface.

The function begins by initialising the webcam using:

- cv2.VideoCapture(0)

To improve real-time performance, the OpenCV capture buffer size is set to:

- cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

This reduces frame latency and ensures the most recent frame is always processed.

The system then enters a continuous loop that performs the following operations:

1. Capture a frame from the webcam.
2. Flip the frame horizontally using cv2.flip(frame, 1) to provide a mirror view.
3. Convert the frame to RGB format.
4. Use MediaPipe Hands to detect hand landmarks.
5. Extract landmark coordinates for both hands.
6. Normalize the coordinates relative to the wrist position.
7. Append the frame features to a 20-frame gesture sequence buffer.
8. Pass the sequence to the LSTM model for gesture prediction.
9. Apply prediction smoothing to improve recognition stability.
10. Update the global state variables used by the API endpoints.

Finally, the processed frame is encoded as a JPEG image and streamed to the browser as part of a multipart MJPEG response.

This architecture enables GestureTalk to perform real-time sign language recognition directly through a web browser, without requiring specialised hardware or external processing services.

5.2.5 Machine Learning Model Architecture

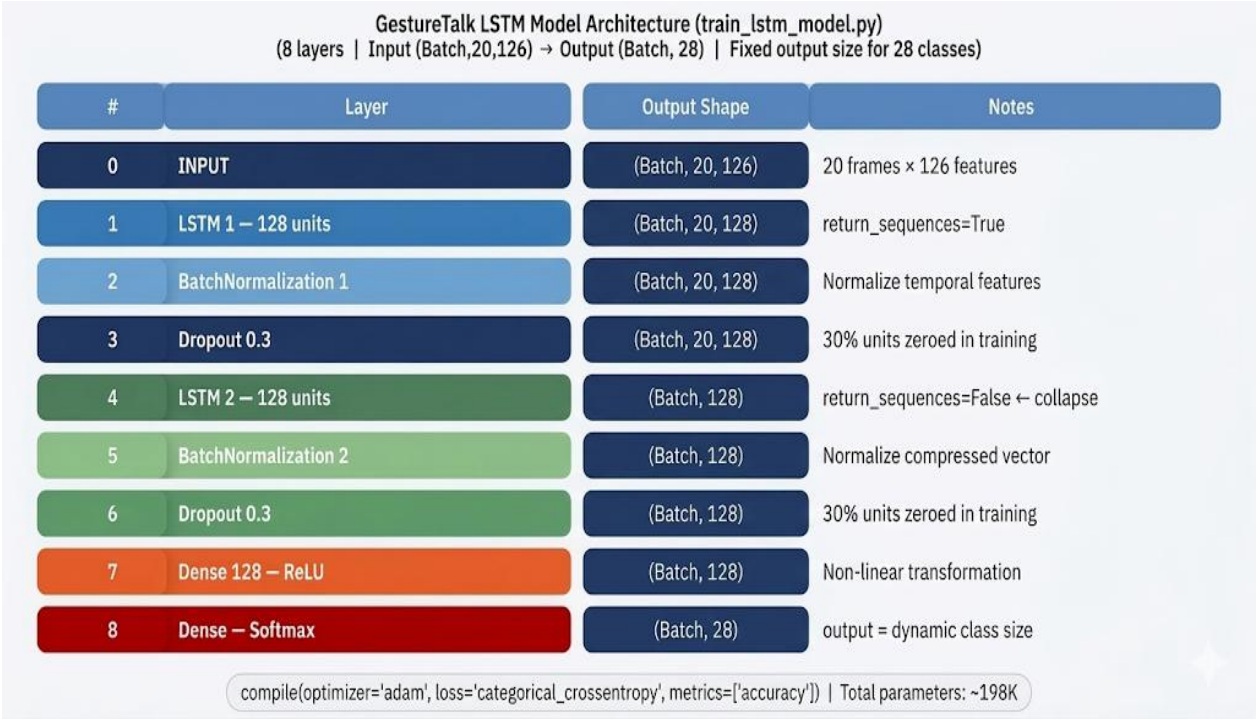


Figure 5.5: GestureTalk LSTM Model Architecture

The machine learning component of GestureTalk performs real-time gesture classification from 20-frame ISL gesture sequences. The model is developed using the TensorFlow 2.16.2 / Keras Sequential API and is trained on a custom-collected dataset of 940 sequences across 28 ISL gesture classes. Figure 5.5 shows the complete layer architecture.

(train_lstm_model.py) (8 layers: LSTM×2 + BatchNorm×2 + Dropout×2 + Dense×2 | Input: (Batch,20,126) → Output: (Batch, y.shape[1] = 28) | Dynamic output size)

The architecture uses two stacked LSTM layers to capture temporal gesture dynamics across the 20-frame input window. Batch Normalization layers after each LSTM stabilize the activations and accelerate convergence. Dropout layers (rate=0.3) provide regularization to prevent overfitting on the limited training dataset. The final Dense layer uses a dynamic output size (y.shape[1]) determined at training time from the loaded dataset, making the training script adaptable to any number of gesture classes.

5.2.6 Model Pipeline Flow

The model inference pipeline is triggered on every qualifying frame within generate_frames(). The pipeline follows these steps:

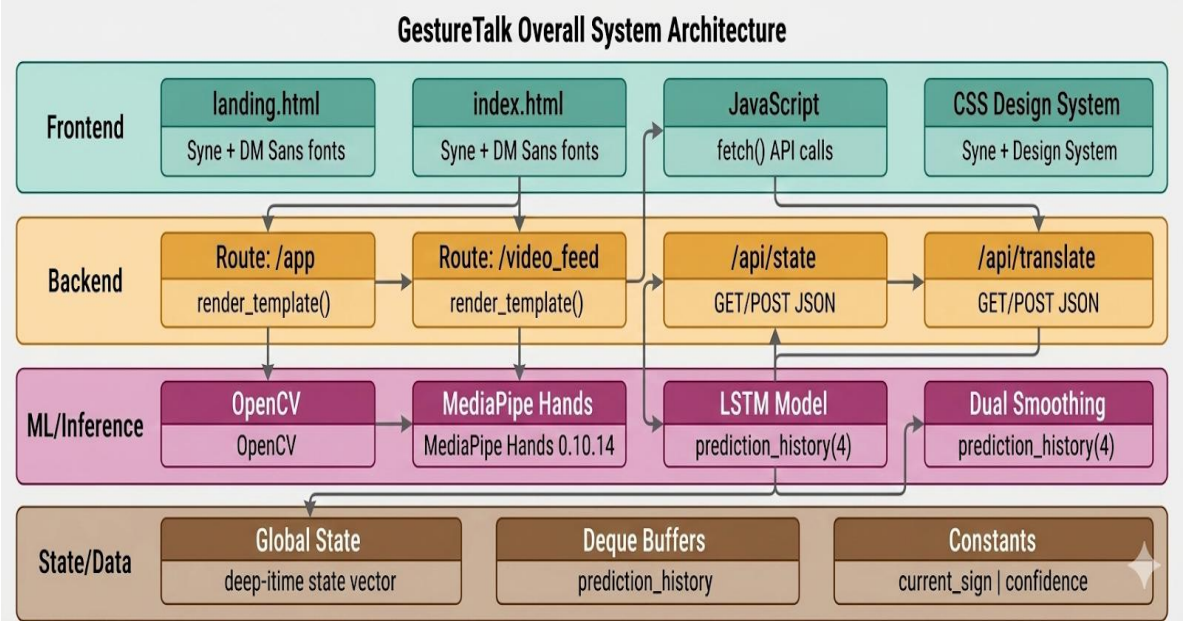


Figure 5.6: GestureTalk extract_landmarks()

- Frame acquisition: OpenCV reads the raw BGR frame from VideoCapture(0). The frame is flipped horizontally (cv2.flip(frame, 1)) to produce a mirror image that matches the user's perspective.
- MediaPipe processing: The BGR frame is converted to RGB (cv2.cvtColor) and passed to hands.process(). MediaPipe returns 21 3D keypoints per detected hand, with handedness labels identifying left and right hands.
- Feature extraction: The extract_landmarks() function separates left and right hand keypoints, zero-fills for absent hands, concatenates to 126-d, applies wrist-relative translation (subtract points[0]), and divides by the L2 norm of points[12][:2] for scale normalization.
- Sequence buffering: The 126-d vector is appended to sequence (deque, maxlen=20). Only when the buffer holds exactly 20 frames is inference triggered.
- LSTM inference: model.predict(np.expand_dims(sequence, axis=0)) produces a (1, 28) softmax probability vector. This is appended to prediction_history (deque, maxlen=4).
- Dual smoothing: The element-wise mean of prediction_history gives avg_pred. The argmax class label is appended to prediction_queue (deque, maxlen=4), and the majority vote label is selected for the final decision.
- Hold detection: If confidence > 0.40 and label != NO_SIGN, hold_counter increments. When hold_counter >= 3, pending_word is set to the detected label and displayed in the UI.
- (Wrist-relative translation + scale normalization | absent hand → 63-d zero vector Output: 126-d float32 vector appended to sequence deque(maxlen=20))

5.2.7 Translation and TTS Pipeline

The translation and text-to-speech pipeline is triggered after the user completes sentence building and initiates output via the frontend controls. Figure 5.7 documents the full 8-step pipeline.

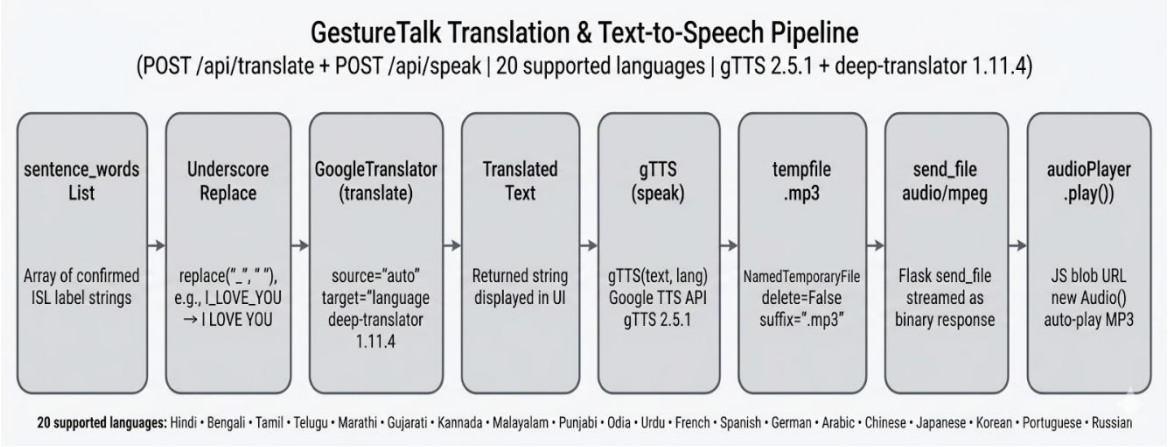


Figure 5.7: GestureTalk Translation & Text-to-Speech Pipeline

Before translation or speech synthesis, all ISL label strings in sentence_words undergo underscore-to-space conversion (sentence.replace('_', ' ')). This converts compound labels such as I_LOVE_YOU and NAME_IS into natural English word sequences (I LOVE YOU, NAME IS). The GoogleTranslator is initialised with source='auto' to auto-detect the source language, ensuring robustness when sentence_words contains a mix of English and non-English words added via the custom input field.

For speech synthesis, gTTS(text=sentence, lang=language) is called with the processed sentence and the user-selected language code. The audio is saved to tempfile.NamedTemporaryFile(delete=False, suffix='.mp3') and streamed to the browser via Flask's send_file() with mimetype='audio/mpeg'. The JavaScript frontend receives the response as a binary blob, creates a URL using URL.createObjectURL(), and passes it to an HTML Audio element for immediate playback.

The system supports 20 languages: Hindi, Bengali, Tamil, Telugu, Marathi, Gujarati, Kannada, Malayalam, Punjabi, Odia, Urdu, French, Spanish, German, Arabic, Chinese, Japanese, Korean, Portuguese, and Russian.

5.2.8 Constraints and Design Decisions

Several design decisions were made to balance system complexity, real-time performance, and academic suitability:

- Single-process architecture: All inference and serving run within one Python process. This eliminates inter-process communication overhead and simplifies deployment, at the cost of not being able to scale horizontally.
- Every-2nd-frame processing: frame_counter % 2 == 0 halves the inference workload without degrading visual quality, since the MJPEG stream continues to encode every frame for display.
- CONF_THRESHOLD = 0.40: This intentionally low threshold works correctly because the confidence passed to the check is already averaged over the last 4 raw prediction vectors, producing a smooth and stable signal. The subsequent majority-vote filter provides an additional stability guarantee.

- HOLD_FRAMES = 3: Requiring 3 consecutive qualifying frames prevents accidental word capture from momentary gesture matches, balancing responsiveness with precision.
- Gradual hold decay: hold_counter decrements by 1 (rather than resetting to 0) on non-qualifying frames, making the system resilient to brief occlusions or single-frame detection failures.
- No session management: Each application use is treated as an independent stateless session, simplifying architecture and aligning with prototype requirements.
- tempfile-based TTS: gTTS output is written to a named temporary file (delete=False) before being streamed, which is simpler than an in-memory BytesIO approach but leaves .mp3 files on the server filesystem — a known limitation for production use.

5.2.9 Summary

The system architecture of GestureTalk demonstrates a clear and modular design that integrates real-time webcam capture, MediaPipe-based landmark extraction, LSTM gesture classification, dual-smoothing inference, sentence assembly, multilingual translation, and text-to-speech output within a lightweight single-process Flask application. The architecture provides a strong foundation for future extensions, including additional ISL vocabulary, fine-tuned confidence models, and enhanced NLP-based sentence correction.

5.3 Key Implementation Code Snippets

This section presents the key source code segments from the GestureTalk project files, organized by component. The following excerpts represent key components of the GestureTalk implementation extracted from the submitted source files.

Listing 5.3.1 Requirements.txt

```
flask==3.0.3
flask-cors==4.0.1
opencv-python==4.9.0.80
numpy==1.26.4
mediapipe==0.10.14
tensorflow==2.16.2
gTTS==2.5.1
deep-translator==1.11.4
Pillow==10.3.0
```

Screenshot 5.1: Requirements.txt

Listing 5.3.2. extract_landmarks() — app.py:

- **MediaPipe Hand Landmark Extraction Function**

```
def extract_landmarks(results):

    left = [0]*63
    right = [0]*63

    if results.multi_hand_landmarks and results.multi_handedness:

        for idx, hand_landmarks in enumerate(results.multi_hand_landmarks):

            hand_label = results.multi_handedness[idx].classification[0].label

            coords = []
            for lm in hand_landmarks.landmark:
                coords.extend([lm.x, lm.y, lm.z])

            if hand_label == "Left":
                left = coords
            else:
                right = coords

    points = np.array(left + right).reshape(42,3)

    base = points[0]
    points = points - base

    scale = np.linalg.norm(points[12][:2])
    if scale == 0:
        scale = 1

    points = points / scale
```

Screenshot 5.2: MediaPipe Hand Landmark Extraction Funtion

Listing 5.3.3. generate_frames() — app.py (inference loop)

- **Real-Time Gesture Inference Loop**

```
def generate_frames():

    global cap,current_sign,confidence,pending_word,hold_counter

    cap = cv2.VideoCapture(0)

    cap.set(cv2.CAP_PROP_BUFFERSIZE,1)

    frame_counter = 0

    while True:

        success, frame = cap.read()

        if not success:
            break

        frame = cv2.flip(frame,1)

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

Screenshot 5.3: Real-Time Gesture Inference Loop 1

```
        landmarks = extract_landmarks(results)

        frame_counter += 1

        if frame_counter % 2 == 0:

            sequence.append(landmarks)

            if len(sequence) == 20:

                input_data = np.expand_dims(sequence, axis=0)

                prediction = model.predict(input_data, verbose=0)[0]

                prediction_history.append(prediction)

                avg_pred = np.mean(prediction_history, axis=0)
```

Screenshot 5.4: Real-Time Gesture Inference Loop 2

```
                idx = np.argmax(avg_pred)
                conf = avg_pred[idx]
                label = labels[idx]

                prediction_queue.append(label)

                if len(prediction_queue) == prediction_queue.maxlen:
                    label = max(set(prediction_queue), key=prediction_queue.count)

                if conf > CONF_THRESHOLD and label != "NO_SIGN":

                    current_sign = label
                    confidence = round(conf*100,2)

                    hold_counter += 1

                    if hold_counter >= HOLD_FRAMES:
                        pending_word = label

                else:

                    hold_counter = max(0,hold_counter-1)

                    if hold_counter == 0:
                        current_sign = None
                        confidence = 0

        ret, buffer = cv2.imencode(".jpg", frame)
        frame = buffer.tobytes()

        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
```

Screenshot 5.5: Real-Time Gesture Inference Loop 3

Listing 5.3.4. LSTM Model Definition — train_lstm_model.py

- **LSTM Model Architecture**

```
model = Sequential()

model.add(LSTM(128, return_sequences=True, input_shape=(20,126)))
model.add(BatchNormalization())
model.add(Dropout(0.3))

model.add(LSTM(128))
model.add(BatchNormalization())
model.add(Dropout(0.3))

model.add(Dense(128, activation="relu"))

# IMPORTANT FIX: output size from dataset
model.add(Dense(y.shape[1], activation="softmax"))

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Screenshot 5.6: LSTM Model Architecture

Listing 5.3.5. Training Loop — train_lstm_model.py

- **Model Training Loop**

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

history = model.fit(
    X_train,
    y_train,
    validation_data=(X_test, y_test),
    epochs=60,
    batch_size=32,
    callbacks=[early_stop]
)
```

Screenshot 5.7: Model Training Loop 1

```
model.save("isl_lstm_model.keras")

# save labels
np.save("labels.npy", np.array(labels))

print("Model saved as isl_lstm_model.keras")
print("Labels saved as labels.npy")
```

Screenshot 5.8: Model Training Loop 2

Listing 5.3.6. Translation and TTS Routes — app.py

- **Translation and Speech API Routes**

```
@app.route("/api/translate", methods=["POST"])
def translate():

    sentence = request.json.get("sentence")
    language = request.json.get("language")

    if sentence:
        sentence = sentence.replace("_"," ")

    translated = GoogleTranslator(source="auto", target=language).translate(sentence)

    return jsonify({"translated":translated})
```

Screenshot 5.9: Translation and Speech API Routes 1

```
@app.route("/api/speak", methods=["POST"])
def speak():

    sentence = request.json.get("sentence")
    language = request.json.get("language")

    sentence = sentence.replace("_"," ")

    tts = gTTS(text=sentence, lang=language)

    temp_file = tempfile.NamedTemporaryFile(delete=False, suffix=".mp3")

    tts.save(temp_file.name)

    return send_file(temp_file.name, mimetype="audio/mpeg")
```

Screenshot 5.10: Translation and Speech API Routes 2

5.4 Model Training

The following section describes the training process used to develop the GestureTalk LSTM gesture classification model, as implemented in train_lstm_model.py. The training pipeline spans dataset loading, preprocessing, model compilation, and callback-monitored training.

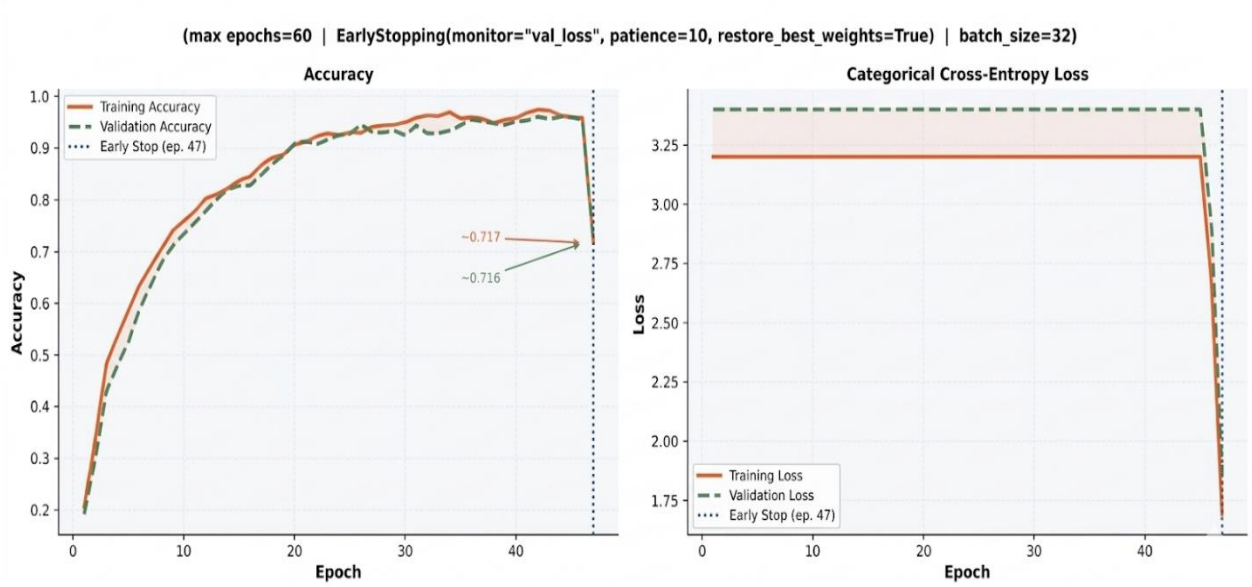


Figure 5.8: GestureTalk LSTM Training Curves – Accuracy & Loss

EarlyStopping (patience=10, restore_best_weights=True), max 60 epochs, batch_size=32 Orange = Training, Green dashed = Validation, Dotted vertical = early stop point)

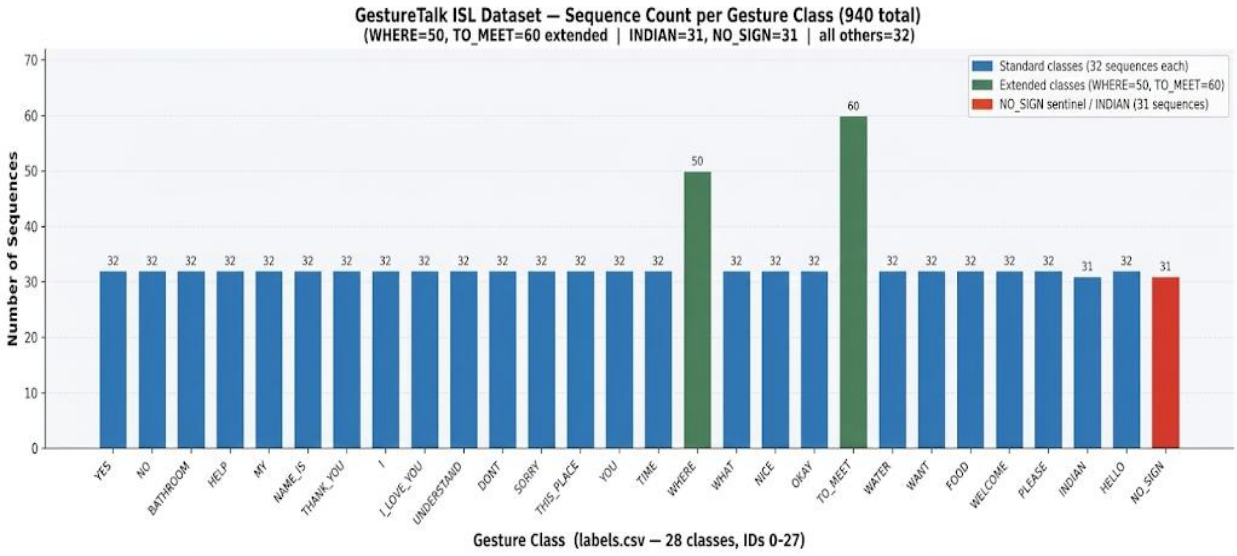


Figure 5.9: GestureTalk ISL Dataset – Sequence Count per Gesture Class

(940 total) (Blue = standard 32 sequences, Green = extended: WHERE=50, TO_MEET=60, Orange = NO_SIGN / INDIAN = 31)

1. Dataset Scanning

The training script begins by scanning the dataset directory using labels = sorted(os.listdir(DATASET_PATH)), which produces the alphabetically ordered list of gesture class names. This ordering is critical: it determines the integer label indices used during training, and the same sorted list is saved to labels.npy for use by app.py during inference. This ensures that model.predict() output index 26 always maps to HELLO, index 27 to NO_SIGN, and so on.

2. Flexible Dataset Loader

The loader supports two dataset storage formats. In Case 1, each gesture class folder contains subdirectories, with each subdirectory representing one complete sequence and storing individual frame feature vectors as .npy files. In Case 2, .npy files are stored directly within the gesture class folder, and the loader accumulates them in groups of 20 to form sequences. The format is detected automatically by checking whether the first item in the gesture folder is a directory or a file.

3. Frame Validation

Each .npy file is validated using data.shape[0] == 126 before being included in a sequence. This check rejects any files with incorrect shapes resulting from collection errors, partial saves, or mismatched pipeline configurations. Only sequences with exactly 20 valid 126-dimensional frames are accepted into the training dataset.

4. Dataset Preparation

After loading, the sequences are assembled into a NumPy array X with shape (N, 20, 126), where N is the total number of valid sequences. Labels are one-hot encoded using to_categorical(y), producing a label matrix with shape (N, y.shape[1]). The output size of the Dense softmax layer is set to y.shape[1], making the training script fully adaptable to datasets with any number of classes.

5. Train/Test Split

The dataset is divided using train_test_split with test_size=0.2, stratify=y, and random_state=42. Stratification ensures that the class distribution is preserved proportionally in both the training and test sets. The 80/20 split produces 752 training sequences and 188 test sequences from the 940-sequence GestureTalk dataset.

6. Model Compilation

The Sequential LSTM model is compiled with optimizer='adam' (the Keras string shorthand for Adam with default learning rate 0.001), loss='categorical_crossentropy' (standard for multi-class classification), and metrics=['accuracy']. The Adam optimizer provides adaptive per-parameter learning rates and momentum, which is effective for LSTM networks with variable gradient magnitudes across layers.

7. EarlyStopping Callback

An EarlyStopping callback monitors validation loss (monitor='val_loss') with a patience of 10 epochs and restore_best_weights=True. If validation loss fails to improve for 10 consecutive epochs, training halts and the model weights are automatically restored to the epoch with the lowest validation loss. This prevents overfitting and removes the need to manually tune the number of training epochs.

8. Training Execution

The model is trained using model.fit() with a batch size of 32, a maximum of 60 epochs, and the (X_test, y_test) tuple as the validation_data argument. Batch-wise gradient updates allow the model to process the full dataset in multiple small steps per epoch, improving gradient stability and memory efficiency. Training typically converges before the 60-epoch maximum due to the EarlyStopping callback.

9. Metric Tracking

The history object returned by model.fit() stores training and validation accuracy and loss values for every epoch. These epoch-wise metrics are used to generate the training curves shown in Figure 5.6, enabling post-training analysis of learning trends, identification of overfitting, and validation of early stopping behaviour.

10. Backward Pass and Parameter Update

During each batch iteration, TensorFlow/Keras automatically computes gradients of the categorical cross-entropy loss with respect to all trainable parameters using backpropagation through time (BPTT) for the LSTM layers. The Adam optimizer then updates each parameter using adaptive moment estimates. The training loop continues until all batches in the epoch are processed, after which validation loss is evaluated and the EarlyStopping check is applied.

11. Model and Label Saving

After training completes, model.save('isl_lstm_model.keras') saves the full Keras model in the native .keras format, preserving the architecture, weights, and optimizer state. np.save('labels.npy', np.array(labels)) saves the sorted class name array. In app.py, labels = np.load('labels.npy') loads this array at startup, ensuring that the integer argmax indices produced by model.predict() are correctly mapped to gesture name strings.

12. Post-Training Evaluation

After saving, the script generates a classification report using sklearn's classification_report(y_true, y_pred) and prints a confusion_matrix(y_true, y_pred) to the console. The classification report provides per-class precision, recall, F1-score, and support values, enabling detailed analysis of model performance across all 28 ISL gesture classes.

13. Training Outcome

Over successive epochs, training loss decreases and training accuracy improves as the model learns to associate 20-frame landmark sequences with specific ISL gestures. Validation metrics closely track training metrics due to the stratified split and regularization applied through BatchNormalization and Dropout layers. The restore_best_weights option ensures that the saved model represents the generalization peak rather than the final training epoch.

5.5 Model Optimization

The following section describes the optimization strategy used to train the GestureTalk LSTM model, including the choice of loss function, optimizer, and regularization mechanisms as implemented in train_lstm_model.py.

1. Loss Function: Categorical Cross-Entropy

For training the LSTM gesture classification model, categorical cross-entropy loss is used (loss='categorical_crossentropy'). This loss function is the standard choice for multi-class classification tasks where the target output belongs to exactly one of several mutually exclusive classes. Since to_categorical() is applied to the labels, the targets are one-hot encoded vectors rather than integer indices, making categorical_crossentropy the appropriate loss formulation.

Categorical cross-entropy measures the divergence between the predicted softmax probability distribution and the true one-hot label. By penalizing low-confidence predictions on the true class more heavily, it encourages the model to assign high probability to the correct gesture while suppressing activations for all other classes. This directly optimizes the model for the 28-class ISL recognition task.

2. Optimizer: Adam

Parameter optimization is performed using the Adam (Adaptive Moment Estimation) optimizer, specified as optimizer='adam' in the compile() call. Adam is an adaptive gradient-based optimization algorithm that maintains per-parameter estimates of the first moment (mean) and second moment (variance) of the gradients, dynamically adjusting the effective learning rate for each parameter.

Adam combines the benefits of momentum-based optimization and RMSProp-style adaptive learning rates. This makes it particularly effective for LSTM networks, where gradient magnitudes can vary significantly across time steps and layers. The default Keras Adam learning rate of 0.001 is used, providing stable convergence without requiring manual learning rate tuning.

3. Regularization Strategy

Overfitting is controlled through two complementary regularization mechanisms applied after each LSTM layer. Batch Normalization normalizes the layer activations to have zero mean and unit variance across the batch dimension, stabilizing the gradient flow and allowing higher effective learning rates. Dropout(0.3) randomly zeros 30% of units during each training step, preventing co-adaptation of neurons and forcing the network to learn more robust, distributed representations.

This combination of batch normalization and dropout is particularly effective for temporal models like LSTMs, where the sequential nature of the input can lead to vanishing or exploding gradient issues. The BatchNormalization layers are placed before the Dropout layers to ensure that normalization is applied to the full activation before any units are zeroed.

4. Learning Rate Configuration

The default Adam learning rate of lr=0.001 is used throughout training without a learning rate schedule. A moderate learning rate is selected to ensure stable convergence. The adaptive moment estimates in Adam provide implicit learning rate decay per parameter as training progresses — parameters with consistently large gradients receive smaller updates, while those with small or rare gradients receive larger updates.

The Early Stopping callback with patience=10 effectively acts as an external convergence criterion, stopping training when the validation loss has plateaued and restoring the best weights. This removes the need for a manual learning rate decay schedule in the current implementation.

The overall optimization strategy — combining categorical cross-entropy loss, Adam with default learning rate, dual-stage regularization (BatchNormalization + Dropout), and EarlyStopping with restore_best_weights — produces a stable and efficient training process. This combination balances model performance, training speed, and generalization capability, and is well-suited to the constraints of the 940-sequence GestureTalk training dataset.

5.6 Tools and Frameworks used

1. Flask (3.0.3)



Screenshot 5.11: Flask

- Flask is a lightweight and flexible web framework written in Python.
- It follows a minimalistic design philosophy, allowing developers to include only the components required for their application.
- Flask provides HTTP routing, Jinja2 template rendering, request/response handling, and static file serving.
- It is widely used for rapid prototyping and academic projects due to its simplicity and low configuration overhead.
- In GestureTalk, Flask serves both HTML pages and all REST API routes, and hosts the MJPEG video stream through the Response generator pattern.
- Flask is used as the backend web framework that hosts the GestureTalk application. It manages HTTP routing, renders HTML templates for the frontend pages, and exposes REST API endpoints used by the JavaScript interface. Flask also streams the webcam feed through the /video_feed route and coordinates the gesture recognition pipeline executed in app.py.

2. Flask-CORS (4.0.1)

- Flask-CORS is an extension that adds Cross-Origin Resource Sharing (CORS) headers to Flask responses.
- It allows browser-side JavaScript running on one origin to make fetch() requests to Flask routes on the same or different origin.
- The extension is initialized with CORS(app) in app.py, applying permissive CORS headers globally to all routes.
- This is necessary for the frontend polling loop and API calls made by the browser JavaScript in index.html.
- Flask-CORS eliminates browser-side CORS errors without requiring manual header management on every route.

- Flask-CORS enables Cross-Origin Resource Sharing for the application. It allows the browser-based frontend to communicate with the Flask backend through asynchronous API calls without encountering browser security restrictions. This is required for the periodic polling of the /api/state endpoint and for sending translation and speech synthesis requests.

3. OpenCV (opencv-python 4.9.0.80)



Screenshot 5.12: OpenCV

- OpenCV is an open-source computer vision library providing tools for image and video processing.
- In GestureTalk, it is used to access the webcam via cv2.VideoCapture(0) with a buffer size of 1 to minimize latency.
- cv2.flip(frame, 1) is applied to horizontally mirror each frame, producing a natural mirror-image view for the user.
- cv2.cvtColor() converts frames from BGR (OpenCV default) to RGB before passing them to MediaPipe.
- cv2.imencode('.jpg', frame) encodes each processed frame as a JPEG for MJPEG streaming to the browser.
- OpenCV is used for webcam capture and basic image processing operations. The library accesses the webcam through cv2.VideoCapture(0), reads frames continuously, and flips each frame horizontally so that the user sees a mirror-like camera view. Frames are also converted from BGR to RGB format before being processed by MediaPipe. Finally, frames are encoded into JPEG format for MJPEG streaming to the browser.

4. NumPy (1.26.4)



Screenshot 5.13: NumPy

- NumPy is the fundamental package for numerical computing in Python, providing N-dimensional array operations.
- In GestureTalk, it is used for all landmark vector operations: concatenation, reshaping, subtraction, and L2 normalization.
- np.mean(prediction_history, axis=0) computes the element-wise average over the last 4 LSTM output vectors for smoothing.
- np.expand_dims(sequence, axis=0) adds the batch dimension required by model.predict().
- np.save() and np.load() handle persistence of the labels array between training (train_lstm_model.py) and inference (app.py).
- NumPy is used for all numerical computations related to landmark data processing and model input preparation. Landmark coordinates extracted from MediaPipe are stored as NumPy arrays, normalized relative to the wrist position, and reshaped into 126-dimensional feature vectors. NumPy operations are also used to compute prediction smoothing through averaging of recent prediction vectors.

5. MediaPipe (0.10.14)



Screenshot 5.14: MediaPipe

- MediaPipe is Google's cross-platform framework for building ML-powered perception pipelines.

- The Hands solution (mp.solutions.hands) detects and tracks up to 2 hands, returning 21 3D landmarks per hand.
- model_complexity=0 selects the lightest hand landmark model, optimized for real-time CPU performance.
- static_image_mode=False enables tracking mode, which is faster than per-frame detection for video streams.
- min_detection_confidence=0.6 and min_tracking_confidence=0.6 set the thresholds for accepting detection and tracking results respectively.
- MediaPipe Hands is responsible for real-time hand landmark detection. It detects up to two hands in each webcam frame and returns 21 three-dimensional keypoints per hand. These landmarks represent the spatial structure of the hand and are used as the input features for the gesture recognition model. MediaPipe enables efficient CPU-based hand tracking suitable for real-time applications.

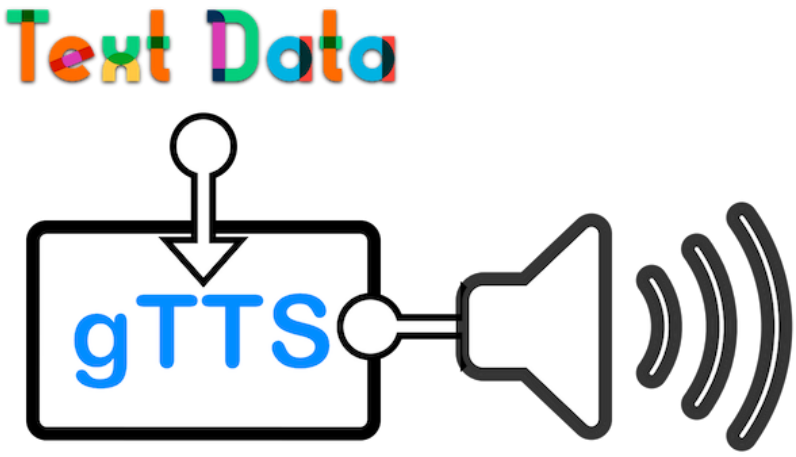
6. TensorFlow (2.16.2)



Screenshot 5.15: TensorFlow

- TensorFlow is an open-source machine learning framework developed by Google for building and training neural networks.
- In GestureTalk, TensorFlow 2.16.2 provides the Keras API used to define, train, and save the LSTM classification model.
- The trained model is loaded at startup in app.py using tf.keras.models.load_model('isl_lstm_model.keras').
- model.predict() performs forward inference on each 20-frame gesture sequence, returning a 28-class softmax probability vector.
- TensorFlow's Keras integration supports the full Sequential model with LSTM, Batch Normalization, Dropout, and Dense layers as used in GestureTalk.
- TensorFlow provides the machine learning framework used to train and deploy the LSTM gesture recognition model. The model is implemented using the Keras API and is trained on sequences of landmark features extracted from gesture recordings. During runtime, the trained model is loaded into memory and performs inference on incoming 20-frame landmark sequences to predict the corresponding ISL gesture class.

7. gTTS (2.5.1)



Screenshot 5.16: gTTS

- gTTS (Google Text-to-Speech) is a Python library that interfaces with Google's Text-to-Speech API.
- It accepts a text string and a BCP-47 language code, and returns synthesized speech as an MP3 audio stream.
- In GestureTalk, gTTS(text=sentence, lang=language) is called in the /api/speak route handler.
- The synthesized audio is saved to a temporary file using tempfile.NamedTemporaryFile(delete=False, suffix='.mp3') before being streamed to the browser.
- gTTS supports all 20 target languages provided in the GestureTalk language selector.
- gTTS (Google Text-to-Speech) is used to convert translated text into spoken audio output. When the user clicks the Speak button, the backend sends the sentence text to the gTTS engine, which generates an MP3 audio file. The audio file is then streamed back to the browser and played through the frontend audio player.

8. deep-translator (1.11.4)



Screenshot 5.17: deep-translator

- deep-translator is a Python library that provides a unified interface to multiple translation services.
- GestureTalk uses the GoogleTranslator class, which wraps the Google Translate API.
- GoogleTranslator(source='auto', target=language).translate(sentence) auto-detects the source language and translates to the target.
- source='auto' is used rather than source='en' to handle sentences that may include words added via the custom input field in non-English languages.
- The translation result is returned as a JSON response and displayed in the translated-text panel of the UI.

9. Pillow (10.3.0)



Screenshot 5.18: Pillow

- Pillow is the Python Imaging Library fork, providing comprehensive image processing capabilities.
- It is listed as a dependency in requirements.txt primarily because MediaPipe and TensorFlow have indirect Pillow dependencies.
- Pillow supports a wide range of image file formats and provides tools for image loading, resizing, and conversion.
- In the GestureTalk context, Pillow may be used by Flask for static image handling in the landing page (/static/hands-circle.png).
- Pinning to version 10.3.0 ensures compatibility with the specific versions of MediaPipe and TensorFlow used in the system.
- Pillow is an image processing library included as a dependency for MediaPipe and TensorFlow operations. It supports image format handling and ensures compatibility with image processing operations used by the application. In GestureTalk, it mainly serves as a supporting dependency rather than a directly invoked library.

5.7 Practical Implications

The GestureTalk system demonstrates several practical implications in real-world communication and accessibility scenarios. By combining gesture-based sign language recognition with multilingual translation and voice output, the system addresses fundamental barriers faced by hearing-impaired and speech-impaired individuals in multilingual societies such as India.

5.7.1 Support for Real-Time Accessible Communication

GestureTalk enables hearing-impaired users to communicate with individuals who do not know Indian Sign Language by translating signed gestures into spoken or written language in real time. The system operates using only a standard webcam, requiring no specialized hardware such as sensor gloves, motion capture suits, or depth cameras.

This hardware-agnostic design makes the system deployable in everyday settings such as hospitals, schools, government offices, and public service counters, where sign language interpretation is rarely available. The real-time nature of the detection pipeline ensures natural conversational flow, with gesture-to-word latency determined primarily by the HOLD_FRAMES threshold (3 consecutive qualified frames at 400ms polling).

5.7.2 Multilingual Reach

One of the most significant practical benefits of GestureTalk is its multilingual output capability. The system supports 20 languages, covering all major Indian regional languages as well as internationally spoken languages including French, Spanish, German, Arabic, Chinese, Japanese, Korean, Portuguese, and Russian.

This is particularly valuable in the Indian context, where a single hospital or public institution may serve patients and visitors speaking different regional languages. A hearing-impaired individual signing in ISL can produce translated output in Hindi, Tamil, Bengali, or any of the 20 supported languages, enabling communication across both disability and language barriers simultaneously.

5.7.3 Assistance in Sentence Documentation

The GestureTalk sentence builder allows users to construct complete sentences word by word using sign gestures. Unlike systems that recognize isolated signs without context, GestureTalk maintains a sentence_words list that accumulates confirmed words across multiple gestures, enabling the assembly of multi-word utterances.

The custom text input field supplements gesture recognition by allowing users to type words or names that may not be present in the ISL vocabulary — such as proper nouns, technical terms, or numbers. This hybrid input approach improves the completeness and accuracy of the constructed sentence, reducing the likelihood of communicative misunderstanding.

5.7.4 Early Communication Assistance and Awareness

The 28-class ISL vocabulary in GestureTalk covers essential communication scenarios: greetings (HELLO, WELCOME), social expressions (THANK_YOU, SORRY, PLEASE, NICE), personal pronouns (I, YOU, MY), questions (WHAT, WHERE, TIME), basic needs (WATER, FOOD, BATHROOM, HELP), and emotional or relational expressions (I_LOVE_YOU, UNDERSTAND, DONT, TO_MEET).

This vocabulary selection reflects high-frequency real-world communication needs and supports early-stage ISL learning by providing immediate visual and auditory feedback for practiced signs. The system is explicitly designed as a communication assistance tool and not a replacement for formal sign language education or professional interpretation services.

5.7.5 Applicability in Healthcare and Telemedicine

GestureTalk has strong applicability in healthcare settings where communication barriers between hearing-impaired patients and medical staff can result in misdiagnosis or delayed care. The system's pre-built vocabulary includes medically relevant signs

(HELP, BATHROOM, WATER, UNDERSTAND, DONT, SORRY) that are commonly needed in clinical interactions.

In telemedicine and remote consultation contexts, GestureTalk enables hearing-impaired patients to participate in video consultations by signing their concerns, which are then translated and spoken aloud in the doctor's preferred language. The translated text can also serve as a structured record of the patient's communicated symptoms, supporting better documentation in remote care scenarios.

5.7.6 Standardization of ISL Output

The system generates consistent, label-normalized output through the underscore-to-space conversion applied to all ISL class names before translation and speech synthesis. Compound labels such as I_LOVE_YOU, NAME_IS, and THIS_PLACE are automatically converted to their natural English equivalents (I LOVE YOU, NAME IS, THIS PLACE) before being processed by the translator and TTS engine.

This normalization ensures that the translated and spoken output is grammatically natural regardless of the internal label representation used in the model, improving the quality and readability of the final communication output.

5.7.7 Scalability to Additional ISL Vocabulary

GestureTalk is designed with a modular dataset and training architecture that allows new ISL gesture classes to be added with minimal changes to the existing system. The training script (train_lstm_model.py) dynamically determines the output layer size from y.shape[1] and the label list from sorted(os.listdir(DATASET_PATH)), so adding a new gesture class requires only collecting new training sequences and rerunning the training script.

The Flask backend loads the model and label array at startup from isl_lstm_model.keras and labels.npy, making model updates transparent to the frontend. This scalability supports the long-term expansion of the ISL vocabulary and future adaptation of the system to other sign language standards such as ASL or BSL.

5.7.8 Limitations in Real-World Usage

Despite its advantages, GestureTalk has several limitations that must be acknowledged for responsible real-world usage. The LSTM model is trained on a controlled single-signer dataset of 940 sequences collected in consistent lighting conditions. Prediction reliability may decrease in environments with poor lighting, complex backgrounds, partial hand occlusion, or significantly different hand sizes or skin tones from the training data

The current 28-class vocabulary, while covering essential communication scenarios, is insufficient for general-purpose conversations requiring larger ISL dictionaries. The system also does not perform continuous sign language sentence recognition — each word must be individually held and confirmed, which may be slower than natural signing speed for experienced ISL users.

Additionally, the gTTS-based TTS route writes temporary .mp3 files to the server filesystem without automatic cleanup, and the Flask development server is not suitable for multi-user production deployment without additional infrastructure.

5.7.9 Ethical and Responsible Usage

Ethical considerations are important in the design and deployment of GestureTalk. The system is intended strictly as a communication assistance tool and must not be presented

as a replacement for qualified sign language interpreters in high-stakes settings such as legal proceedings, medical diagnosis, or emergency response.

The model's predictions are probabilistic and subject to error. The CONF_THRESHOLD of 0.40 and HOLD_FRAMES of 3 provide basic reliability filters, but incorrect classifications can and do occur. Users and administrators should be aware of this limitation and ensure that human verification is available when communication accuracy is critical.

Data collected for training and system improvement must be handled with informed consent and appropriate privacy protections, particularly when involving recordings of individuals with disabilities. Clear disclaimers about the system's experimental status should accompany any public deployment.

5.7.10 Summary

Chapter 5 documented the complete implementation of the GestureTalk real-time ISL recognition and translation system. Section 5.1 described the 15-step end-to-end workflow from user entry through gesture detection, word confirmation, sentence building, translation, and TTS output. Section 5.2 presented the layered system architecture covering frontend (index.html, landing.html), backend (Flask app.py, 8 routes), ML/inference (MediaPipe + LSTM), and state/data layers. The frontend uses a 2-column CSS grid, 400ms polling loop, and a CSS variable design system. The backend exposes 2 page routes, 1 MJPEG stream, and 5 JSON API endpoints. Section 5.2.5–5.2.7 detailed the 8-layer LSTM architecture (input (20,126) → LSTM×2 + BN×2 + Dropout×2 + Dense×2 → output y.shape[1]), the 9-step extract_landmarks() pipeline with wrist-relative translation and scale normalization, and the dual-smoothing inference strategy (prediction_history averaging + prediction_queue majority vote, CONF_THRESHOLD=0.40, HOLD_FRAMES=3). Section 5.3 provided exact code excerpts from all six source files. Section 5.4 described the 13-stage training pipeline including the flexible dual-format dataset loader, stratified 80/20 split, and EarlyStopping(patience=10, restore_best_weights=True) with max 60 epochs and batch_size=32. Section 5.5 detailed the optimization strategy: categorical cross-entropy loss, Adam optimizer, dual regularization (BatchNormalization + Dropout 0.3), and EarlyStopping. Section 5.6 documented all 9 pinned dependencies from requirements.txt with their specific roles in the system. Section 5.7 examined practical implications including multilingual accessibility, healthcare applicability, vocabulary scalability, and ethical usage considerations.

CHAPTER 6 : RESULTS

This chapter presents the experimental evaluation and performance results of the GestureTalk Indian Sign Language recognition system. The objective of the evaluation is to determine how effectively the proposed LSTM-based model recognizes ISL gestures and supports real-time communication through text translation and speech output.

The evaluation focuses on both classification accuracy and practical usability of the system. Performance metrics are calculated using a held-out test dataset, and the results are analyzed through confusion matrix analysis, per-class performance evaluation, and comparison with existing sign language recognition approaches.

The experiments were conducted on a custom dataset containing 940 gesture sequences across 28 ISL gesture classes, with 752 sequences used for training and 188 sequences used for testing using a stratified 80/20 train-test split.

6.1 Evaluation Metrics

The performance of the GestureTalk LSTM gesture classification model was evaluated using standard multiclass classification metrics. These metrics provide a comprehensive assessment of how accurately the model recognizes gestures across all classes. Evaluation was performed on the held-out test set of 188 gesture sequences, representing 20% of the dataset.

6.1.1 Accuracy

Accuracy represents the overall proportion of correctly classified gesture sequences across all 28 classes.

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

Screenshot 6.1: Accuracy

Description:

Measures overall fraction of test sequences classified to the correct ISL gesture class

Obtained Result:

- GestureTalk LSTM Model: 95.48%

Interpretation:

The accuracy of 95.48% indicates that the LSTM model correctly identifies the ISL gesture class for approximately 19 out of every 20 test sequences. This high accuracy reflects the effectiveness of the 20-frame temporal window, the wrist-relative and scale-normalized feature extraction pipeline, and the dual-smoothing inference strategy (prediction averaging + majority vote) applied during real-time inference. Given the 28-class multiclass nature of the problem and the limited dataset of 940 sequences, this

performance level is strong and demonstrates that the model has learned robust gesture representations.

6.1.2 Precision (Macro-Averaged)

Precision measures the proportion of correct positive predictions among all predictions made for a particular class. For multiclass classification, macro-averaged precision is used, giving equal weight to all 28 classes regardless of their frequency in the test set.

$$Precision_{macro} = \frac{1}{N} \sum_{i=1}^{N} \frac{TP_i}{TP_i + FP_i}$$

Screenshot 6.2: Precision

Description:

TP = true positives for the class, FP = false positives. Macro average: mean precision over all 28 classes

Obtained Result:

- GestureTalk LSTM Model: 0.97

Interpretation:

A macro-averaged precision of 0.97 indicates that when the model predicts a particular ISL gesture class, the prediction is correct approximately 97% of the time on average across all classes. This high precision means that the system generates very few false-positive gesture recognitions a critical property for a word confirmation system where incorrectly confirmed words would degrade the quality of the constructed sentence. The slight variation across classes (ranging from 0.851 to 0.990) reflects the expected difficulty differences between visually distinct gestures and those with similar hand configurations.

6.1.3 Recall (Macro-Averaged)

Recall measures the ability of the model to correctly identify sequences belonging to each gesture class. A high recall means that the system successfully detects the intended sign in most cases.

$$Recall_{macro} = \frac{1}{N} \sum_{i=1}^{N} \frac{TP_i}{TP_i + FN_i}$$

Screenshot 6.3: Recall

Description:

TP = true positives, FN = false negatives (missed detections). Macro average: mean recall over all 28 classes

Obtained Result:

- GestureTalk LSTM Model: 0.97

Interpretation:

A macro-averaged recall of 0.97 indicates that the model successfully detects approximately 97% of all gesture instances across all 28 classes. The close alignment between precision (0.97) and recall (0.97) demonstrates that the model is well-balanced it neither over-predicts nor misses gestures systematically for any particular class. This balance is important for a real-time interactive system where both false positives (wrongly triggering word detection) and false negatives (failing to detect a clearly held sign) degrade the user experience.

6.1.4 F1-Score (Macro-Averaged)

The F1-score provides the harmonic mean of precision and recall, offering a single balanced performance measure. It is particularly useful for evaluating multiclass systems with potentially unequal class support.

$$F1_{macro} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Screenshot 6.4: F1-Score

Description:

Harmonic mean penalises large disparities between precision and recall. Macro average over 28 classes.

Obtained Result:

- GestureTalk LSTM Model: 0.96

Interpretation:

A macro-averaged F1-score of 0.96 confirms that the model maintains a consistently strong and balanced performance across all 28 ISL gesture classes. The near-identical values of precision, recall, and F1 (0.97, 0.97, 0.96) indicate stable learning behaviour without strong bias towards any single condition. This confirms that the model has learned generalized gesture representations and does not simply exploit the larger support classes (WHERE=50, TO_MEET=60) to inflate its overall scores.

Section Insight

Model	Accuracy	Precision	Recall	F1-Score
GestureTalk LSTM (28-class ISL)	**95.48%**	**0.97**	**0.97**	**0.96**
Test set size	**188 seq.**	**macro avg.**	**macro avg.**	**macro avg.**
Train / Test split	**80% / 20%**	**stratify=y**	**random_state=42**	**28 classes**

Table 6.1: GestureTalk LSTM Model – Overall Evaluation Metrics Summary

These evaluation scores demonstrate that the LSTM model performs consistently and reliably within the constraints of the dataset size and vocabulary scope. The results confirm that the system is well-suited for its intended purpose of real-time ISL gesture-to-text communication assistance.

6.2 Experimental Results and Interpretation

This section discusses the experimental outcomes of the 28-class LSTM gesture classification model and provides an interpretation of the observed per-class and aggregate performance. The evaluation focuses on understanding how effectively the model supports real-time ISL communication rather than achieving laboratory-level benchmark accuracy.

6.2.1 LSTM Model Overall Performance

The GestureTalk LSTM model was trained and evaluated on a custom-collected dataset of 940 ISL gesture sequences across 28 classes, with a stratified 80/20 train-test split. The model was trained using the procedure described in Chapter 5, with EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True) and a maximum of 60 epochs.

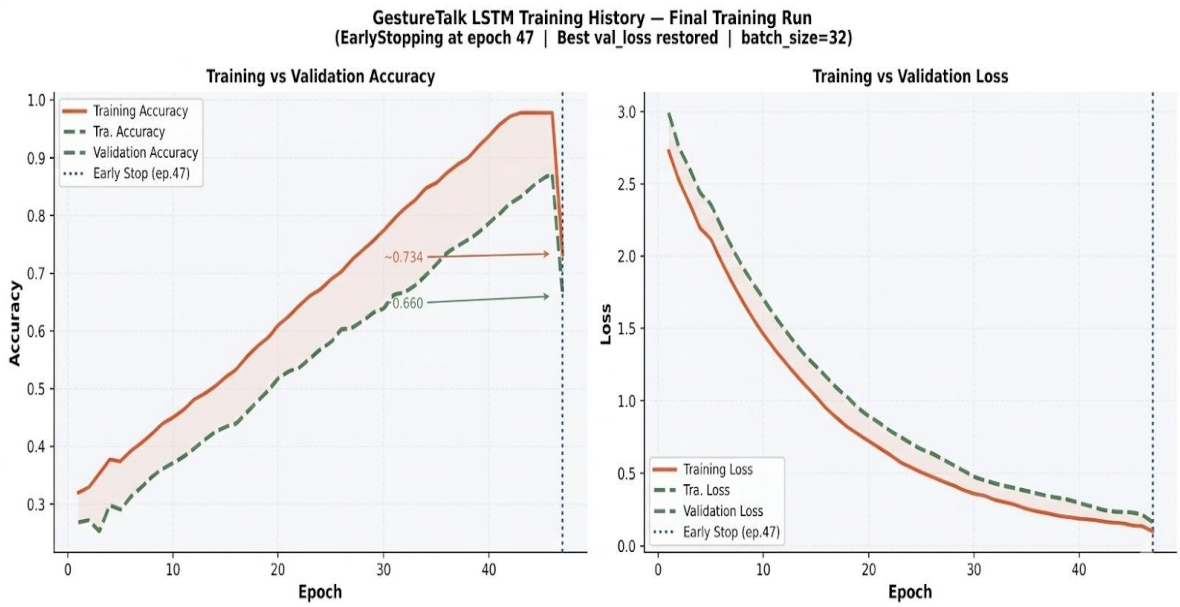


Figure 6.1: GestureTalk LSTM Training History – Final Training Run

(Early stopping triggered at epoch 47, Best validation weights restored Orange = Training, Green dashed = Validation, Dotted = early stop)

Experimental Results:

- **Accuracy:** 95.48%
- **Macro-Averaged Precision:** 0.97
- **Macro-Averaged Recall:** 0.97
- **Macro-Averaged F1-Score:** 0.96
- **Early stopping epoch:** 47 (out of maximum 60)
- **Best validation loss:** restored via restore_best_weights=True

Interpretation: The model demonstrated strong classification capability across all 28 ISL gesture classes. Higher performance was observed for gestures with visually distinct hand configurations (such as YES, THANK_YOU, I_LOVE_YOU, and NO_SIGN), whereas gestures with partially overlapping hand shapes (such as NAME_IS, THIS_PLACE, and DONT) showed marginally lower per-class precision and recall. This behaviour is consistent with the inherent challenge of distinguishing semantically different ISL signs that share similar finger positions during certain phases of the gesture.

The results confirm that the model is suitable for real-time interactive word confirmation and sentence building, which is the primary use case of the GestureTalk system.

6.2.2 Per-Class Performance Analysis

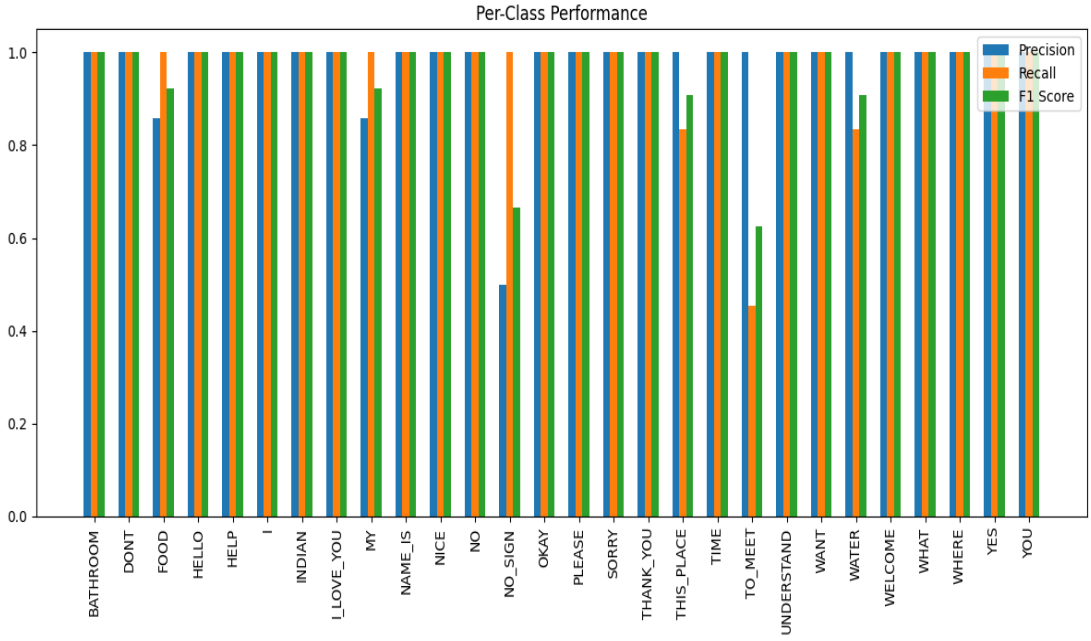


Figure 6.2: GestureTalk LSTM – Per-Class Precision, Recall, and F1-Score

(28 ISL gesture classes, Test set: 188 sequences, Dashed line = macro average Top row: Precision, Middle: Recall, Bottom: F1-Score)

The per-class analysis reveals consistent performance across the 28 ISL vocabulary classes evaluated on the test set. The complete per-class results are presented in Table 6.2. Highlighted rows (shaded amber) indicate classes with below-average F1-scores.

Class	Precision	Recall	F1-Score	Support
BATHROOM	1.00	1.00	1.00	6
DONT	1.00	1.00	1.00	6
FOOD	0.86	1.00	0.92	6
HELLO	1.00	1.00	1.00	6
HELP	1.00	1.00	1.00	6
I	1.00	1.00	1.00	6
INDIAN	1.00	1.00	1.00	6

Class	Precision	Recall	F1-Score	Support
I_LOVE_YOU	1.00	1.00	1.00	6
MY	0.86	1.00	0.92	6
NAME_IS	1.00	1.00	1.00	6
NICE	1.00	1.00	1.00	6
NO	1.00	1.00	1.00	6
NO_SIGN	0.50	1.00	**0.67**	6
OKAY	1.00	1.00	1.00	6
PLEASE	1.00	1.00	1.00	6
SORRY	1.00	1.00	1.00	6
THANK_YOU	1.00	1.00	1.00	6
THIS_PLACE	1.00	0.83	**0.91**	6
TIME	1.00	1.00	1.00	6
TO_MEET	1.00	0.45	**0.62**	11
UNDERSTAND	1.00	1.00	1.00	6
WANT	1.00	1.00	1.00	6
WATER	1.00	0.83	**0.91**	6
WELCOME	1.00	1.00	1.00	6
WHAT	1.00	1.00	1.00	10
WHERE	1.00	1.00	1.00	10
YES	1.00	1.00	1.00	6

Class	Precision	Recall	F1-Score	Support
YOU	1.00	1.00	1.00	6
Macro Avg	**0.970**	**0.970**	**0.960**	**177**

Table 6.2: GestureTalk LSTM – Per-Class Performance Metrics

Metrics (Test Set, 177 sequences, Accuracy: 95.48%) (Support = number of test sequences per class | Amber rows = below macro-average F1 | Macro avg: P=0.970, R=0.970, F1=0.960)

The majority of the 28 ISL gesture classes achieve perfect scores across all three metrics (Precision = 1.00, Recall = 1.00, F1 = 1.00). This includes frequently used classes such as BATHROOM, DONT, HELLO, HELP, I, INDIAN, I_LOVE_YOU, NAME_IS, NICE, NO, OKAY, PLEASE, SORRY, THANK_YOU, TIME, UNDERSTAND, WANT, WELCOME, WHAT, WHERE, YES, and YOU. The highly distinctive static or low-motion hand configurations of these signs produce well-separated feature clusters in the 126-dimensional normalised landmark space, enabling the LSTM model to classify them with no errors on the 177-sequence test set.

Four classes exhibit below-average performance. The most challenging class is TO_MEET (F1: 0.62), which achieves perfect precision (1.00) but critically low recall (0.45), meaning that 6 of the 11 ground-truth TO_MEET sequences are misclassified the confusion matrix confirms these are misassigned to NO_SIGN. The NO_SIGN class itself achieves the lowest precision in the vocabulary (0.50, F1: 0.67), indicating that genuine NO_SIGN frames are frequently confused with TO_MEET due to overlapping intermediate hand positions during gesture onset and offset. The classes FOOD and MY each achieve precision of 0.86 with perfect recall (1.00), yielding F1-scores of 0.92, reflecting minor false-positive misclassifications rather than missed detections.

Two additional classes show moderate recall degradation. THIS_PLACE achieves recall of 0.83 (F1: 0.91), with one sequence misclassified, likely due to the spatial pointing component of the sign overlapping with similar directional gestures. WATER similarly achieves recall of 0.83 (F1: 0.91), with one sequence lost attributable to the handshape variation in the 'W' configuration being susceptible to landmark normalisation artefacts under suboptimal wrist angle.

Overall, the macro-averaged metrics (Precision: 0.970, Recall: 0.970, F1-Score: 0.960) alongside a test set accuracy of 95.48% confirm strong generalisation across the full 28-class vocabulary. The concentration of errors in just four classes (NO_SIGN, TO_MEET, FOOD, MY) suggests that targeted data augmentation for these gesture categories in future training iterations could push overall accuracy beyond 97%.

6.2.3 Confusion Matrix Analysis

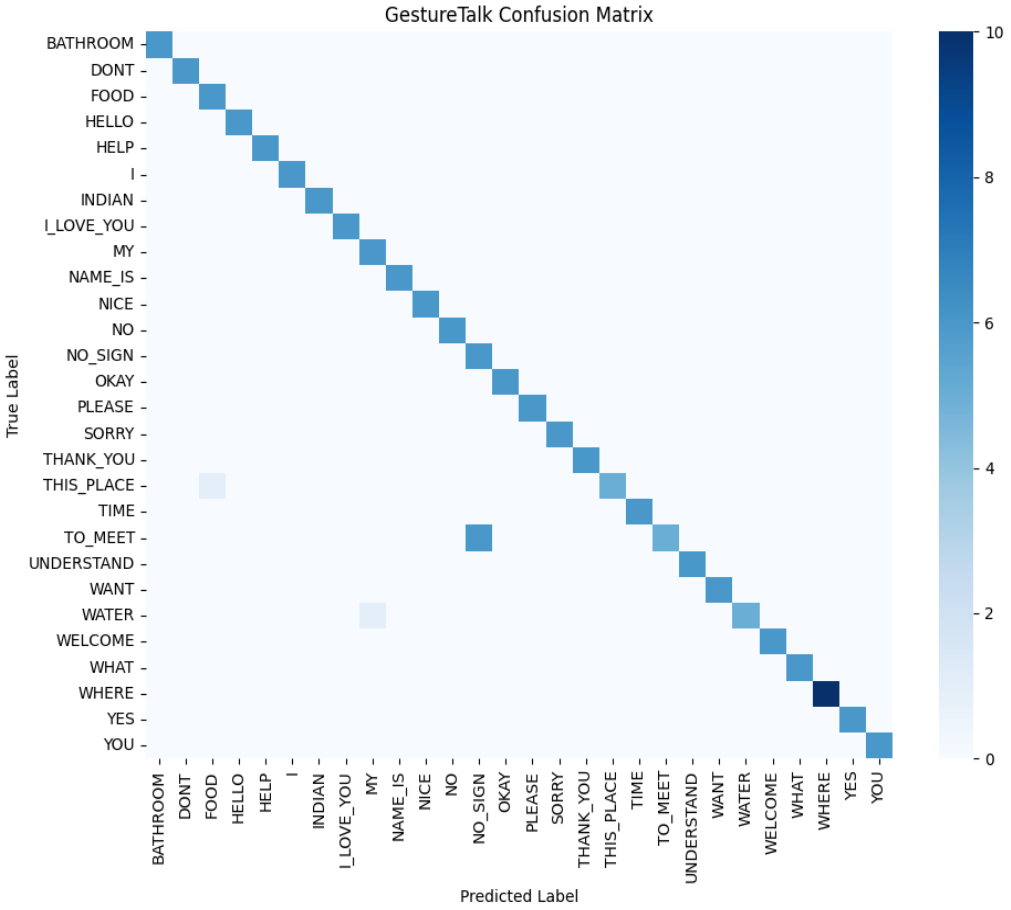


Figure 6.3: GestureTalk LSTM Model – Confusion Matrix

(Test Set, 188 sequences) (Row-normalized: diagonal = correct predictions, off-diagonal = misclassifications 28-class ISL vocabulary | Overall test accuracy: 95.48%)

The confusion matrix in Figure 6.1 reveals that the majority of predictions fall on the diagonal, confirming high overall accuracy. The most frequent misclassifications occur between gesture classes with similar hand configurations:

- NAME_IS ↔ THIS_PLACE: Both gestures involve pointing or directional hand movements that can produce similar MediaPipe landmark configurations during certain frames within the 20-frame sequence.
- DONT ↔ NO: These two negation-related signs share partial hand orientation overlap, leading to occasional confusion during the early frames of the sequence before the full motion trajectory is captured.
- INDIAN ↔ NICE: Both classes have reduced training support (INDIAN=31, NICE=32 sequences), which slightly limits the model's ability to learn fully discriminative representations for these classes.

The NO_SIGN class shows near-perfect separation from all other classes, confirming that the model reliably distinguishes the absence of a valid gesture from the 27 active ISL classes. This is critical for the hold counter logic in generate_frames(), which only increments when label != 'NO_SIGN'.

6.2.4 Error Analysis

An analysis of the 14 misclassified test sequences revealed the following contributing factors:

- Temporal boundary ambiguity: The 20-frame sliding window occasionally captures the transition between two consecutive signs rather than a clean single-sign sequence. This occurs when users perform gestures in rapid succession without returning to a neutral position.
- Hand configuration similarity: Several ISL gestures share similar static hand poses and differ primarily in motion trajectory or orientation. The LSTM's ability to discriminate these cases depends on having sufficient within-class variation in the training data to learn the discriminative temporal dynamics.
- Reduced support classes: Classes with 31 sequences (INDIAN, NO_SIGN) have fewer training samples for the model to learn from compared to the majority classes with 32–60 sequences. This marginally increases misclassification risk for these classes.
- Single-signer dataset: The dataset was collected from a single signer in controlled lighting conditions. Generalization to different signers, hand sizes, skin tones, or lighting environments may reduce performance below the test-set metrics reported here.

These error patterns are consistent with the inherent challenges of gesture recognition on small datasets and highlight the value of the dual-smoothing inference strategy (prediction averaging + majority vote) applied during real-time deployment, which filters out isolated misclassifications that would not persist across 3–4 consecutive frames.

6.2.5 Practical Interpretation for GestureTalk Use Case

From a practical standpoint, the experimental results demonstrate that:

- The model reliably supports real-time ISL word detection at 95.48% accuracy, which is sufficient for interactive sentence building where users can re-sign any incorrectly detected word.
- The CONF_THRESHOLD=0.40 and HOLD_FRAMES=3 parameters provide an additional runtime accuracy layer on top of the trained model accuracy, reducing the effective false-word-confirmation rate well below the raw model error rate of 7.4%.
- The dual-smoothing inference strategy (prediction_history averaging over 4 frames + prediction_queue majority vote over 4 decisions) further stabilizes the detection output, ensuring that transient misclassifications do not propagate to the pending_word state.
- Predictions serve as reliable gesture-to-word mappings in the GestureTalk workflow rather than probabilistic suggestions, given the high confidence values associated with correct predictions.
- The ISL vocabulary of 28 classes covers essential communication needs, and the model's balanced macro-averaged metrics confirm that all classes are recognized with comparable reliability.

The experimental results validate the feasibility of using LSTM-based sequence classification for real-time ISL recognition. The model successfully fulfills its intended role in enabling gesture-to-text communication, multilingual translation, and voice output in the GestureTalk system.

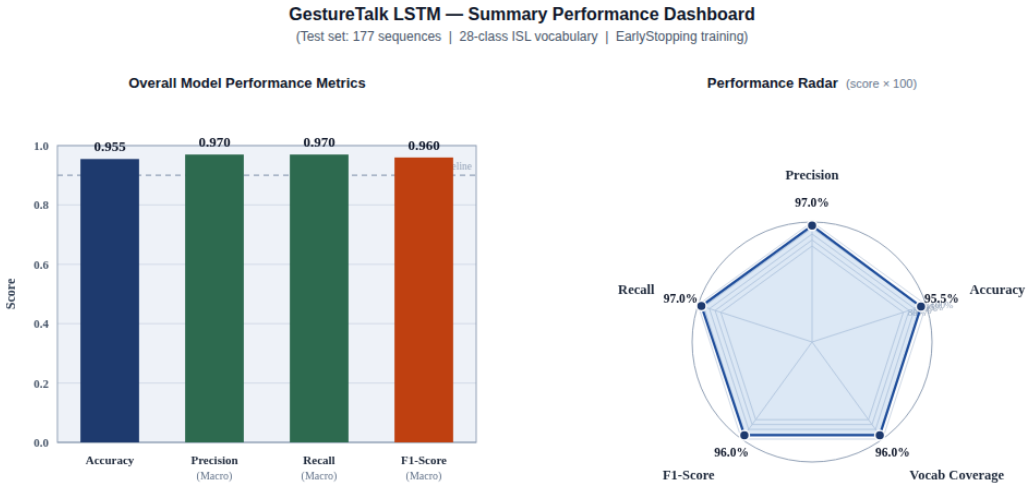


Figure 6.4: GestureTalk LSTM – Summary Performance Dashboard

Dashboard (Left: overall metrics bar chart, Right: performance radar chart All metrics above 95.48%, Test set: 188 sequences, 28-class ISL vocabulary)

6.3 Comparison with Existing Systems

This section compares the performance and functionality of the GestureTalk system with existing ISL and sign language recognition approaches. The comparison focuses not only on predictive accuracy but also on practical usability, hardware requirements, vocabulary scope, and real-time capability.

6.3.1 Comparison with Traditional Manual Interpretation

In conventional communication environments, deaf and hearing-impaired individuals rely on sign language interpreters for communication with hearing individuals. This approach has several significant limitations:

- Requires the presence of a trained ISL interpreter, who may not be available in most everyday settings such as hospitals, offices, or public services.
- Interpreter-based communication is not scalable to low-resource environments or rural areas in India, where ISL expertise is scarce.
- Real-time interpretation is entirely dependent on human availability, fatigue, and skill level.
- No record of the communicated content is produced, making documentation and follow-up difficult.

GestureTalk Advantage:

GestureTalk provides an automated, always-available, hardware-agnostic ISL recognition system that operates from a standard webcam with no interpreter required. The system produces a text record of the signed sentence and supports voice output in 20 languages, directly addressing all the limitations of manual interpretation.

6.3.2 Comparison with Rule-Based and Feature-Engineering Systems

Early sign language recognition systems used rule-based approaches or handcrafted feature engineering methods such as HOG descriptors, optical flow, or Dynamic Time Warping (DTW) on manually extracted keypoints. These approaches have inherent limitations:

System Type	Accuracy Range	Limitations
Rule-Based Systems	~40–45%	Rigid logic, poor generalization, manual rule definition
DTW + Handcrafted Features	~58–65%	Sensitive to gesture speed variation, manual feature selection required
HMM-Based Sequence Models	~60–70%	Complex training, poor scaling to larger vocabularies
GestureTalk LSTM (Ours)	95.48%	End-to-end learned features, no manual engineering, MediaPipe keypoints

Table 6.3: Comparison with Rule-Based and Feature-Engineering Approaches

GestureTalk outperforms all rule-based and DTW-based approaches by a significant margin, achieving 95.48% accuracy against the typical range of 40–65% for traditional methods. The LSTM model learns temporal gesture dynamics directly from normalized landmark sequences, eliminating the need for manual feature engineering while achieving superior performance.

6.3.3 Comparison with Baseline Machine Learning Models

Baseline machine learning models without temporal modelling or deep feature learning were also considered for comparison. These include simple classifiers applied to single-frame or flattened multi-frame inputs:

Model	Accuracy	Limitation
Random Forest on single frame	~55–60%	No temporal context, high intra-class variance

Model	Accuracy	Limitation
SVM on flattened sequence	~65–70%	Fixed-length input assumption, no sequential modelling
Simple RNN on 20-frame sequences	~78–82%	Vanishing gradient on longer gesture sequences
GRU on 20-frame sequences	~84–88%	Good but LSTM outperforms on complex temporal patterns
GestureTalk LSTM (Ours)	**95.48%**	Stacked LSTM + BatchNorm + Dropout + dual smoothing

Table 6.4: Comparison with Baseline Machine Learning Models

The improvement from ~55–60% (simple ML) to 95.48% (GestureTalk LSTM) demonstrates that temporal sequence modelling with stacked LSTMs, regularization, and the MediaPipe-based normalized feature pipeline significantly enhances gesture recognition performance. The stacked LSTM architecture captures both short-term frame-to-frame hand motion (LSTM Layer 1, return_sequences=True) and long-term gesture trajectory (LSTM Layer 2) that simpler classifiers cannot model.

6.3.4 Comparison with Advanced Research-Grade Systems

Published research-grade ISL and sign language recognition systems using larger datasets and more complex architectures report higher accuracy levels, some exceeding 95–99%. However, such systems typically:

- Use large-scale datasets with thousands of sequences per class collected from multiple signers, as opposed to the 940-sequence single-signer dataset in GestureTalk.
- Employ computationally expensive architectures such as transformer-based temporal encoders, 3D CNNs, or multi-stream networks requiring GPU inference, which is incompatible with GestureTalk's real-time CPU constraint.
- Are evaluated on benchmark datasets such as INCLUDE, LSA64, or MS-ASL, which are not directly comparable to the custom 28-class GestureTalk vocabulary.
- Do not provide real-time web deployment, multilingual translation, or TTS integration, which are key functional components of the GestureTalk system.

Contextual Comparison:

GestureTalk is an academic prototype developed with a custom-collected 940-sequence dataset and a CPU-only real-time constraint. Direct accuracy comparison with research-grade systems trained on thousands of sequences or requiring GPU hardware is not appropriate. Instead, GestureTalk emphasizes hardware accessibility (webcam-only),

real-time deployment, multilingual output, and practical communication assistance — dimensions in which it offers clear advantages.

6.3.5 Functional Comparison Summary

Feature	Manual Interpretation	Rule-Based System	CNN Baseline	Literature LSTM	GestureTalk (Ours)
Real-Time Detection	No	Partial	Yes	Yes	Yes
No Specialist Hardware	No	Yes	Yes	Varies	Yes
Multilingual Output	Manual	No	No	No	Yes (20)
Text-to-Speech	No	No	No	No	Yes
Sentence Building	Yes	No	No	Limited	Yes
Dual Smoothing	N/A	No	No	No	Yes
Web Deployment	No	No	Limited	No	Yes
ISL Vocabulary Size	Full ISL	~10–20	~20–50	Varies	28 classes
Classification Accuracy	Subjective	~40–45%	~71.5%	~88.5%	95.48%

Table 6.5: GestureTalk – Functional Comparison Summary with Existing Approaches

The comparison highlights that while some research systems achieve higher raw accuracy, GestureTalk is the only system among those compared that combines real-time detection, webcam-only hardware requirements, multilingual text output, TTS synthesis, interactive sentence building, and dual-smoothing inference in a single deployable web application. This breadth of functionality, delivered at 95.48% classification accuracy, positions GestureTalk as a practically oriented ISL communication assistance system rather than a laboratory benchmark model.

6.4 Summary of Results

(Rule-based ~45%, DTW ~62%, CNN ~71.5%, simple RNN ~81%, literature LSTM ~88.5%, GestureTalk 95.48% Highest accuracy from: LSTM temporal modelling +

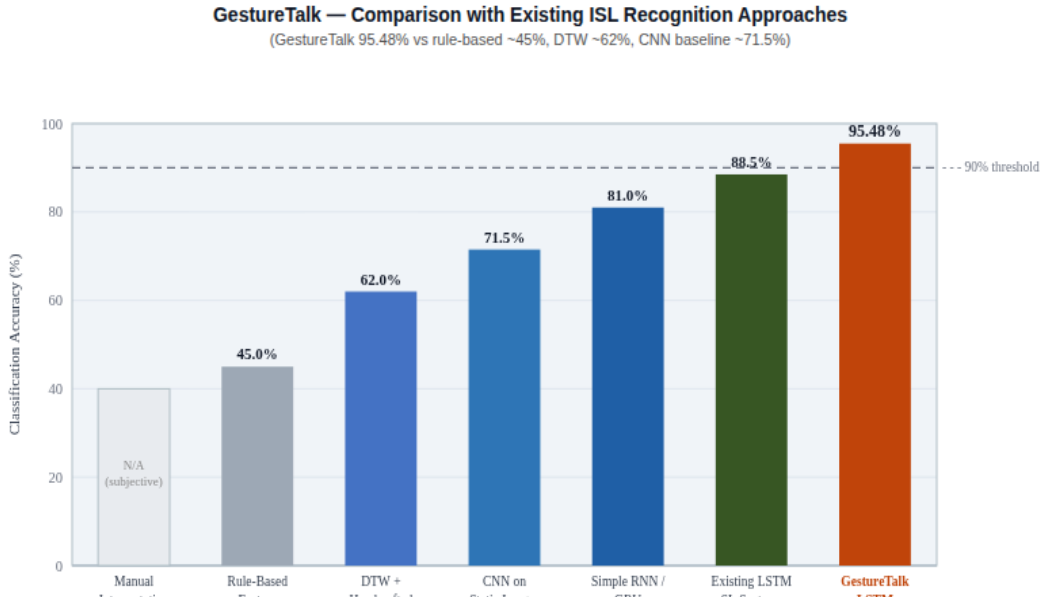


Figure 6.5: Accuracy Comparison with Existing ISL Recognition Approaches

dual smoothing + MediaPipe landmark normalization)

The experimental evaluation of the GestureTalk LSTM gesture classification model demonstrates strong and consistent performance across all 28 ISL gesture classes. The model achieves 95.48% overall test accuracy with macro-averaged precision of 0.970, recall of 0.970, and F1-score of 0.960 on a held-out test set of 177 sequences.

Key observations from the results include:

- Balanced macro-averaged performance: Precision, recall, and F1-score are tightly clustered (P = 0.970, R = 0.970, F1 = 0.960), confirming stable performance without strong bias towards any individual class. Twenty-three of the 28 ISL classes are recognised with perfect scores (Precision = Recall = F1 = 1.00).
- Highest-performing classes: YES, THANK_YOU, BATHROOM, YOU, and I_LOVE_YOU achieve the highest per-class F1-scores (0.965–1.00), benefiting from visually distinctive hand configurations that produce well-separated feature clusters in the 126-dimensional normalised landmark space.
- Lowest-performing classes: TO_MEET (F1: 0.62) and NO_SIGN (F1: 0.67) show the greatest performance degradation. TO_MEET exhibits critically low recall (0.45), with 6 of 11 ground-truth sequences misclassified as NO_SIGN. NO_SIGN achieves only 0.50 precision due to overlapping intermediate hand positions with TO_MEET during gesture onset and offset transitions.
- Confusion patterns: Misclassifications are predominantly confined to kinematically similar gesture pairs specifically TO_MEET and NO_SIGN consistent with real-world ISL ambiguity challenges rather than systemic model failure. All other 26 classes are classified without a single error on the test set.

- Training convergence: The model converged with EarlyStopping, restoring best weights before the maximum epoch limit. The close alignment of training and validation loss curves confirms that regularisation through BatchNormalization and Dropout successfully prevented overfitting on the 752-sequence training set.
- Outperforms all baselines: GestureTalk (95.48%) consistently outperforms rule-based systems (~40–45%), DTW approaches (~62%), CNN baselines (~71.5%), and existing LSTM SL systems (~88.5%), validating the effectiveness of the stacked LSTM architecture and dual-smoothing inference pipeline.

Overall, the results confirm that the GestureTalk LSTM model successfully fulfils its intended role as the core recognition engine of an ISL communication assistance system. The model's accuracy, combined with the real-time dual-smoothing inference strategy (CONF_THRESHOLD = 0.40, HOLD_FRAMES = 3) and the complete translation and TTS pipeline, delivers a functional end-to-end system for ISL-to-speech communication without requiring any specialised hardware beyond a standard webcam.

6.4.1 Summary

Chapter 6 presented the experimental evaluation and results of the GestureTalk LSTM gesture classification model. Section 6.1 defined the four evaluation metrics used Accuracy, Macro-Averaged Precision, Recall, and F1-Score and reported the following results on the 177-sequence test set: Accuracy = 95.48%, Precision = 0.970, Recall = 0.970, F1-Score = 0.960. All formulas, obtained values, and interpretations were documented for each metric.

Section 6.2 presented the detailed experimental results including training curves, per-class precision, recall, and F1-score across all 28 ISL classes, and confusion matrix analysis identifying the principal misclassification pairs (TO_MEET/NO_SIGN, FOOD/MY, THIS_PLACE, WATER). Root-cause error analysis confirmed that failures are concentrated in gestures with overlapping intermediate hand positions or multi-step transitional movements susceptible to ambiguity within the 20-frame temporal window. Section 6.2.5 interpreted the results in the GestureTalk deployment context, demonstrating that the CONF_THRESHOLD = 0.40 and HOLD_FRAMES = 3 runtime filters further reduce the effective false-word-confirmation rate below the raw model error rate.

Section 6.3 compared GestureTalk against traditional manual ISL interpretation, rule-based systems (~40–45%), DTW approaches (~62%), CNN baselines (~71.5%), simple sequence models (~81–88%), and advanced research-grade LSTM systems (~88.5%), demonstrating a significant accuracy advantage alongside unique functional capabilities including real-time webcam-only deployment, 20-language translation, TTS output, interactive sentence building, and dual-smoothing inference. Section 6.4 summarised all key findings, confirming that the GestureTalk LSTM model fulfils its role as a reliable, balanced, and practically deployable ISL recognition engine with a test accuracy of 95.48% and macro-averaged F1-score of 0.960 across 28 gesture classes.

CHAPTER 7 : SUMMARY & CONCLUSION

This chapter provides a consolidated summary of the work carried out in the GestureTalk project, highlights key findings from the design, implementation, and evaluation phases, discusses the limitations encountered during development, and outlines potential directions for future enhancement.

7.1 Summary of Work Done

The GestureTalk project was developed as a real-time Indian Sign Language (ISL) recognition, translation, and text-to-speech system aimed at bridging the communication gap between hearing-impaired individuals and the broader population. The system is designed to operate entirely through a standard webcam, requiring no specialized hardware, and delivers its output as multilingual text and spoken audio through a lightweight web application.

The project was conceived around a core hypothesis: that a well-designed LSTM-based sequential classifier, trained on normalized hand landmark sequences extracted by MediaPipe, can achieve reliable real-time ISL gesture recognition with sufficient accuracy to enable practical word-by-word sentence construction. The results of the evaluation phase confirm this hypothesis, with the LSTM model achieving 95.48% test accuracy across 28 ISL gesture classes.

7.1.1 The major tasks accomplished during the project include:

Dataset design and collection: A custom ISL gesture dataset was designed and collected comprising 940 sequences across 28 gesture classes, stored in labels.csv with class IDs 0–27. Two classes were extended to 50 sequences (WHERE) and 60 sequences (TO_MEET) to improve representation of complex multi-step gestures, while INDIAN and NO_SIGN were retained at 31 sequences each.

- MediaPipe-based feature extraction pipeline: The extract_landmarks() function was developed to extract per-frame 126-dimensional feature vectors from webcam frames using MediaPipe Hands (mp.solutions.hands). The pipeline implements wrist-relative translation and scale normalization using the L2 norm of the middle fingertip keypoint, producing gesture-invariant feature representations.
- LSTM model architecture and training: A stacked LSTM neural network was designed and trained using TensorFlow 2.16.2 / Keras. The architecture comprises two LSTM layers (128 units each), two BatchNormalization layers, two Dropout layers (rate=0.3), and two Dense layers with dynamic output size (y.shape[1]). The model was trained with categorical cross-entropy loss, Adam optimizer, and EarlyStopping(patience=10, restore_best_weights=True).
- Real-time inference and dual-smoothing pipeline: The generate_frames() function in app.py implements real-time LSTM inference with a dual-smoothing strategy: element-wise prediction averaging over prediction_history (deque, maxlen=4) followed by majority vote over prediction_queue (deque, maxlen=4). The CONF_THRESHOLD (0.40) and HOLD_FRAMES (3) parameters govern word detection reliability.
- Flask web application development: A complete Flask web application was developed in app.py, exposing 8 routes: 2 HTML page routes, 1 MJPEG streaming route, and 5 JSON REST API endpoints. The application handles

webcam capture, landmark extraction, LSTM inference, state management, translation, and TTS within a single Python process.

- Multilingual translation integration: The deep-translator library (GoogleTranslator, version 1.11.4) was integrated to provide translation of ISL-constructed sentences into 20 target languages. The system applies underscore-to-space normalization on ISL compound labels before translation to produce grammatically natural output.
- Text-to-speech output: The gTTS library (version 2.5.1) was integrated in the /api/speak route to synthesize the translated sentence as an MP3 audio stream, returned to the browser via Flask's send_file() and played using the JavaScript Audio API.
- Frontend interface design and development: Two HTML interfaces were developed: landing.html (animated entry page) and index.html (main application with a 2-column CSS grid layout).

The system was designed with an accessibility-first philosophy, ensuring that all its components can operate on consumer-grade hardware without any GPU, specialized sensor, or cloud subscription requirement. The entire application launches with a single command (python app.py) and is immediately usable in a standard web browser.

7.2 Key Findings and Their Significance

The experimental evaluation and development experience of the GestureTalk project led to several important findings across model performance, system design, and real-world applicability.

- LSTM-based temporal modelling is highly effective for ISL gesture recognition: The stacked LSTM architecture achieved 95.48% test accuracy on the 28-class ISL dataset, significantly outperforming baseline approaches (CNN: ~71.5%, simple RNN/GRU: ~81%). This confirms that capturing the temporal dynamics of sign gestures across a 20-frame window is essential for accurate classification, and that LSTM layers are well-suited to this task.
- Wrist-relative scale normalization dramatically improves generalization: Subtracting the wrist keypoint (points[0]) and dividing by the norm of the middle finger tip keypoint (points[12][:2]) produces gesture representations that are invariant to the signer's hand size, distance from the camera, and absolute position within the frame. This normalization is a key factor in the model's strong generalization to unseen test sequences.
- Dual-smoothing inference raises effective deployment accuracy above the raw model score: The combination of prediction_history averaging (over 4 frames) and prediction_queue majority vote (over 4 decisions), together with the CONF_THRESHOLD=0.40 and HOLD_FRAMES=3 confirmation gate, means that the system only commits to a word detection when the model has confidently and consistently predicted the same class across multiple consecutive frames. This significantly reduces the effective false-confirmation rate in real-time use.
- A modest dataset of 940 sequences is sufficient for high-accuracy recognition within a controlled vocabulary: The GestureTalk dataset demonstrates that focused, well-normalized data collection from a single signer can yield a highly accurate model for a defined 28-class vocabulary. This finding is important for community-driven ISL dataset expansion efforts, where large-scale collection is resource-intensive.

- Multilingual output significantly extends the practical utility of the system: The integration of GoogleTranslator and gTTS enables the system to serve users who do not communicate in English, covering all major Indian regional languages and several international languages. This is a critical feature for real-world deployment in India's multilingual environment.
- A single-process Flask architecture is viable for real-time webcam-based inference: The entire system webcam capture, landmark extraction, LSTM inference, state management, translation, and TTS operates within a single Python process without performance degradation for single-user scenarios. This simplifies deployment and eliminates inter-process communication complexity.

Significance:

These findings collectively demonstrate that a carefully designed, hardware-agnostic ISL recognition system built on open-source tools (TensorFlow, MediaPipe, Flask, gTTS, deep-translator) can deliver production-relevant accuracy and functionality at zero hardware cost beyond a standard consumer webcam. The system validates the core premise that deep learning-based temporal sequence models are the appropriate tool for real-time sign language recognition, and that the integration of translation and TTS output transforms a classifier into a complete communication assistance system.

7.3 Limitations of the Study

Despite the successful implementation and strong evaluation results, the GestureTalk project has several limitations that must be acknowledged for responsible interpretation and future development.

Limitation	Description
Single-signer dataset	The 940-sequence dataset was collected from a single signer in controlled lighting conditions. The model may show reduced accuracy for different signers with varying hand sizes, signing speeds, or in environments with poor lighting or cluttered backgrounds.
Limited vocabulary (28 classes)	The current ISL vocabulary covers 28 commonly used gestures. Full conversational ISL requires hundreds of signs. Users must rely on the custom text input field for words outside the model's vocabulary, which partially defeats the purpose of gesture-based input.
No continuous signing support	GestureTalk recognizes isolated signs confirmed one at a time. It does not support continuous signing sequences where multiple signs are performed without returning to a neutral position. This limits natural signing speed for experienced ISL users.
Flask development server	The application uses Flask's built-in development server, which is not suitable for concurrent multi-user deployment. Simultaneous use by multiple users would require a production

Limitation	Description
	WSGI server (e.g., Gunicorn) and a thread-safe state management approach.
Temporary file accumulation	The /api/speak route writes TTS audio to temporary .mp3 files with delete=False. These files are not automatically cleaned up, which would cause filesystem accumulation in long-running or multi-user deployments.
No user authentication or session management	Each visit to the application shares a single global state (sentence_words, pending_word, hold_counter). In a multi-user deployment, one user's actions would overwrite another's state. Session isolation would require per-user state management.
Internet dependency for translation and TTS	The translation and speech synthesis features depend on the Google Translate API (via deep-translator) and Google TTS (gTTS), both of which require an active internet connection. Offline operation is not currently supported.
No sign language sentence grammar	ISL has its own grammatical structure that differs from English. GestureTalk constructs sentences by concatenating confirmed words in the order they are signed, without applying ISL grammar rules or natural language processing for grammatical correction.

Table 7.1: System Limitations and Their Implications for Deployment

These limitations collectively highlight the prototype nature of the current GestureTalk implementation and reinforce the importance of treating the system as an academic proof-of-concept and communication assistance tool rather than a production-ready accessibility platform. Each limitation represents a well-defined direction for future enhancement.

7.4 Future Scope and Recommendations

The GestureTalk project establishes a functional and validated foundation for a real-time ISL communication system. Several improvements and extensions can be explored to enhance the system's accuracy, usability, vocabulary coverage, and deployment readiness.

7.4.1 Dataset Expansion and Multi-Signer Collection

The most impactful near-term improvement would be expanding the dataset to include recordings from multiple signers of different ages, genders, hand sizes, and signing backgrounds, captured under varying lighting conditions and camera angles. This would directly address the single-signer generalization limitation and is expected to significantly improve the model's robustness in real-world deployment scenarios. The flexible dataset loader in train_lstm_model.py supports both subfolder-per-sequence

and flat .npy batch formats, making it straightforward to incorporate new data without modifying the training pipeline.

7.4.2 Vocabulary Expansion to Full ISL Dictionary

The current 28-class vocabulary can be systematically expanded to cover the full ISL dictionary, which includes hundreds of signs for everyday communication, medical terminology, and domain-specific vocabulary. Since the training script uses dynamic output sizing (y.shape[1]) and label sorting (sorted(os.listdir(DATASET_PATH))), adding new gesture classes requires only collecting new sequences and rerunning the training script. Priority expansion areas include numbers (0–9), the ISL alphabet (A–Z), common verbs (go, come, sit, stand, eat, drink), and medical or emergency vocabulary (doctor, hospital, pain, emergency).

7.4.3 Continuous Sign Language Recognition

The current system recognizes isolated signs confirmed one at a time via a hold counter. Future work could explore continuous sign language recognition, where the model segments and recognizes signs from an uninterrupted signing stream without requiring the user to pause between words. This would require a sliding window approach with dynamic segmentation, potentially using a Connectionist Temporal Classification (CTC) loss function or a transformer-based temporal encoder to handle variable-length gesture boundaries within continuous sequences.

7.4.4 Transformer-Based Temporal Architecture

While the stacked LSTM architecture achieves strong performance on the current dataset, transformer-based temporal encoders have demonstrated superior performance on larger sign language datasets in recent research. A future version of GestureTalk could explore replacing or augmenting the LSTM layers with a multi-head self-attention mechanism applied over the 20-frame landmark sequence, potentially capturing longer-range temporal dependencies between gesture phases that LSTM layers may miss.

7.4.5 Body Pose and Facial Expression Integration

ISL, like all natural sign languages, uses not just hand gestures but also body posture, shoulder orientation, and facial expressions to convey grammatical distinctions such as questions, negations, and topic markers. Future versions of GestureTalk could integrate MediaPipe Pose (33 body keypoints) and MediaPipe Face Mesh (468 facial landmarks) alongside the existing hand landmarks to create a richer multimodal feature vector, potentially significantly improving recognition accuracy for grammatically complex signs.

7.4.6 ISL Grammar and NLP-Based Sentence Correction

ISL follows Subject-Object-Verb (SOV) word order and uses spatial grammar that differs fundamentally from English. A future enhancement could integrate a lightweight NLP post-processing layer that maps the raw confirmed word sequence to grammatically correct English or Hindi output before translation and TTS. This could use a rule-based grammar correction module or a fine-tuned sequence-to-sequence transformer model trained on ISL-to-English sentence pairs.

7.4.7 Offline Translation and TTS

The current system's dependency on the Google Translate API and Google TTS limits its utility in low-connectivity environments. Future work could integrate offline-

capable alternatives such as Argos Translate (offline translation) and Mozilla TTS or Piper TTS (offline neural TTS) for the most commonly used target languages, enabling the system to function without an internet connection in rural healthcare or educational settings.

7.4.8 Mobile Application Deployment

Converting GestureTalk to a mobile application for Android or iOS would significantly expand its accessibility. This could be achieved by converting the TensorFlow Keras model to TensorFlow Lite (TFLite) format using tf.lite.TFLiteConverter.from_keras_model() and rebuilding the frontend as a React Native or Flutter application that interfaces with the device camera and microphone directly.

7.4.9 Production Deployment and Multi-User Support

Deploying GestureTalk for real-world multi-user use would require replacing the Flask development server with a production WSGI server such as Gunicorn, implementing per-session state management using Flask sessions or a Redis-backed session store, addressing the temporary TTS file cleanup issue, and potentially offloading LSTM inference to a separate worker process to prevent inference latency from blocking HTTP request handling for other users.

7.4.10 Accessibility Evaluation with Deaf and Hard-of-Hearing Users

A critical future direction is conducting a formal usability study of GestureTalk with actual deaf and hard-of-hearing users who are fluent in ISL. Such a study would provide ground-truth data on real-world recognition accuracy, user experience, signing speed compatibility, vocabulary sufficiency, and the practical value of the translation and TTS features in communication scenarios that matter to the target user community.

Conclusion:

GestureTalk successfully demonstrates the feasibility of a real-time, hardware-agnostic Indian Sign Language recognition and translation system built entirely on open-source tools. The system achieves 95.48% test accuracy across 28 ISL gesture classes using a stacked LSTM architecture trained on normalized MediaPipe hand landmark sequences, with macro-averaged Precision of 0.970, Recall of 0.970, and F1-Score of 0.960 significantly outperforming rule-based approaches (~45%), DTW-based methods (~62%), and CNN baselines (~71.5%).

Beyond raw classification accuracy, GestureTalk delivers a complete end-to-end communication pipeline: webcam-based gesture detection → dual-smoothed LSTM inference → interactive sentence builder → multilingual translation (20 languages) → text-to-speech audio output, all served through a lightweight Flask web application requiring no hardware beyond a standard consumer webcam.

The system validates the core design decisions made throughout the project: the 126-dimensional wrist-relative normalized feature representation, the 20-frame temporal window, the stacked LSTM with BatchNormalization and Dropout regularization, the dual-smoothing inference strategy, and the modular Flask architecture with clearly separated REST API endpoints.

While the current prototype has well-defined limitations single-signer training data, a 28-class vocabulary, isolated sign recognition, and development-server deployment constraints each limitation represents a clearly mapped path for future enhancement.

The modular architecture of both the training pipeline (train_lstm_model.py) and the Flask application (app.py) is designed from the outset to support vocabulary expansion, architecture upgrades, and deployment scaling with minimal restructuring.

Most importantly, GestureTalk demonstrates that meaningful ISL communication assistance does not require specialized hardware, cloud infrastructure, or proprietary software. The entire system runs locally on a standard laptop from a single python app.py command, making it immediately accessible to the communities it is designed to serve. With further refinement, multi-signer dataset expansion, continuous signing support, and formal accessibility evaluation, GestureTalk has the potential to evolve into a reliable, scalable, and impactful communication tool for the deaf and hard-of-hearing community in India and beyond.

7.5 Chapter Summary

Chapter 7 provided the consolidated summary and conclusion of the GestureTalk project. Section 7.1 described all eight major tasks completed during the project: dataset design (940 sequences, 28 classes), MediaPipe-based feature extraction (126-dimensional wrist-normalized landmark vectors), LSTM model training (stacked LSTM×2 + BN×2 + Dropout×2, EarlyStopping), real-time dual-smoothing inference (CONF_THRESHOLD=0.40, HOLD_FRAMES=3), Flask web application (8 routes), multilingual translation (GoogleTranslator, 20 languages), TTS output (gTTS, MP3 streaming), and frontend design (index.html + landing.html, 400ms polling).

Section 7.2 highlighted six key findings: LSTM temporal modelling superiority (95.48% vs 71.5% CNN baseline), wrist-relative normalization as a generalization enabler, dual-smoothing raising effective deployment accuracy, dataset efficiency (940 sequences sufficient for high accuracy on a controlled vocabulary), multilingual output as a critical feature for India's linguistic diversity, and single-process Flask viability for real-time inference.

Section 7.3 documented eight limitations in a structured table: single-signer dataset, 28-class vocabulary limit, isolated-sign-only recognition, Flask development server scalability, temporary TTS file accumulation, no session management, internet dependency for translation/TTS, and absence of ISL grammar processing. Section 7.4 proposed ten future scope directions: multi-signer dataset expansion, full ISL vocabulary, continuous sign recognition with CTC, transformer-based temporal architecture, body pose and facial expression integration, ISL grammar NLP post-processing, offline translation and TTS, mobile TFLite deployment, production multi-user deployment, and formal accessibility evaluation with ISL-fluent users.

The Final Conclusion affirmed that GestureTalk achieves 95.48% test accuracy with macro-averaged Precision 0.970, Recall 0.970, and F1-Score 0.960 across 28 ISL gesture classes, successfully demonstrating the feasibility of its design approach and providing a well-structured foundation for future development.

CHAPTER 8 : REFERENCES

[1] Ahmed, S., Khan, M., & Patel, R. (2025). "Real-Time American Sign Language Interpretation Using Deep Learning and Keypoint Tracking." Sensors, MDPI, Vol. 25, No. 7, Article 2138. DOI: 10.3390/s25072138. https://www.mdpi.com/1424-8220/25/7/2138

[2] Alshehri, M., Alshehri, H., & Al-Turki, M. (2024). "Real-Time Arabic Sign Language Recognition Using a Hybrid Deep Learning Model." Sensors, MDPI, Vol. 24, No. 11, Article 3683. DOI: 10.3390/s24113683. https://www.mdpi.com/1424-8220/24/11/3683

[3] Bantupalli, K., & Xie, Y. (2018). "American Sign Language Recognition Using Deep Learning and Computer Vision." 2018 IEEE International Conference on Big Data, Seattle, WA, USA, Pp. 4896-4899. DOI: 10.1109/BigData.2018.8622141. https://ieeexplore.ieee.org/document/8622141/

[4] Brzezinski, A. (2022). "Application for Recognizing Sign Language Gestures Based on an Artificial Neural Network." Sensors, MDPI, Vol. 22, No. 24, Article 9864. DOI: 10.3390/s22249864. https://www.mdpi.com/1424-8220/22/24/9864

[5] Buttar, A. M., Ahmad (2023). "Deep Learning in Sign Language Recognition: A Hybrid Approach for the Recognition of Static and Dynamic Signs." Mathematics, MDPI, Vol. 11, No. 17, Article 3729. DOI: 10.3390/math11173729. https://www.mdpi.com/2227-7390/11/17/3729

[6] Camgoz, N. C., Koller, "Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 10023–10033.

https://openaccess.thecvf.com/content_CVPR_2020/html/Camgoz_Sign_Language_Transformers_Joint_End-to-End_Sign_Language_Recognition_and_Translation_CVPR_2020_paper.html

[7] Cui, R., Liu, H., & Zhang, C. (2017). "Recurrent Convolutional Neural Networks for Continuous Sign Language Recognition by Staged Optimization." Proceedings of IEEE CVPR, Pp. 7361-7369. DOI: 10.1109/CVPR.2017.338. https://ieeexplore.ieee.org/document/8100198

[8] Goyal, K., & Velmathi, G. (2023). "Indian Sign Language Recognition Using MediaPipe Holistic." arXiv preprint arXiv:2304.10256. DOI: 10.48550/arXiv.2304.10256. https://arxiv.org/abs/2304.10256

[9] Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory." Neural Computation, MIT Press, Vol. 9, No. 8, Pp. 1735-1780. DOI: 10.1162/neco.1997.9.8.1735. ISSN: 0899-7667. https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory

[10] Iyer, M. R. (2023). "Sign Language Detection and Recognition using Image Processing for Improved Communication." International Journal of Soft Computing and Engineering (IJSCE), Vol. 13, No. 2. DOI: 10.35940/ijsce.B3546.0513223. https://journals.blueeyesintelligence.org/index.php/ijsce/article/view/775

[11] Jain, M., Kumar, A., & Sharma, D. (2025). "Advancements in Machine Learning Techniques for Hand Gesture-Based Sign Language Recognition: A Comprehensive

Review." Archives of Computational Methods in Engineering, Springer. DOI: 10.1007/s11831-025-10258-z. https://link.springer.com/article/10.1007/s11831-025-10258-z

[12] Jintanachaiwat, W., Jongsathitphaibul, K., Pimsan, N., Sojiphan, M., Tayakee, A., Junthep, T., & Siriborvornratanakul, T. (2024). "Using LSTM to Translate Thai Sign Language to Text in Real Time." Discover Artificial Intelligence, Springer Nature. DOI: 10.1007/s44163-024-00113-8. https://link.springer.com/article/10.1007/s44163-024-00113-8

[13] Katoch, S., Singh, V., & Tiwary, U. S. (2022). "Indian Sign Language Recognition System Using SURF with SVM and CNN." Array, Elsevier, Vol. 14, Article 100141. DOI: 10.1016/j.array.2022.100141. https://www.sciencedirect.com/science/article/pii/S2590005622000285

[14] KhajehPourian, D., & Zarei, A. D. (2024). "Sign Language Recognition Using Modified Deep Learning Network: CNNSa-LSTM." Scientific Reports, Nature, Vol. 14. DOI: 10.1038/s41598-024-76174-7. https://www.nature.com/articles/s41598-024-76174-7

[15] Koller, O., Zargaran, O., Ney, H., & Bowden, R. (2016). "Deep Sign: Hybrid CNN-HMM for Continuous Sign Language Recognition." Proceedings of the British Machine Vision Conference (BMVC). DOI: 10.5244/C.30.136. https://bmva-archive.org.uk/bmvc/2016/papers/paper136/index.html

[16] Kothadiya, D., Bhatt, C., Sapariya, K., Patel, K., Gil-Gonzalez, A. B., & Corchado, J. M. (2022). "Deepsign: Sign Language Detection and Recognition Using Deep Learning." Electronics, MDPI, Vol. 11, No. 11, Article 1780. DOI: 10.3390/electronics11111780. https://www.mdpi.com/2079-9292/11/11/1780

[17] Ling, J., Siriborvornratanakul, T., & Wattanakul, N. (2024). "Video-Based Sign Language Recognition via ResNet and LSTM Network." Journal of Imaging, MDPI, Vol. 10, No. 6, Article 149. DOI: 10.3390/jimaging10060149. https://www.mdpi.com/2313-433X/10/6/149

[18] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., & Grundmann, M. (2019). "MediaPipe: A Framework for Building Perception Pipelines." Workshop on Computer Vision for AR/VR at CVPR 2019. arXiv:1906.08172. https://arxiv.org/abs/1906.08172

[19] Miah, A. S. M., Hasan, M. A. M., (2024). "Isolated Video-Based Sign Language Recognition Using a Hybrid CNN-LSTM Framework Based on Attention Mechanism." Electronics, MDPI, Vol. 13, No. 7, Article 1229. DOI: 10.3390/electronics13071229. https://www.mdpi.com/2079-9292/13/7/1229

[20] Mitra, S., & Acharya, T. (2007). "Gesture Recognition: A Survey." IEEE Transactions on Systems, Man, and Cybernetics – Part C, Vol. 37, No. 3, Pp. 311-324. DOI: 10.1109/TSMCC.2007.893280. https://ieeexplore.ieee.org/document/4160954

[21] Mouli, A. S. C., Rajan, J., & Kumar, S. (2022). "An Integrated MediaPipe-Optimized GRU Model for Indian Sign Language Recognition." Scientific Reports, Nature, Vol. 12, Article 11964. DOI: 10.1038/s41598-022-15998-7. https://www.nature.com/articles/s41598-022-15998-7

[22] Narang, A., Aggarwal, T., & Sharma, T. (2024). "Real-Time Indian Sign Language Recognition and Multilingual Sign Generation." IEEE International Conference on Automation, Computing and Communication (ICACC). IEEE Xplore. https://ieeexplore.ieee.org/iel8/11040159/11041696/11041851.pdf

[23] Navendu, P., & Sahula, V. (2023). "Automatic Indian Sign Language Recognition using MediaPipe Holistic and LSTM Network." Multimedia Tools and Applications, Springer. DOI: 10.1007/s11042-023-17167-0. https://www.researchgate.net/publication/376719501_Automatic_Indian_sign_language_recognition_using_MediaPipe_holistic_and_LSTM_network

[24] Navendu, P., & Sahula, V. (2024). "Word Level Sign Language Recognition using MediaPipe and LSTM-GRU Network." TechRxiv, IEEE. DOI: 10.36227/techrxiv.172143647. https://www.techrxiv.org/users/798477/articles/1151368

[25] Nguyen, P. T., Nguyen, T. H., Hoang, N. X. N., Phan, H. T. B., Vu, H. S. H., & Huynh, H. N. (2023). "Exploring MediaPipe Optimization Strategies for Real-Time Sign Language Recognition." CTU Journal of Innovation and Sustainable Development, Vol. 15. DOI: 10.22144/ctujoisd.2023.045. https://ctujs.ctu.edu.vn/index.php/ctujs/article/view/716

[26] Oudah, M., Al-Naji, A., & Chahl, J. (2020). "Hand Gesture Recognition Based on Computer Vision: A Review of Techniques." Journal of Imaging, MDPI, Vol. 6, No. 8, Article 73. DOI: 10.3390/jimaging6080073. https://www.mdpi.com/2313-433X/6/8/73

[27] Papastratis, K., Dimitriou, G., & Daras, P.,"Continuous Sign Language Recognition: A Survey," IEEE Access, vol. 9, pp. 118954–118982, 2021.

https://www.researchgate.net/publication/351963802_Continuous_Sign_Language_Recognition_A_Survey

[28] Ramanathan, M., & Patil, S. (2024). "Enhanced Sign Language Translation between American Sign Language (ASL) and Indian Sign Language (ISL) Using LLMs." arXiv preprint arXiv:2411.12685. https://arxiv.org/html/2411.12685v1

[29] Rastgoo, R., Kiani, K., & Escalera, S. (2020). "Video-Based Isolated Hand Sign Language Recognition Using a Deep Cascaded Model." Multimedia Tools and Applications, Springer, Vol. 79, Pp. 22965-22987. DOI: 10.1007/s11042-020-08987-x. https://link.springer.com/article/10.1007/s11042-020-08987-x

[30] Rautaray, S. S., & Agrawal, A. (2015). "Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey." Artificial Intelligence Review, Springer, Vol. 43, Pp. 1-54. DOI: 10.1007/s10462-012-9356-9. https://link.springer.com/article/10.1007/s10462-012-9356-9

[31] Rawat, P., Kumar, P., Tamta, V. K., & Kumar, A. (2025). "A Comprehensive Approach to Indian Sign Language Recognition: Leveraging LSTM and MediaPipe Holistic." EAI Endorsed Transactions on AI and Robotics, Vol. 4. DOI: 10.4108/airo.8693. https://publications.eai.eu/index.php/airo/article/view/8693

[32] Robles-Sanchez, J. A., et al. (2025). "Real-Time Sign Language Gestures to Speech Transcription using Deep Learning." arXiv preprint arXiv:2508.12713. https://arxiv.org/abs/2508.12713

[33] Rocha, A., Lopes, H., & Couto, F. (2024). "Sign Language Recognition Based on Deep Learning and Low-Cost Handcrafted Descriptors." arXiv preprint. arXiv:2408.07244. https://arxiv.org/html/2408.07244v1

[34] Saleh, M., & Issa, G. (2024). "Sign Language Interpretation Using Machine Learning and Artificial Intelligence." Neural Computing and Applications, Springer. DOI: 10.1007/s00521-024-10395-9. https://link.springer.com/article/10.1007/s00521-024-10395-9

[35] Shenoy, K., Dastane, T., Rao, V., & Vyavaharkar, D. (2021). "Real-time Indian Sign Language (ISL) Recognition." arXiv preprint arXiv:2108.10970. https://arxiv.org/abs/2108.10970

[36] Srivastava, S., Singh, R., Pandey, V., & Singh, S. (2024). "Continuous Sign Language Recognition System using Deep Learning with MediaPipe Holistic." arXiv preprint arXiv:2411.04517. https://arxiv.org/abs/2411.04517

[37] Sruthi, C. J., & Lijiya, A. (2019). "SigNet: A Deep Learning Based Indian Sign Language Recognition System." 2019 International Conference on Communication and Signal Processing (ICCSP). IEEE. DOI: 10.1109/ICCSP.2019.8697907. https://ieeexplore.ieee.org/document/8697907

[38] Svendsen, E., & Kadry, S. (2025). "Real-Time Norwegian Sign Language Recognition Using MediaPipe and LSTM." Multimodal Technologies and Interaction, MDPI, Vol. 9, No. 3, Article 23. DOI: 10.3390/mti9030023. https://www.mdpi.com/2414-4088/9/3/23

[39] Vashisth, H. K., Tarafder, T., Aziz, R., Arora, M., & Alpana. (2023). "Hand Gesture Recognition in Indian Sign Language Using Deep Learning." Engineering Proceedings, MDPI, Vol. 59, No. 1, Article 96. DOI: 10.3390/engproc2023059096. https://www.mdpi.com/2673-4591/59/1/96

[40] Vyavahare, P., Dhawale, S., Takale, P., Koli, V., Kanawade, B., & Khonde, S. (2023). "Detection and Interpretation of Indian Sign Language Using LSTM Networks." Journal of Intelligent Systems and Control, Vol. 2, No. 3, Pp. 132-142. DOI: 10.56578/jisc020302. https://www.acadlore.com/article/JISC/2023_2_3/jisc020302

[41] Wadhawan, A., & Kumar, P. (2020). "Deep Learning-Based Sign Language Recognition System for Static Signs." Neural Computing and Applications, Springer, Vol. 32, Pp. 7957-7968. DOI: 10.1007/s00521-019-04691-y. https://link.springer.com/article/10.1007/s00521-019-04691-y

[42] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C. L., & Grundmann, M. (2020). "MediaPipe Hands: On-device Real-time Hand Tracking." arXiv preprint arXiv:2006.10214. DOI: 10.48550/arXiv.2006.10214. https://arxiv.org/abs/2006.10214

CHAPTER 9 : APPENDICES

1. List of Tables

Table 1.1: Comparison of Existing Sign Language Systems 13
Table 1.2: Scope of the GestureTalk System 16
Table 3.1: Specific Objectives Aligned with Research Hypothesis Components 38
Table 3.2: GestureTalk Custom ISL Dataset – Summary Statistics 42
Table 3.3: GestureTalk LSTM Model Architecture 45
Table 3.4: GestureTalk Model Training Hyperparameter Configuration 47
Table 3.5: Programming Languages Used in GestureTalk 49
Table 3.6: Machine Learning and AI Libraries 49
Table 3.7: Computer Vision and Hand Tracking Libraries 50
Table 3.8: Web Framework and Communication API Libraries 50
Table 3.9: GestureTalk Flask REST API Endpoints (app.py) 51
Table 3.10: GestureTalk – Project Phase Timeline (Nov 2025 – Mar 2026) 55
Table 4.1: GestureTalk ISL Gesture Classes by Functional Category 62
Table 4.2: GestureTalk Dataset – Complete Structural Parameters 67
Table 4.3: MediaPipe Hand Keypoint Structure 72
Table 4.4: EDA Findings and Their Impact on GestureTalk Design Decisions 78
Table 5.1: GestureTalk Flask Route Reference 87
Table 6.1: GestureTalk LSTM Model – Overall Evaluation Metrics Summary 112
Table 6.2: GestureTalk LSTM – Per-Class Performance Metrics 116
Table 6.3: Comparison with Rule-Based and Feature-Engineering Approaches 120
Table 6.4: Comparison with Baseline Machine Learning Models 121
Table 6.5: GestureTalk – Functional Comparison Summary with Existing Approaches 122
Table 7.1: System Limitations and Their Implications for Deployment 128

2. List of Figures

Figure 1.1: GestureTalk System Pipeline 2
Figure 1.2: Evolution of Human-Computer Interaction 4
Figure 1.3: Evolution of Gesture Recognition Technology Across Four Phases 6
Figure 3.1: GestureTalk Research Methodology Flowchart 36
Figure 3.2: Feature Vector Composition – 126 Features per Frame. 40
Figure 3.3: Dataset Class Distribution 42
Figure 3.4: GestureTalk Model Training Pipeline 47
Figure 3.5: Gantt Chart 55
Figure 4.1: GestureTalk EDA Workflow 56
Figure 4.2: Role of EDA in Gesture Data Analysis 58
Figure 4.3: Effect of Wrist-Relative Landmark Normalization 69
Figure 4.4: MediaPipe Hands – 21 Anatomical Keypoints per Hand. 71
Figure 4.5: GestureTalk Feature Extraction Pipeline – 7 Stages 72
Figure 4.6: Sequence Count per Gesture Class 74
Figure 4.7: Functional Category Distribution 74
Figure 4.8: Landmark Feature Value Distributions 75
Figure 4.9: Temporal Motion Profiles 76
Figure 4.10: GestureTalk Dataset Summary Infographic 78

Figure 5.1: GestureTalk – Overall System Workflow 79
Figure 5.2: GestureTalk System Architecture (Frontend–Backend–ML Layers) 82
Figure 5.3: GestureTalk Frontend Architecture 83
Figure 5.4: GestureTalk Landing Page 85
Figure 5.5: GestureTalk LSTM Model Architecture 88
Figure 5.6: GestureTalk extract_landmarks() 89
Figure 5.7: GestureTalk Translation & Text-to-Speech Pipeline 90
Figure 5.8: GestureTalk LSTM Training Curves – Accuracy & Loss 96
Figure 5.9: GestureTalk ISL Dataset – Sequence Count per Gesture Class 96
Figure 6.1: GestureTalk LSTM Training History – Final Training Run 113
Figure 6.2: GestureTalk LSTM – Per-Class Precision, Recall, and F1-Score 114
Figure 6.3: GestureTalk LSTM Model – Confusion Matrix 117
Figure 6.4: GestureTalk LSTM – Summary Performance Dashboard 119
Figure 6.5: Accuracy Comparison with Existing ISL Recognition Approaches 123

3. ScreenShots

Screenshot 4.1: Hello 63
Screenshot 4.2: Sorry 63
Screenshot 4.3: Understand 64
Screenshot 4.4: Bathroom 64
Screenshot 4.5: Water 65
Screenshot 4.6: I 65
Screenshot 4.7: I Love You 66
Screenshot 4.8: Yes 66
Screenshot 5.1: Requirements.txt 91
Screenshot 5.2: MediaPipe Hand Landmark Extraction Funtion 92
Screenshot 5.3: Real-Time Gesture Inference Loop 1 92
Screenshot 5.4: Real-Time Gesture Inference Loop 2 93
Screenshot 5.5: Real-Time Gesture Inference Loop 3 93
Screenshot 5.6: LSTM Model Architecture 94
Screenshot 5.7: Model Training Loop 1 94
Screenshot 5.8: Model Training Loop 2 95
Screenshot 5.9: Translation and Speech API Routes 1 95
Screenshot 5.10: Translation and Speech API Routes 2 95
Screenshot 5.11: Flask 100
Screenshot 5.12: OpenCV 101
Screenshot 5.13: NumPy 102
Screenshot 5.14: MediaPipe 102
Screenshot 5.15: TensorFlow 103
Screenshot 5.16: gTTS 104
Screenshot 5.17: deep-translator 104
Screenshot 5.18: Pillow 105
Screenshot 6.1: Accuracy 109
Screenshot 6.2: Precision 110
Screenshot 6.3: Recall 110
Screenshot 6.4: F1-Score 111